MACHINE LEARNING: DEEP LEARNING

Eric Nalisnick
Johns Hopkins University

Last Update:

April 13, 2026

Contents

1 Supervised Learning with Univariate Linear Models **4**

1.1 Predictive Modeling 4
1.2 Univariate Linear Model for Real-Valued Responses 4
1.3 Maximum Likelihood Estimation: A General Recipe 7
1.4 Generalized Linear Models 10
1.5 Univariate Logistic Regression for Binary Labels 11
1.6 Gradient Descent 12
1.7 Models of Artificial Neurons & the Perceptron 14

2 Supervised Learning with Multiple Linear Regression **16**

2.1 Multiple Features and Feature Expansions 16
2.2 Revisiting Gradient Descent with Multivariate Derivatives 18
2.3 Multiple Output Dimensions 20

3 Model Evaluation, Model Selection, and Capacity Control **24**

3.1 Model Evaluation 24
3.1.1 Train-Validation-Test Splits 25
3.1.2 Example Evaluation Metrics 25
3.2 Model Selection 28
3.3 Capacity Control 29

4 Feedforward Neural Networks **32**

4.1 Adaptive Features and the Importance of Non-Linearities 32
4.2 Neural Networks 33
4.3 Deep Neural Networks 34
4.4 Gradient-Based Learning of Neural Networks 36
4.5 Backpropagation 37
4.5.1 Vectorized Implementation 39
4.5.2 Skip Connections 40
4.5.3 Initializations 41
4.5.4 Normalization Layers 42
4.6 Capacity Control for Deep Neural Networks 43
4.6.1 Data Augmentation (Capacity Control by Inductive Bias) 43
4.6.2 Weight Decay (a.k.a. $L2$ Regularization) 44
4.6.3 Dropout: Stochastic Capacity Control via Random Subnetworks . . . 44
4.6.4 Ensembling (Deep Ensembles and Prediction Averaging) 45

5 Stochastic, Adaptive Optimizers **45**

5.1 Mini-Batch Gradient Descent 46
5.2 Momentum 47
5.3 Adaptive Moment Estimation (Adam) 47
5.4 Practical Considerations 48

6 Convolutional Neural Networks **48**
6.1 Convolutional Layers 49
6.2 Pooling Layers 50
6.3 Backpropagation for Convolutions (Index Form) 51
6.4 Common CNN Design Patterns 51

7 Models for Sequential Data **52**
7.1 Recurrent Neural Networks 52
7.2 Overview of Architectures 59
7.3 Unaligned Sequence-to-Sequence with Encoder-Decoder Architecture 61

8 Attention & Transformers **62**
8.1 Attention 62
8.2 Encoder-Decoder Architecture with Attention 64
8.3 The Transformer 64
8.3.1 Encoder 65
8.3.2 Decoder 68

9 Autoencoders and Deep Generative Models **71**
9.1 Dimensionality Reduction with Autoencoders 72
9.2 Regularized Autoencoders 73
9.3 Contrastive Representation Learning 76
9.4 The Variational Autoencoder: a Probabilistic Autoencoder for Generative Modeling 79
9.5 Denoising Diffusion Models 80
9.6 Other Types of Neural Generative Models 84

1 Supervised Learning with Univariate Linear Models

The first topic we will discuss is predictive modeling using linear models—that is, models that are linear in their parameters. This will provide the building blocks with need to eventually stack these 'shallow' models into the 'deep' models that give this course its title.

1.1 Predictive Modeling

Consider the task of *predictive modeling*. Imagine that we are creating a system that, given a medical image, can predict if the patient has a particular disease, e.g. pneumonia. Such a system will be used by bringing patients into the clinic to perform the imaging, and then once the image is taken, the image will be passed to some sort of predictive model, that will generate the prediction that the radiologist will consider to inform their diagnosis. Let $\mathcal{X}$ denote a feature space, which in the example above, is the space of all valid medical images. You can think of this as a matrix in which entries are the pixel values, intensities, or some other property of the image. Let $\mathcal{Y}$ denote the label / response space, which in the above setting is a discrete encoding of the potential diseases. Our goal is to design some model $\hat{\mathrm{y}} = f(\mathbf{x})$ that takes features $\mathbf{x} \in \mathcal{X}$ as input and outputs an accurate prediction $\hat{y} \in \mathcal{Y}$.

Data Generating Process Ideally, we want the above model $f(\mathbf{x})$ to match the true underlying process that generated the data. In the medical imaging example, this means that $f(\mathbf{x})$ would faithfully capture whatever is the underlying medical process that results in a person, with that given image, having the biological conditions that present as their true clinical diagnosis. Mathematically, we can say that the world generates these diseases according to a distribution $\mathbb{P}(\mathrm{y}|\mathbf{x})$, and thus the goal of predictive modeling is to have $f(\mathbf{x}) = \mathbb{P}(\mathrm{y}|\mathbf{x})$. Although, in practice, we are often satisfied with a close approximation.

Training Data As we will see below, we will construct $f(\mathbf{x})$ in a data-driven way. That is, instead of just hand-engineering rules or some other function for $f(\mathbf{x})$, we will *learn* a good predictive model from *data* that represents or encodes our problem of interest. Ideally, we would like to know and work with $\mathbb{P}(\mathrm{y}|\mathbf{x})$ directly, but this is usually never the case in practice. And if we did have access to $\mathbb{P}(\mathrm{y}|\mathbf{x})$, why then would we need to train a model $f(\mathbf{x})$? In practice, we usually just have samples from $\mathbb{P}(\mathrm{y}|\mathbf{x})$, i.e. $\mathrm{y} \sim \mathbb{P}(\mathrm{y}|\mathbf{x})$. For the features $\mathbf{x}$, we will also assume we have samples from another underlying generative process $\mathbf{x} \sim \mathbb{P}(\mathbf{x})$. One could try to model $\mathbb{P}(\mathbf{x})$ in addition to $\mathbb{P}(\mathrm{y}|\mathbf{x})$; this is usually called *generative modeling*, a topic we will get into later in the course. We will assume that, for purposes of training data, we are able to collect N samples of feature-label pairs, making our N-element training set $\mathcal{D} = \{(\boldsymbol{x}_n, y_n)\}_{n=1}^{N}$.

1.2 Univariate Linear Model for Real-Valued Responses

We will now get into our first (or many) concrete instantiations of $f(\mathbf{x})$, and we will start with a (seemingly) simple function: the line, with one slope parameter. Assume for the time being that the features $\mathrm{x} \in \mathbb{R}$ and $\mathrm{y} \in \mathbb{R}$ are both real-valued, unconstrained scalar variables. We will define the *univariate linear model* as $f(\mathrm{x}; \mathrm{w}) \triangleq \mathrm{w} \cdot \mathrm{x}$, where x is a scalar feature value and w is a scalar parameter that we wish to learn from data. To pick apart

the notation, $f(\mathrm{x};\mathrm{w})$ means that we have a function of the features x and the function is determined by parameters w. This model encodes a very simple predictive relationship: the prediction $\hat{y}$ is proportional or inversely proportional to the feature value x.

Loss Function Given an N-sample training set $\mathcal{D}$ and the linear model $f(\mathrm{x};\mathrm{w})$, the next step is to fit the model to the data. An intuitive way to do this is to define a *loss function* that quantifies how far off the model's predictions are from the observed data. While we will later given a complete recipe for deriving loss functions, one natural choice for real-valued, unconstrained data is the squared loss: $\ell\left(f;x,y\right)=\left(f(\mathrm{x})-y\right)^2$. Clearly, this will be zero when $f(\mathrm{x})=y$ and grow quadratically as $f(x)$ makes worse and worse predictions. Also notice that this loss doesn't care if the predictions under or over estimate y, which could be inappropriate for some applications. For example, in the American game show *The Price is Right*, contestants had to guess the sale price of items, and if they overestimated the price, they instantly lost. If you were building an AI agent to play The Price is Right, you would certainly want to train it with a loss function that treats over- and under- estimates differently. Now that we have devised a loss for one data point, we can compute the loss over the full training set by summing the losses for each data point:

$$\ell(\mathrm{w};\mathcal{D}) \;=\; \frac{1}{N}\sum_{n=1}^{N}\ell(\mathrm{w};x_n,y_n) \;=\; \frac{1}{N}\sum_{n=1}^{N}\left(f(x_n;\mathrm{w})-y_n\right)^2 \;=\; \frac{1}{N}\sum_{n=1}^{N}\left(\mathrm{w}\cdot x_n-y_n\right)^2. \quad (1)$$

Note that these loss functions are a function of *the model*, with the data treated as a constant, because we want to assess how well the model fits the data and not vice versa.

Optimizing a Loss Function Now that we have defined a loss function, we want to use it to find the best setting of the parameter w. This boils down to the following optimization problem:

$$\begin{aligned} w^* &= \arg\min_{\mathrm{w}}\ \ell(\mathrm{w};\mathcal{D}) \\ &= \arg\min_{\mathrm{w}}\ \frac{1}{N}\sum_{n=1}^{N}\left(f(x_n;\mathrm{w})-y_n\right)^2 \\ &= \arg\min_{\mathrm{w}}\ \frac{1}{N}\sum_{n=1}^{N}\left(\mathrm{w}\cdot x_n-y_n\right)^2. \end{aligned} \quad (2)$$

Thus, w^* will be the parameter that minimizes the squared distance between the model predictions and the training responses y. It is unlikely the value of the loss will be exactly zero when computed using $f(x_n;w^*)$, so when we speak of 'minimizing the loss,' it is constrained by the best training performance achievable under the fixed model class—which in this case, is the univariate linear model. The loss function might be able to be driven to exactly zero if we were to choose a different model, especially one that can represent more flexible functions than a line.

Now how should we find the exact form of w^*. Fortunately, for linear models, we can do this exactly and in 'closed form,' meaning that we can get an explicit equation for w^*. This will not be the case for most of the course, and we'll often have to resort to approximate, numerical techniques. Yet, in all cases, we will reply upon tools from calculus.

Recall that the points at which a derivative equals zero represent the *critical points* of a function, meaning that that point can be a maxima, minima, or saddle point. For this linear model with the squared loss, fortunately there is just one (non-trivial) critical point and it represents the global minimum. While a proper course on optimization would go into the details of validating this claim, we will mostly ignore these details since deep learning methodologies often need to reply upon so many approximations that such proofs are not that informative of practice.

Moving on to the mechanics of taking the derivative of the loss with respect to the model parameter, we have:

$$\begin{aligned}
\frac{d}{d\mathrm{w}}\ell(\mathrm{w};\mathcal{D}) &= \frac{d}{d\mathrm{w}}\left[\frac{1}{N}\sum_{n=1}^{N}\left(\mathrm{w}\cdot x_n - y_n\right)^2\right] \\
&= \frac{1}{N}\sum_{n=1}^{N}\frac{d}{d\mathrm{w}}\left[\left(\mathrm{w}\cdot x_n - y_n\right)^2\right] \\
&= \frac{1}{N}\sum_{n=1}^{N}2\cdot\left(\mathrm{w}\cdot x_n - y_n\right)\cdot\frac{d}{d\mathrm{w}}\left[\mathrm{w}\cdot x_n - y_n\right] \\
&= \frac{1}{N}\sum_{n=1}^{N}2\cdot\left(\mathrm{w}\cdot x_n - y_n\right)\cdot x_n \\
&= \frac{2}{N}\left\{\left(\sum_{n=1}^{N}\mathrm{w}\cdot x_n^2\right) - \left(\sum_{n=1}^{N}y_n\cdot x_n\right)\right\}.
\end{aligned} \tag{3}$$

Now we can find w^* by setting the derivative to zero and solving for w:

$$\begin{aligned}
0 &= \frac{d}{d\mathrm{w}}\ell(\mathrm{w};\mathcal{D}) = \frac{2}{N}\left\{\left(\sum_{n=1}^{N}\mathrm{w}\cdot x_n^2\right) - \left(\sum_{n=1}^{N}y_n\cdot x_n\right)\right\} \\
\implies 0 &= \left(\sum_{n=1}^{N}\mathrm{w}\cdot x_n^2\right) - \left(\sum_{n=1}^{N}y_n\cdot x_n\right) \\
\implies \sum_{n=1}^{N}\mathrm{w}\cdot x_n^2 &= \sum_{n=1}^{N}y_n\cdot x_n \\
\implies \mathrm{w} &= \frac{\sum_{n=1}^{N}y_n\cdot x_n}{\sum_{n=1}^{N}x_n^2} \triangleq w^*.
\end{aligned} \tag{4}$$

We have finally arrived at the 'trained' version of our model: computing $\sum_{n=1}^{N}y_n\cdot x_n / \sum_{n=1}^{N}x_n^2$ will give the value that we should plug in for the optimal parameter w^*.

Vectorized Version The *Graphics processing unit* (GPU) and linear algebra libraries of a modern computers make *vectorized* implementations much faster—i.e. writing your computations as vector or matrix products will make your code much faster than using for-loops. We can do this for the simple linear model above as follows. Firstly, regarding the data, we can write the collection of N features as $\boldsymbol{x} = [x_1, \ldots x_N]^T$, and similarly, the responses as $\boldsymbol{y} = [y_1, \ldots y_N]^T$. Now the vectorized form of the loss function is:

$$\ell(\mathrm{w};\mathcal{D}) = \frac{1}{N}\ ||\ \mathrm{w}\cdot\boldsymbol{x} - \boldsymbol{y}\ ||_2^2 \tag{5}$$

where $|| \cdot ||_2^2$ is the squared (Euclidean) two norm. Following the same derivation as above but keeping the vector notation, the optimal setting of the weights can then be written in vectorized form as: $w^* = \left(\boldsymbol{y}^T \boldsymbol{x}\right) / \left(\boldsymbol{x}^T \boldsymbol{x}\right)$.

1.3 Maximum Likelihood Estimation: A General Recipe

While sensible, the above procedure we used for finding w^* could still seem arbitrary and unsound. For example, recalling that the goal of predictive modeling is to capture $\mathbb{P}(\mathrm{y}|\mathbf{x})$, how does what we did relate to $\mathbb{P}(\mathrm{y}|\mathbf{x})$? Moreover, are they other choices than the squared loss function? We will now give a general procedure for deriving optimization objectives known as *maximum likelihood estimation*.

Statistical Divergences Yet before introducing maximum likelihood estimation, we need to visit the concept of a statistical *divergence*. A divergence is like a loss function but applied to probability distributions. The most commonly employed divergence is the *Kullback–Leibler divergence* (KLD):

$$\mathbb{KLD}[p(\mathrm{z})||q(\mathrm{z})] \triangleq \mathbb{E}_{p(\mathrm{z})}\left[\log \frac{p(\mathrm{z})}{q(\mathrm{z})}\right] = \int_{\mathrm{z}} p(\mathrm{z}) \left(\log \frac{p(\mathrm{z})}{q(\mathrm{z})}\right) dz,$$

where z is the random variable of interest, and we want to compare two distributions over z: $p(\mathrm{z})$ vs $q(\mathrm{z})$. The KLD is an information theoretic quantity that represents the number of bits lost when $q(\mathrm{z})$ is used to approximate $p(\mathrm{z})$. This means that the KLD is *not* symmetric: $\mathbb{KLD}[p(\mathrm{z})||q(\mathrm{z})]$ does not necessarily equal $\mathbb{KLD}[q(\mathrm{z})||p(\mathrm{z})]$, thus making the order of the arguments important. However, no matter the order of the arguments, the KLD will be exactly zero when $p(\mathrm{z}) = q(\mathrm{z})$:

$$\mathbb{KLD}[p(\mathrm{z})||p(\mathrm{z})] = \mathbb{E}_{p(\mathrm{z})}\left[\log \frac{p(\mathrm{z})}{p(\mathrm{z})}\right] = \mathbb{E}_{p(\mathrm{z})}\left[\log 1\right] = \log 1 = 0.$$

There are other divergences, such as the (squared) Hellinger divergence:

$$\mathcal{H}^2[p(\mathrm{z})||q(\mathrm{z})] \triangleq 1 - \int_{\mathrm{z}} \sqrt{p(\mathrm{z}) \cdot q(\mathrm{z})}\, dz.$$

The Hellinger divergence is symmetric, but unfortunately, it is less commonly employed due to it having an integral that is usually more difficult to evaluate. Both the Hellinger and KLD are members of the family of f-divergences.

Models as Probability Distributions Previously, we defined the model just as a generic function $f(\mathrm{x})$. Now we will be more particular interpreting $f(\mathrm{x})$, embedding it within a distribution function. This will, firstly, allow us to give a probabilistic interpretation to the model itself, unlocking operations such as marginalization, sampling, etc. Secondly, it will allow us to apply a statistical divergence to the model, directly quantifying the gap between the model and the true generative process $\mathbb{P}(\mathrm{y}|\mathrm{x})$. To do this, we will pick a probability distribution $p(\mathrm{y};\theta)$, where y still denotes the label and θ denotes the parameter(s). For example, for the normal distribution $\theta = \{\mu, \sigma\}$, the mean and the standard deviation respectively. Since in our running example $\mathrm{y} \in \mathbb{R}$, technically we can pick any distribution

with support over all the real numbers—e.g. normal, Laplace, student-t, etc.—but each comes with its own probabilistic assumptions. For example, the student-t distribution has heavier tails than the normal distribution, meaning that building a model with the student-t assumes that we will see more outlying points.

We will consider the general construction again in a later section. For now, let's assume that we choose $p(\mathrm{y};\theta)$ to be a normal distribution. Now taking our linear model $f(\mathrm{x}) = \mathrm{w}\cdot\mathrm{x}$, we will use this model to parameterize just the mean μ:

$$\begin{aligned} p\left(\mathrm{y}; f(\mathrm{x})\right) &\triangleq \text{Normal}\left(\mathrm{y}; \mu = f(\mathrm{x}), \sigma\right) \\ &= \frac{1}{\sigma\sqrt{2\pi}} \exp\left\{-\frac{(\mu-\mathrm{y})^2}{2\sigma^2}\right\} \\ &= \frac{1}{\sigma\sqrt{2\pi}} \exp\left\{-\frac{(f(\mathrm{x})-\mathrm{y})^2}{2\sigma^2}\right\} \\ &= \frac{1}{\sigma\sqrt{2\pi}} \exp\left\{-\frac{(\mathrm{w}\cdot\mathrm{x}-\mathrm{y})^2}{2\sigma^2}\right\}. \end{aligned} \tag{6}$$

The parameter σ will also have to be set somehow. We could set it with the same function, tying the mean and standard deviation, e.g. $\sigma = \exp\{f(\mathrm{x})\}$, where the $\exp\{\cdot\}$ ensures that the standard deviation is positive. However, this would mean that as the mean increases, so does the standard deviation, which is an assumption that would be inappropriate for many applications. Alternatively, we could set the standard deviation via a second linear model: $\sigma = \exp\{f'(\mathrm{x})\} = \exp\{\mathrm{u}\cdot\mathrm{x}\}$, where $\mathrm{u}\in\mathbb{R}$ is another parameter that defines this second linear model. In the statistics literature, regression models that have a σ that is constant w.r.t. x are called *homoskedastic*. If σ varies with x, then the model is called *heteroskedastic*. Both of the cases above, where $\sigma = \exp\{f(\mathrm{x})\}$ or $\sigma = \exp\{f'(\mathrm{x})\}$, are heteroskedastic. In this course, for simplicity, we will often assume the models are homoskedastic.

Divergence as an Optimization Objective We now have the pieces in place to derive the maximum likelihood estimation procedure—the standard procedure we will use to train models throughout the course. Maximum likelihood estimation means that we seek to minimize the KLD between the true generative distribution and our probabilistic predictive model:

$$\begin{aligned} \mathbb{KLD}\left[\ \mathbb{P}(\mathrm{y}|\mathrm{x}) \ ||\ p(\mathrm{y}; f(\mathrm{x}))\ \right] &= \mathbb{E}_{\mathbb{P}(\mathrm{y}|\mathrm{x})}\left[\log\frac{\mathbb{P}(\mathrm{y}|\mathrm{x})}{p(\mathrm{y}; f(\mathrm{x}))}\right] \\ &= \underbrace{\mathbb{E}_{\mathbb{P}(\mathrm{y}|\mathrm{x})}\left[\log\mathbb{P}(\mathrm{y}|\mathrm{x})\right]}_{-\mathbb{H}[\mathbb{P}(\mathrm{y}|\mathrm{x})]} - \mathbb{E}_{\mathbb{P}(\mathrm{y}|\mathrm{x})}\left[\log p(\mathrm{y}; f(\mathrm{x}))\right] \\ &= \mathbb{E}_{\mathbb{P}(\mathrm{y}|\mathrm{x})}\left[-\log p(\mathrm{y}; f(\mathrm{x}))\right] \quad - \quad \underbrace{\mathbb{H}\left[\mathbb{P}(\mathrm{y}|\mathrm{x})\right]}_{\text{constant w.r.t. } p(\mathrm{y};f(\mathrm{x}))} \end{aligned} \tag{7}$$

where $\mathbb{H}[p(\mathrm{x})] = \int_{\mathrm{x}} p(\mathrm{x})(-\log p(\mathrm{x}))d\mathrm{x}$ denotes the differential entropy of the distribution $p(\mathrm{x})$. $\mathbb{H}\left[\mathbb{P}(\mathrm{y}|\mathrm{x})\right]$ denotes the entropy of the true generating process $\mathbb{P}(\mathrm{y}|\mathrm{x})$, and thus it does not involve the model. This matters because we will eventually optimize this KLD w.r.t. $f(\mathrm{x})$, and in turn, terms that are not a function of $f(\mathrm{x})$ 'fall out of' the optimization problem. We can see this explicitly when we take the derivative w.r.t. the model parameters, since $\frac{d}{d\mathrm{w}}\mathbb{H}\left[\mathbb{P}(\mathrm{y}|\mathrm{x})\right] = 0$.

Yet, notice that Equation 7 involves a particular value of the features x. Or in other words, both the model and true distribution are conditioned on a particular feature value, and we are evaluating the KLD only at those features. Of course, in the training data, we have multiple feature observations. This means that there is another distribution to be concerned with: the distribution that generates the features, $\mathbb{P}(\mathrm{x})$. We do not model this distribution directly. Rather, we will incorporate it into the maximum likelihood formulation by adding an outer expectation over $\mathbb{P}(\mathrm{x})$:

$$\mathbb{E}_{\mathbb{P}(\mathrm{x})}\mathbb{KLD}\left[\ \mathbb{P}(\mathrm{y}|\mathrm{x})\ ||\ p(\mathrm{y};f(\mathrm{x}))\ \right]\ =\ \mathbb{E}_{\mathbb{P}(\mathrm{x})}\mathbb{E}_{\mathbb{P}(\mathrm{y}|\mathrm{x})}\left[-\log p(\mathrm{y};f(\mathrm{x}))\right]-\mathbb{E}_{\mathbb{P}(\mathrm{x})}\left[\mathbb{H}\left[\mathbb{P}(\mathrm{y}|\mathrm{x})\right]\right]$$

Putting everything together and taking the model parameter to again be w, we have the complete maximum likelihood optimization problem:

$$\begin{aligned} w^* &= \arg\min_{\mathrm{w}}\ \mathbb{E}_{\mathbb{P}(\mathrm{x})}\mathbb{KLD}\left[\ \mathbb{P}(\mathrm{y}|\mathrm{x})\ ||\ p(\mathrm{y};f(\mathrm{x};\mathrm{w}))\ \right] \\ &= \arg\min_{\mathrm{w}}\ \mathbb{E}_{\mathbb{P}(\mathrm{x})}\mathbb{E}_{\mathbb{P}(\mathrm{y}|\mathrm{x})}\left[-\log p(\mathrm{y};f(\mathrm{x};\mathrm{w}))\right]\ -\ \mathbb{E}_{\mathbb{P}(\mathrm{x})}\left[\mathbb{H}\left[\mathbb{P}(\mathrm{y}|\mathrm{x})\right]\right] \\ &= \arg\min_{\mathrm{w}}\ \mathbb{E}_{\mathbb{P}(\mathrm{x})}\mathbb{E}_{\mathbb{P}(\mathrm{y}|\mathrm{x})}\left[-\log p(\mathrm{y};f(\mathrm{x};\mathrm{w}))\right] \end{aligned} \tag{8}$$

where, again, the entropy term drops out because it does not involve the model and in turn, the parameter w that we are optimizing.

Monte Carlo Approximation There is one remaining obstacle to solving the optimization problem in Equation 8. It requires taking the expectation w.r.t. the true generative process, $\mathbb{P}(\mathrm{y},\mathrm{x})$. As stated above, we do not have access to this distribution except through the samples that constitute our training data: $\mathrm{y}\sim\mathbb{P}(\mathrm{y}|\mathrm{x})$ and $\mathrm{x}\sim\mathbb{P}(\mathrm{x})$. Fortunately, this allows us to compute what's called a *Monte Carlo* (MC) approximation of the expectation; for a generic distribution $P(\mathrm{x})$ and a function of the random variable $\phi(\mathrm{x})$, this is:

$$\mathbb{E}_{P(\mathrm{x})}\left[\phi(\mathrm{x})\right]\ \approx\ \frac{1}{S}\sum_{s=1}^{S}\phi(x_s),\quad x_s\sim P(\mathrm{x}),$$

where $S\in\mathbb{N}^+$ is the number of samples. While this approximation is valid for any number of samples (above zero), the approximation becomes better and better as $S\to\infty$, becoming exact only asymptotically. Applying the MC expectation to Equation 8, we have:

$$\begin{aligned} w^* &= \arg\min_{\mathrm{w}}\ \mathbb{E}_{\mathbb{P}(\mathrm{x})}\mathbb{E}_{\mathbb{P}(\mathrm{y}|\mathrm{x})}\left[-\log p(\mathrm{y};f(\mathrm{x};\mathrm{w}))\right] \\ &\approx \arg\min_{\mathrm{w}}\ \frac{1}{N}\sum_{n=1}^{N}-\log p\left(y_n;f(x_n;\mathrm{w})\right) \end{aligned} \tag{9}$$

where the sum is over the training samples $\{(x_n,y_n)\}_{n=1}^N$. It follows that, the larger training set we have, the better we should be approximating our true optimization target, $\mathbb{E}_{\mathbb{P}(\mathrm{x})}\mathbb{KLD}\left[\ \mathbb{P}(\mathrm{y}|\mathrm{x})\ ||\ p(\mathrm{y};f(\mathrm{x};\mathrm{w}))\ \right]$.

Deriving the Squared Loss Function We will now work through an end-to-end derivation and eventually arrive at the same loss function used in Equation 2. Keeping with the same setup, we assume $f(\mathrm{x};\mathrm{w})=\mathrm{w}\cdot\mathrm{x}$ and $p(\mathrm{y};f(\mathrm{x};w))=\mathrm{N}(\mathrm{y};\mu=f(\mathrm{x};w),\sigma=1)$, where

the normal distribution's variance is fixed at one (i.e. the homoskedastic assumption). The final optimization problem is then:

$$
\begin{aligned}
w^* &= \arg\min_{\mathrm{w}} \; \mathbb{E}_{\mathbb{P}(\mathrm{x})}\mathbb{KLD}\left[\; \mathbb{P}(\mathrm{y}|\mathrm{x}) \;||\; p(\mathrm{y}; f(\mathrm{x};\mathrm{w}))\;\right] \\
&= \arg\min_{\mathrm{w}} \; \mathbb{E}_{\mathbb{P}(\mathrm{x})}\mathbb{E}_{\mathbb{P}(\mathrm{y}|\mathrm{x})}\left[-\log p(\mathrm{y}; f(\mathrm{x};\mathrm{w})\right] \quad \text{(drop entropy term)} \\
&\approx \arg\min_{\mathrm{w}} \; \frac{1}{N}\sum_{n=1}^{N} -\log p\left(y_n; f(x_n;\mathrm{w})\right) \quad \text{(Monte Carlo approximation)} \\
&= \arg\min_{\mathrm{w}} \; \frac{1}{N}\sum_{n=1}^{N} -\log \mathrm{N}(y_n; \mu_n = f(x_n;\mathrm{w}), \sigma = 1) \\
&= \arg\min_{\mathrm{w}} \; \frac{1}{N}\sum_{n=1}^{N} -\log\left\{\frac{1}{\sigma\sqrt{2\pi}}\;\exp\left\{-\frac{(\mu_n - y_n)^2}{2\sigma^2}\right\}\right\} \\
&= \arg\min_{\mathrm{w}} \; \frac{1}{N}\sum_{n=1}^{N} -\log\left\{\frac{1}{\sqrt{2\pi}}\;\exp\left\{-\frac{(f(x_n;\mathrm{w}) - y_n)^2}{2}\right\}\right\} \qquad (10) \\
&= \arg\min_{\mathrm{w}} \; \frac{1}{N}\sum_{n=1}^{N} -\log\exp\left\{-\frac{(f(x_n;\mathrm{w}) - y_n)^2}{2}\right\} + \log\left\{\sqrt{2\pi}\right\} \\
&= \arg\min_{\mathrm{w}} \; \frac{1}{N}\sum_{n=1}^{N} \frac{1}{2}(f(x_n;\mathrm{w}) - y_n)^2 + \log\left\{\sqrt{2\pi}\right\} \\
&= \arg\min_{\mathrm{w}} \; \frac{1}{2}\frac{1}{N}\sum_{n=1}^{N}(f(x_n;\mathrm{w}) - y_n)^2 \quad \text{(drop } \sqrt{2\pi} \text{ constant)} \\
&= \arg\min_{\mathrm{w}} \; \frac{1}{2}\frac{1}{N}\sum_{n=1}^{N}(\mathrm{w}\cdot x_n - y_n)^2
\end{aligned}
$$

where the $\log\left\{\sqrt{2\pi}\right\}$ term is dropped because it does not depend on the parameters w. Thus, the maximum likelihood perspective gives us a more principled justification for use of the squared loss function as well as making its probabilistic assumptions explicit. If we were optimizing a heteroskedastic model w.r.t. the parameters controlling the variance, then this term would not drop from the optimization problem. The last line above is nearly the same as the final form of Equation 2 except for the constant 1/2. Yet the presence of this constant does not change the solution, as the *maximum likelihood estimator* (MLE) for w^* is still the same as derived in Equation 4. In fact, the derivative becomes a bit simpler as the 1/2 cancels the factor of 2 that is introduced when applying the power rule.

1.4 Generalized Linear Models

Above we discussed how a predictive model can be thought of as a probability distribution, with a specific function determining the mean variable. Specifically, we chose $p(\mathrm{y}; f(\mathrm{x})) = \mathrm{N}(\mathrm{y}; \mu = f(\mathrm{x}), \sigma)$. This flexible, modular framework allows us to construct models that meet our constraints or assumptions for the problem at hand. We call a linear model of the following form a *generalized linear model* (GLM):

$$
\mathbb{E}_p[\mathrm{y}|\mathrm{x}] \;=\; g^{-1}\left(f(\mathrm{x};\mathrm{w})\right) \;=\; g^{-1}\left(\mathrm{w}\cdot\mathrm{x}\right) \qquad (11)
$$

where $f(\mathrm{x};\mathrm{w}) = \mathrm{w}\cdot\mathrm{x}$, the linear model, and $g(\cdot)$ is known as the *link function*. In turn, $g^{-1}(\cdot)$ is called the *inverse link function*. Firstly, notice that in our running example of $p(\mathrm{y};f(\mathrm{x})) = \mathrm{N}(\mathrm{y};\mu = f(\mathrm{x}),\sigma)$, there is no g function since $\mu = f(\mathrm{x})$. Or to put it precisely, the link function is simply the identity function. This works in this case since the valid values of μ match the range of $f(\mathrm{x})$, namely all real numbers $\mathbb{R}$. But this will not be the case for all models of interest. Consider binary responses $\mathrm{y}\in\{0,1\}$. A common distribution with support over binary vales is the *Bernoulli* distribution: $p(\mathrm{y};\pi) = \pi^{\mathrm{y}}\cdot(1-\pi)^{1-\mathrm{y}}$, where $\pi\in[0,1]$ is the mean parameter. We cannot set $\pi = \mathrm{w}\cdot\mathrm{x}$ in this case since this would mean $\pi\in(-\infty,\infty)$, breaking the definition of the Bernoulli distribution. To fix this issue, we need to choose a g^{-1} function that transforms the output of $f(\mathrm{x})$ onto $(0,1)$. We will examine one popular implementation for the Bernoulli case in the next section. If $\mathrm{y}\in\mathbb{N}_{\geq 0}$, the non-negative natural numbers, the Poisson distribution is a commonly employed distribution with this support: $p(\mathrm{y};\lambda) = \lambda^{\mathrm{y}}e^{-\lambda}/\mathrm{y}!$, where $\lambda\in(0,\infty)$. In this case, it is common to choose the inverse link g^{-1} as the exponential function: $\lambda = \exp\{f(\mathrm{x})\}$.

Maximum Likelihood Derivative For the GLM under the maximum likelihood objective, we can write a general form for the chain rule derivative since it will always have the same four components:

$$\frac{d}{d\mathrm{w}}\mathbb{E}_{\mathbb{P}(\mathrm{x})}\mathbb{KLD}\left[\ \mathbb{P}(\mathrm{y}|\mathrm{x})\ ||\ p\left(\mathrm{y};g^{-1}\circ f(\mathrm{x};\mathrm{w})\right)\ \right] = \left(\frac{d}{dp}\mathbb{E}_{\mathbb{P}(\mathrm{x})}\mathbb{KLD}\left[\ \mathbb{P}(\mathrm{y}|\mathrm{x})\ ||\ p\left(\mathrm{y}\right)\right)\ ]\right)\left(\frac{d}{dg^{-1}}p\left(\mathrm{y};g^{-1}\right)\right)\left(\frac{d}{df}g^{-1}(f)\right)\left(\frac{d}{d\mathrm{w}}f(\mathrm{x};\mathrm{w})\right).$$

While for linear models $\frac{d}{d\mathrm{w}}f(\mathrm{x};\mathrm{w})$ is usually easy to evaluate, this will be the most computationally intensive term for deep learning models.

1.5 Univariate Logistic Regression for Binary Labels

We will now examine a particular GLM mentioned in the preceding section—namely, a model for binary labels known as *logistic regression*. Assuming $\mathrm{y}\in\{0,1\}$, we can define the following predictive model:

$$\begin{aligned} p\left(\mathrm{y};f(\mathrm{x};\mathrm{w})\right) &= \mathrm{Bernoulli}\left(\mathrm{y};\pi = g^{-1}(f(\mathrm{x};\mathrm{w}))\right) \\ &= \pi^{\mathrm{y}}\cdot(1-\pi)^{1-\mathrm{y}} \\ &= g^{-1}(f(\mathrm{x};\mathrm{w}))^{\mathrm{y}}\cdot(1-g^{-1}(f(\mathrm{x};\mathrm{w})))^{1-\mathrm{y}} \\ &= g^{-1}(\mathrm{w}\cdot\mathrm{x})^{\mathrm{y}}\cdot(1-g^{-1}(\mathrm{w}\cdot\mathrm{x}))^{1-\mathrm{y}}. \end{aligned} \tag{12}$$

Now we need to select the form of g^{-1} such that $g^{-1}:\mathbb{R}\mapsto(0,1)$. The most common choice of g^{-1} is the *logistic function*, which is where the name *logistic regression* originates:

$$\texttt{logistic}\left(\mathrm{z}\right) \triangleq \frac{1}{1+\exp\left\{-\mathrm{z}\right\}}.$$

We call this the *logistic function* because it is the cummulative distribution function of the standard logistic distribution ('standard' meaning location 0, scale 1). Plugging this into

the above expressions gives us our final form of logistic regression:

$$\begin{aligned} p\left(\mathrm{y}; f(\mathrm{x};\mathrm{w})\right) &= \text{Bernoulli}\left(\mathrm{y}; \pi = \texttt{logistic}(f(\mathrm{x};\mathrm{w}))\right) \\ &= \texttt{logistic}(f(\mathrm{x};\mathrm{w}))^{\mathrm{y}} \cdot \left(1 - \texttt{logistic}(f(\mathrm{x};\mathrm{w}))\right)^{1-\mathrm{y}} \\ &= \left(\frac{1}{1+\exp\left\{-f(\mathrm{x};\mathrm{w})\right\}}\right)^{\mathrm{y}} \cdot \left(1 - \frac{1}{1+\exp\left\{-f(\mathrm{x};\mathrm{w})\right\}}\right)^{1-\mathrm{y}} \\ &= \left(\frac{1}{1+\exp\left\{-\mathrm{w}\cdot\mathrm{x}\right\}}\right)^{\mathrm{y}} \cdot \left(1 - \frac{1}{1+\exp\left\{-\mathrm{w}\cdot\mathrm{x}\right\}}\right)^{1-\mathrm{y}}. \end{aligned} \tag{13}$$

Plugging this model into the maximum likelihood objective, we have:

$$\begin{aligned} w^* &= \arg\min_{\mathrm{w}} \; \mathbb{E}_{\mathbb{P}(\mathrm{x})} \mathbb{KLD}\left[\, \mathbb{P}(\mathrm{y}|\mathrm{x}) \;||\; \text{Bernoulli}\left(\mathrm{y}; \pi = \texttt{logistic}(f(\mathrm{x};\mathrm{w}))\right) \,\right] \\ &= \arg\min_{\mathrm{w}} \; \mathbb{E}_{\mathbb{P}(\mathrm{x})} \mathbb{E}_{\mathbb{P}(\mathrm{y}|\mathrm{x})} \left[-\log \text{Bernoulli}\left(\mathrm{y}; \pi = \texttt{logistic}(f(\mathrm{x};\mathrm{w}))\right)\right] \quad \text{(drop entropy term)} \\ &\approx \arg\min_{\mathrm{w}} \; \frac{1}{N}\sum_{n=1}^{N} -\log \text{Bernoulli}\left(y_n; \pi = \texttt{logistic}(f(x_n;\mathrm{w}))\right) \quad \text{(Monte Carlo approximation)} \\ &= \arg\min_{\mathrm{w}} \; \frac{1}{N}\sum_{n=1}^{N} -\log \left\{\texttt{logistic}(f(x_n;\mathrm{w}))^{y_n} \cdot \left(1 - \texttt{logistic}(f(x_n;\mathrm{w}))\right)^{1-y_n}\right\} \\ &= \arg\min_{\mathrm{w}} \; \frac{1}{N}\sum_{n=1}^{N} -y_n \log \texttt{logistic}(f(x_n;\mathrm{w})) \;-\; (1-y_n)\log\left(1 - \texttt{logistic}(f(x_n;\mathrm{w}))\right) \\ &= \arg\min_{\mathrm{w}} \; \frac{1}{N}\sum_{n=1}^{N} -y_n \log \texttt{logistic}(\mathrm{w}\cdot x_n) \;-\; (1-y_n)\log\left(1 - \texttt{logistic}(\mathrm{w}\cdot x_n)\right) \\ &= \arg\min_{\mathrm{w}} \; \frac{1}{N}\sum_{n=1}^{N} -y_n \log \left(\frac{1}{1+\exp\left\{-\mathrm{w}\cdot\mathrm{x}\right\}}\right) \;-\; (1-y_n)\log\left(1 - \left(\frac{1}{1+\exp\left\{-\mathrm{w}\cdot\mathrm{x}\right\}}\right)\right). \end{aligned}$$

This does not have as intuitive a form as the squared error loss function we examined earlier, but this expression is known as the *binary cross-entropy loss*. The 'cross-entropy' part is a bit of mis-characterization of this particular loss. Cross-entropy is defined as $\mathbb{H}[p(\mathrm{x}), q(\mathrm{x})] = -\mathbb{E}_{p(\mathrm{x})}\left[\log q(\mathrm{x})\right]$ for two distributions $p(\mathrm{x})$ and $q(\mathrm{x})$, which is what we have in step #2 of the above derivation but also in step #2 of Equation 10. Thus, squared error could also be called a 'cross-entropy loss', but when people say that, they usually are assuming the labels take on binary or (as we'll see later) categorical values.

1.6 Gradient Descent

If you try to take the derivative of the cross-entropy loss above, set it to zero, and solve for w, you will find that you cannot isolate w to one side of the equation, meaning that the optimization problem has no 'closed-form' solution. Instead we need to find a numerical solution that will only approximate the true optimum. We will use a procedure known as 'gradient descent', a.k.a. 'steepest descent'. The intuition is that we'll start with an initial guess at the value of w and slowly walk down the loss surface, following the direction of steepest descent according to the derivative at our current point. For a generic function

$\phi(\mathrm{z})$ that we wish to minimize, we can apply gradient descent by iterating the equation:

$$z_{t+1} \;=\; z_t \;-\; \alpha \cdot \frac{d}{dz_t}\phi(z_t)$$

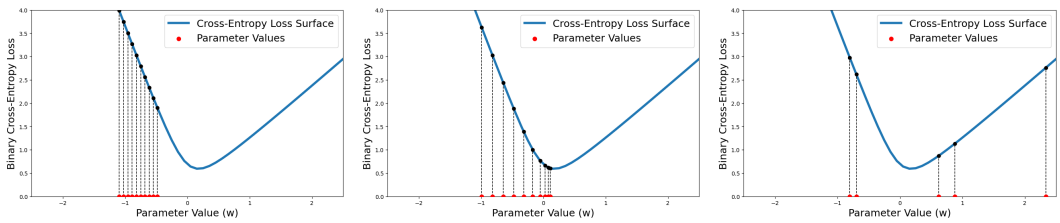
where $\alpha > 0$ is the *learning rate* or *step size* that controls how aggressively we move down the path of steepest descent at each iteration. It is common to set α to be large for the early iterations and then slowly decay it, taking smaller and smaller step sizes, when the procedure gets close to a minimum. We can know if we've approximately converged by tracking the derivative $\frac{d}{dz_t}\phi(z_t)$ since it should be exactly zero at the minimum (or any other critical point—a topic for later in the course).

Applying Gradient Descent to Logistic Regression We will now apply gradient descent to the logistic regression model. The first step will be to take the derivative of the cross-entropy loss w.r.t. the parameter w. When doing this, we will use the fact that the derivative of the logistic function is: $d/dz\ \texttt{logistic}\,(\mathrm{z}) = \texttt{logistic}\,(\mathrm{z}) \cdot (1 - \texttt{logistic}\,(\mathrm{z}))$. I'll leave showing this to the reader as an exercise.

$$\begin{aligned}
\frac{d}{d\mathrm{w}}\ell(\mathrm{w};\mathcal{D}) \;&=\; \frac{d}{d\mathrm{w}}\left[\frac{1}{N}\sum_{n=1}^{N} -\log \text{Bernoulli}\,(y_n; \pi = \texttt{logistic}(f(x_n;\mathrm{w})))\right] \\
&=\; \frac{1}{N}\sum_{n=1}^{N} -y_n \frac{d}{d\mathrm{w}}\left[\log \texttt{logistic}(\mathrm{w}\cdot x_n)\right] \;-\; (1-y_n)\frac{d}{d\mathrm{w}}\left[\log\left(1 - \texttt{logistic}(\mathrm{w}\cdot x_n)\right]\right) \\
&=\; \frac{1}{N}\sum_{n=1}^{N} -y_n \cdot \frac{\texttt{logistic}(\mathrm{w}\cdot x_n)\cdot(1-\texttt{logistic}(\mathrm{w}\cdot x_n))}{\texttt{logistic}(\mathrm{w}\cdot x_n)} \cdot x_n \\
&\qquad\qquad - (1-y_n)\frac{-\texttt{logistic}(\mathrm{w}\cdot x_n)\cdot(1-\texttt{logistic}(\mathrm{w}\cdot x_n))}{1-\texttt{logistic}(\mathrm{w}\cdot x_n)} \cdot x_n \\
&=\; \frac{1}{N}\sum_{n=1}^{N} -y_n \cdot (1-\texttt{logistic}(\mathrm{w}\cdot x_n))\cdot x_n \;+\; (1-y_n)\texttt{logistic}(\mathrm{w}\cdot x_n)\cdot x_n \\
&=\; \frac{1}{N}\sum_{n=1}^{N} (\texttt{logistic}(\mathrm{w}\cdot x_n) - y_n)\cdot x_n \\
&=\; \frac{1}{N}\sum_{n=1}^{N} (\mathbb{E}_p[\mathrm{y}|x_n] - y_n)\cdot x_n \quad \text{(via definition of the logistic regression GLM).}
\end{aligned}$$

Using the definition of logistic regression as a GLM reveals that the derivative takes on a very sensible form: the difference between the expected value of y and its actual value, y_n, scaled by the feature value x_n. When $(\mathbb{E}_p[\mathrm{y}|x_n] - y_n) \approx 0$, the model is a *confident* and *accurate* predictor and thus the contribution of this data point is negligible. If the total derivative is zero, then the model is accurately predicting all training points with *maximal* confidence. Plugging this derivative into the gradient descent equation, we have:

$$\begin{aligned}
w_{t+1} \;&=\; w_t \;-\; \alpha \cdot \frac{d}{dw_t}\ell(w_t;\mathcal{D}) \\
&=\; w_t \;-\; \alpha\left[\frac{1}{N}\sum_{n=1}^{N}(\texttt{logistic}(w_t\cdot x_n) - y_n)\cdot x_n\right].
\end{aligned} \tag{14}$$



(a) Step size is too small. (b) Good step size. (c) Step size is too high.

Figure 1: *Gradient descent for logistic regression, with varying step sizes.* For each run, gradient descent is run for 10 steps with a fixed step size. In (a), the step size is too small ($\alpha = .02$); in (b), the step size is suitable ($\alpha = .05$); in (c), the step size is too large ($\alpha = 1.5$).

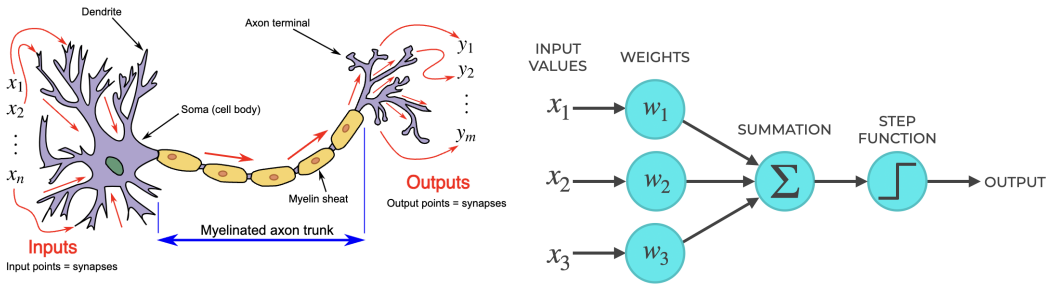
We would implement the above equation numerically by guessing an initial value, w_0, making a prediction using w_0 (i.e. evaluate the logistic output) for all training points, compute the derivative by combining the predictions, the labels and features, as shown above, and lastly updating the weight to arrive at w_1. That process is then repeated for a maximum number of rounds or until the derivative is sufficiently close to zero (e.g. 1×10^{-4}).

Figure 1 shows a simulation for a one-parameter logistic regression model. The (binary) cross-entropy loss surface is visualized by the blue line. The intermediate parameter estimates (w_t) are shown along the x-axis in red, and the dotted line connects them to the loss value that they resulted in. Subfigure 1a shows when the step size is too small, as the maximum number of iterations is reached before the minimum is found. Subfigure 1 shows the other extreme: the step size is too big and so the minimum cannot be found. Instead, the optimizer 'jumps' over the minimum and flys off to the right-hand side of the plot. Subfigure 1b shows when the step size is properly set, as the optimizer gracefully descends to the minimum.

1.7 Models of Artificial Neurons & the Perceptron

So far, the narrative of these notes takes a purely statistical perspective. However, deep learning also has roots in artificial intelligence, which at times has taken inspiration from biological intelligence. The first major step towards formulating a computational model of biological neurons was taken in 1943 by McCulloch and Pitts. A biological neuron can be, very roughly, thought of as a model that takes an input signal (dendrite), performs computation (soma), and passes the output to other connected neurons (axon to synapse to other neuron's dendrite). A visualization is shown in Figure 2a. McCulloch and Pitts proposed a model whose input can be either *excitatory* or *inhibitory*. Its output can then be either *quiet* or *firing*, depending on if the the number of excitatory inputs is equal to or greater than some fixed threshold.

In 1957, Frank Rosenblatt famously implemented a very similar model on an IBM 704 computer, calling it *the perceptron*, and demonstrated that it could 'learn' to classify simple patterns. This demonstration received wide media attention; the New York Times had an article with the headline: "Electronic 'Brain' Teaches Itself." The perceptron can be defined



(a) Biological Neuron (b) The Perceptron

Figure 2: *Biological vs Artificial Neurons.*

for an input $\mathrm{x} \in \mathbb{R}$ and parameters $\mathrm{w} \in \mathbb{R}$ and $\mathrm{b} \in \mathbb{R}$ as:

$$\hat{\mathrm{y}} \;=\; \psi\left(\mathrm{w} \cdot \mathrm{x} + \mathrm{b}\right), \text{ where } \; \psi(\mathrm{z}) = -1 + 2 \cdot \mathbb{I}\left[\mathrm{z} > 0\right],$$

with $\mathbb{I}[\cdot]$ denoting the indicator function that evaluates to 1 if its argument is true and 0 otherwise. In turn, $\psi(\mathrm{z}) = +1$ if its input is greater than zero and -1 otherwise. Thus $\hat{\mathrm{y}}$ is the prediction, and it should model binary data represented as $\mathrm{y} \in \{-1, +1\}$. The similarity to the McCulloch and Pitts model is that the input x is modulated by a weight w, and if $\mathrm{w} \cdot \mathrm{x} > -\mathrm{b}$, then $\hat{\mathrm{y}}$ 'fires,' having a value of one. Thus $-\mathrm{b}$ serves as the activation threshold proposed by McCulloch and Pitts; though it was discrete in the original model and real-valued here. A visualization of the perceptron is shown in Figure 2b.

The perceptron learning algorithm is as follows for an N-sized data set $\mathcal{D} = \{x_n, y_n\}_{n=1}^{N}$, $\mathrm{x} \in \mathbb{R}$, $\mathrm{y} \in \{-1, +1\}$. The algorithm starts by randomly initializing w and b; denote these values as w_0 and b_0. Then for every time step t, a training pair is selected at random (x_r, y_r), the perceptron's prediction is computed as $\hat{y}_r = \psi(w_0 \cdot x_r + b_0)$, and this prediction is checked for correctness, $\hat{y}_r = y_r$. If the prediction is correct, the process repeats for another feature-label pair. If the prediction is incorrect, then the following update is performed to obtain w_{t+1} and b_{t+1}:

$$w_{t+1} = w_t + y_r \cdot x_r, \quad b_{t+1} = b_t + y_r.$$

This process is repeated for either a maximum number of iterations or until all points in the training data have their label correctly predicted.

The update rule for the perceptron learning algorithm looks a bit mysterious, and it is certainly not clear, at least at a first glance, why iterating them amounts to 'learning.' Yet we can examine some parallels to the steepest descent equations for logistic regression to gain intuition. Let's consider the logistic regression update rule given in Section 1.6. But we will make two simplifying assumptions: the update will be computed for just one data point and with a step size of $\alpha = 1$:

$$w_{t+1} \;=\; w_t - (\mathbb{E}[\mathrm{y}|x] - y) \cdot x \;=\; w_t + (y - \mathbb{E}[\mathrm{y}|x]) \cdot x.$$

By comparing this equation to the perceptron's weight update, the only difference is the factor that is multiplied with the features x: $(y - \mathbb{E}[\mathrm{y}|x])$ for logistic regression (for $\mathrm{y} \in \{0, 1\}$) vs y for the perceptron. Now let's assume logistic regression's output can only take

on the extreme values of 0 and 1, i.e. $\mathbb{E}[\mathrm{y}|x] \in \{0, 1\}$. Thus when $\mathbb{E}[\mathrm{y}|x] = y$, their difference will be zero and there will be no update to the parameters. When $\mathbb{E}[\mathrm{y}|x] \neq y$, the difference will either be $+1$ when $y = 1$ or -1 when $y = 0$. Notice that the perceptron learning rule is then recovered *exactly* due to the label support being defined as $\{-1, +1\}$. The update for the second parameter, b, can be recovered by thinking of the features as being a two dimensional vector: $\mathbf{x} = [\mathrm{x}_0, \mathrm{x}_1]^T$ with x_0 always being set to one. Then given the parameter vector $\theta = [\mathrm{b}, \mathrm{w}]^T$, we have $\boldsymbol{\theta}^T\mathbf{w} = \mathrm{b} \cdot \mathrm{x}_0 + \mathrm{w} \cdot \mathrm{x}_1 = \mathrm{w} \cdot \mathrm{x}_1 + \mathrm{b}$. We will discuss this representation in more detail in the next section. In general, we can think of the perceptron as performing a 'hard' update since its predictions are either completely correct or incorrect. On the other hand, logistic regression's learning rule is more precise since it has a more granular notion of how far off its predictions (as represented by $\mathbb{E}[\mathrm{y}|\mathrm{x}]$) are from the true label.

While the invention of the perceptron generated much excitement, this excitement was short-lived. In 1969, Marvin Minsky and Seymour Papert published a book entitled, *Perceptrons: An Introduction to Computational Geometry*. The authors demonstrated the perceptron's limitations—namely, that it could only learn linear decision functions. Thus the perceptron could not model a simple logical function like XOR (i.e. *exclusive or*). This dissolved enthusiasm for artificial neurons as a potential path towards artificial intelligence, and symbolic approaches began to be favored instead. This period generally saw the decline in funding for artificial intelligence in the 1970's and 1980's. When similar approaches started to gain traction again in the 1990's, they were often re-branded as 'machine learning' instead of 'artificial intelligence.'

2 Supervised Learning with Multiple Linear Regression

In the previous section, we have built up our foundation for specifying statistical predictive models and their loss functions. We also covered how to train them using steepest descent. Now we will look to expand these models such that they can have multiple input features and multiple output features. The underlying principles will primarily be the same, but now as parameters and outputs will be vector-valued, we will need to use vector calculus for deriving the corresponding learning updates.

2.1 Multiple Features and Feature Expansions

We will now consider regression models for which the input features are multi-dimensional. In turn, we will now need to specify multiple weight parameters—one for each input dimension. Thus, our (generalized) regression models will be specified as:

$$\mathbb{E}\left[\mathrm{y}|\mathbf{x}\right] \triangleq g^{-1}\left(\mathbf{w}^T\mathbf{x}\right), \quad \text{where} \quad \mathbf{w} = \begin{bmatrix} \mathrm{w}_1 \\ \vdots \\ \mathrm{w}_D \end{bmatrix} \in \mathbb{R}^D \quad \text{and} \quad \mathbf{x} = \begin{bmatrix} \mathrm{x}_1 \\ \vdots \\ \mathrm{x}_D \end{bmatrix} \in \mathbb{R}^D, \tag{15}$$

and where $g^{-1}(\cdot)$ is still the (inverse) link function and y is the scalar label. The most obvious case where this formulation is applicable is for applications that have multiple feature observations. For example, y might be a binary indicator representing the presence

of heart disease, and $\mathbf{x}$ could be various health features such as age, blood pressure, weight, cortisol levels, etc. We will now consider two other examples.

Vector Representation of Offset Parameter One simple but common use for the vector representation, even when there is only input feature, is to encode the offset a.k.a. bias parameter. In the perceptron model, this was b. The regression models we considered so far (except the perceptron) had just one parameter, meaning that we can interpret it as modeling the slope of a line. However, we might also want to model the bias / offset from the x-axis. Or in logistic regression, the offset determines where the decision boundary of $p(\mathrm{y}|\mathbf{x}) = 0.5$ resides. We can treat the offset parameter like any other weight by appending a constant value of one to every feature vector. Thus we have:

$$\mathbb{E}\left[\mathrm{y}|\mathrm{x}\right] \triangleq g^{-1}\left(\mathrm{w}_1 \cdot \mathrm{x} + \mathrm{w}_0\right) = g^{-1}\left(\mathbf{w}^T\mathbf{x}\right), \text{ where } \mathbf{w} = \begin{bmatrix}\mathrm{w}_0\\ \mathrm{w}_1\end{bmatrix} \in \mathbb{R}^D \text{ and } \mathbf{x} = \begin{bmatrix}1\\ \mathrm{x}\end{bmatrix} \in \mathbb{R}^D,$$

where w_0 is the offset parameter and w_1 is the weight that represents the slope parameter, like before. While this may seem to be just a notational trick, this allows for cleaner programmatic implementations since the offset parameter does not have to be treated differently than the other weight parameters. In this course, we will usually assume offset parameters are treated in this way, by appending a constant 1 to the feature vectors.

Polynomial Basis Expansion Linear models may seem limited, at first glance, to encoding only straight lines. Yet, one trick to make linear models more expressive while not adding much computational overhead is to use a *feature expansion*. Let's assume we have a scalar data $\mathrm{x} \in \mathbb{R}$ and $\mathrm{y} \in \mathbb{R}$. Our previous models for this data used only one parameter and thus encoded the slope of a line through the origin. Yet what if the relationship between the features and labels is more complex than that? One simple trick is to create 'dummy' features by replicating the existing feature. One simple but still very expressive way is to use a K-degree *polynomial basis*. This means that we create a new feature vector by taking x to powers up to degree K:

$$\tilde{\mathbf{x}} = \left[\mathrm{x}^0\ \mathrm{x}^1\ \mathrm{x}^2\ \ldots\ \mathrm{x}^K\right]^T = \left[1\ \mathrm{x}\ \mathrm{x}^2\ \ldots\ \mathrm{x}^K\right]^T.$$

Then when we use this expanded feature set in the linear model, the model encodes a K-degree polynomial, with the $K+1$ parameters / weights serving as the coefficients:

$$\mathbb{E}\left[\mathrm{y}|\mathrm{x}\right] \triangleq g^{-1}\left(\mathbf{w}^T\tilde{\mathbf{x}}\right) = g^{-1}\left(\sum_{k=0}^{K} \mathrm{w}_k \cdot \mathrm{x}^k\right), \text{ where } \mathbf{w} = \begin{bmatrix}\mathrm{w}_0\\ \vdots\\ \mathrm{w}_K\end{bmatrix} \in \mathbb{R}^{K+1} \text{ and } \tilde{\mathbf{x}} = \begin{bmatrix}1\\ \vdots\\ \mathrm{x}^K\end{bmatrix} \in \mathbb{R}^{K+1}.$$

Figure 3 shows a simulation in which the true function we are trying to model is a $K = 7$ degree polynomial. The plots show when real-valued regression is run with degrees ranging from one (linear) to fifteen. We see that once the degree of the model surpasses the degree of the true function ($K = 10$), then the model is well-fit to the data.

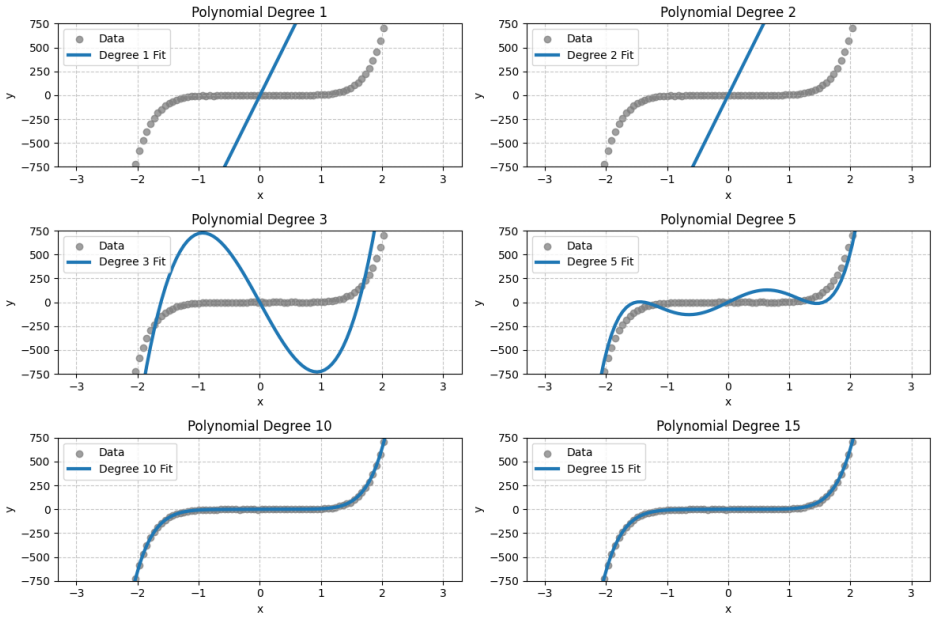


Figure 3: *Linear Regression with Polynomial Basis*. The data represents a 7th-degree polynomial. We see that as the degree of the model's feature basis increases, the model can express richer and richer functions.

2.2 Revisiting Gradient Descent with Multivariate Derivatives

Now that we have seen vector-based models, fitting them will require that we compute the derivative for each parameter. This will require that we adopt tools from multivariate calculus—namely, the gradient operator. Consider an arbitrary differentiable function $f : \mathbb{R}^D \mapsto \mathbb{R}$, meaning that its input is a D-dimensional vector and its output is a scalar. We define the *gradient* of this function to be:

$$\nabla_{\boldsymbol{x}} f(\boldsymbol{x}) \;=\; \left[\frac{df}{d\boldsymbol{x}}\right]^T \;=\; \left[\frac{\partial f(\boldsymbol{x})}{\partial x_1} \cdots \frac{\partial f(\boldsymbol{x})}{\partial x_d} \cdots \frac{\partial f(\boldsymbol{x})}{\partial x_D}\right].$$

Notice that the gradient is *row vector* $\mathbb{R}^{1\times D}$; this will be important when we start to chain together derivatives when differentiating function compositions.

Returning to the steepest descent equations, their multivariate version can be written with the gradient operator as follows:

$$\boldsymbol{w}_{t+1} \;=\; \boldsymbol{w}_t \;-\; \alpha \cdot \left[\nabla_{\boldsymbol{w}_t} \ell\left(\boldsymbol{w}_t; \mathcal{D}\right)\right]^T \tag{16}$$

where α is still the step size (a.k.a. learning rate), $\boldsymbol{w}_t$ is the parameter vector, and $\nabla_{\boldsymbol{w}_t} \ell\left(\boldsymbol{w}_t; \mathcal{D}\right)$ is the gradient of the loss function w.r.t. the parameter vector. As we assume that $\boldsymbol{w}_t$ is a column vector, we need to re-orient the result of the gradient (which is assumed to be a row vector) in order to perform the update via subtraction.

In the univariate case, we thought of steepest descent as forming a tangent line that guides how we descend down the curve. Now we should have a multi-dimensional surface in mind, and the gradient gives us, upon each evaluation, a (hyper)-plane that guides our descent. A demonstration for logistic regression with two parameters is shown in Figure 4.

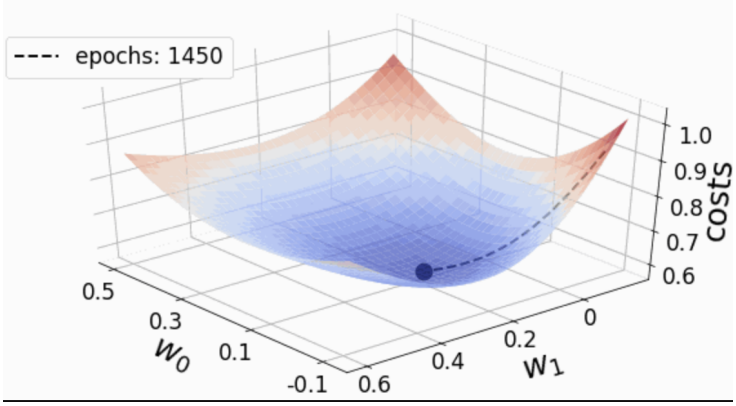


Figure 4: *Optimization Trajectory for Logistic Regression.* This figure shows the path (dashed line) through the optimization surface taken for gradient descent on a logistic regression model. The two parameters are shown on the $x-$ and z-axes and the loss (a.k.a. cost) is shown on the y-axis. Optimization is run for 1450 iterations through the training data (a.k.a. *epochs*).

The two parameters are shown on the $x-$ and z-axes and the cross-entropy loss function (a.k.a. cost function) is shown on the y-axis. The trajectory of the intermediate parameter values is shown by the dashed line, with the final parameter setting denoted by the circle.

Logistic Regression with Multiple Features Lets consider the multi-feature version of the logistic regression model: $\mathbb{E}[\mathrm{y}|\mathbf{x}] = s\left(\mathbf{w}^T\mathbf{x}\right)$ where, as before, $\mathbf{x} \in \mathbb{R}^D$ are the features and $\mathbf{w} \in \mathbb{R}^D$ are the parameters. Yet now the label is binary valued, $\mathrm{y} \in \{0, 1\}$, and $s(\mathrm{z}) = 1/(1 + \exp\{-\mathrm{z}\})$ is the logistic function. This model would be trained by the binary cross-entropy loss function for an N-sized data set:

$$\ell\left(\mathbf{w}; \mathcal{D}\right) \;=\; \frac{1}{N}\sum_{n=1}^{N} -y_n \log\left\{s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\right\} - (1-y_n)\log\left\{1 - s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\right\}.$$

The gradient w.r.t. the weight vector is:

$$\begin{aligned}
&\nabla_{\mathbf{w}}\, \ell\left(\mathbf{w}; \mathcal{D}\right) \\
&= \nabla_{\mathbf{w}}\left[\frac{1}{N}\sum_{n=1}^{N} -y_n \log\left\{s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\right\} - (1-y_n)\log\left\{1 - s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\right\}\right] \\
&= \frac{1}{N}\sum_{n=1}^{N} -y_n \cdot \nabla_{\mathbf{w}}\left[\log\left\{s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\right\}\right] - (1-y_n)\cdot\nabla_{\mathbf{w}}\left[\log\left\{1 - s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\right\}\right] \\
&= \frac{1}{N}\sum_{n=1}^{N} -y_n \cdot \frac{1}{s\left(\mathbf{w}^T\boldsymbol{x}_n\right)}\cdot\nabla_{\mathbf{w}}\left[s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\right] - (1-y_n)\cdot\frac{-1}{1 - s\left(\mathbf{w}^T\boldsymbol{x}_n\right)}\cdot\nabla_{\mathbf{w}}\left[s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\right].
\end{aligned}$$

Focusing on just the gradient of the logistic function, we have:

$$\begin{aligned}
\nabla_{\mathbf{w}}\ s\left(\mathbf{w}^T\boldsymbol{x}_n\right) &= s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\cdot\left(1-s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\right)\cdot\nabla_{\mathbf{w}}\left[\mathbf{w}^T\boldsymbol{x}_n\right] \\
&= s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\cdot\left(1-s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\right)\cdot\left[\frac{\partial\ \mathbf{w}^T\boldsymbol{x}_n}{\partial\ w_1}\ldots\frac{\partial\ \mathbf{w}^T\boldsymbol{x}_n}{\partial\ w_D}\right] \\
&= s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\cdot\left(1-s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\right)\cdot[x_{n,1}\ldots x_{n,D}] \\
&= s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\cdot\left(1-s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\right)\cdot\boldsymbol{x}_n^T.
\end{aligned}$$

Plugging this back into the expression above, we have:

$$\begin{aligned}
&= \frac{1}{N}\sum_{n=1}^{N} -y_n\cdot\frac{1}{s\left(\mathbf{w}^T\boldsymbol{x}_n\right)}\cdot s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\cdot\left(1-s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\right)\cdot\boldsymbol{x}_n^T \\
&\qquad -(1-y_n)\cdot\frac{-1}{1-s\left(\mathbf{w}^T\boldsymbol{x}_n\right)}\cdot s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\cdot\left(1-s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\right)\cdot\boldsymbol{x}_n^T \\
&= \frac{1}{N}\sum_{n=1}^{N}\left[-y_n\cdot\left(1-s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\right)-(y_n-1)\cdot s\left(\mathbf{w}^T\boldsymbol{x}_n\right)\right]\cdot\boldsymbol{x}_n^T \\
&= \frac{1}{N}\sum_{n=1}^{N}\left[s\left(\mathbf{w}^T\boldsymbol{x}_n\right)-y_n\right]\cdot\boldsymbol{x}_n^T.
\end{aligned}$$

This result would be plugged into Equation 16 and iterated until convergence.

2.3 Multiple Output Dimensions

Just as we considered models with multiple input dimensions, we will also want to consider models with multiple *output* dimensions. This is useful for, for example, predicting the trajectory of an object, as its spatial coordinate in two or three dimensions would need to be output by the model. Multi-dimensional outputs are also crucial for defining predictive models for *categorical* data, as we will see in an example below. In general, the GLM formulation for a K-dimensional label denoted by the *vector* $\mathbf{y}$ is:

$$\begin{aligned}
\mathbb{E}\left[\mathbf{y}|\mathbf{x}\right] &\triangleq g^{-1}\left(\mathbf{W}^T\mathbf{x}\right), \quad \text{where } \mathbf{x} = \begin{bmatrix}\mathrm{x}_1\\ \vdots \\ \mathrm{x}_D\end{bmatrix} \in \mathbb{R}^D \text{ and} \\
\mathbf{W} &= \left[\mathbf{w}_1\ldots\mathbf{w}_K\right] = \begin{bmatrix}\mathrm{w}_{1,1} & \ldots & \mathrm{w}_{1,K}\\ \vdots & \ddots & \vdots \\ \mathrm{w}_{D,1} & \ldots & \mathrm{w}_{D,K}\end{bmatrix} \in \mathbb{R}^{D\times K}.
\end{aligned} \tag{17}$$

The two crucial differences needed to arrive at this multi-output formulation are (1) there is now a $(D\times K)$-parameter *matrix* to transform the D-dimensional input features into a K-dimensional output, and (2) the inverse link function is applied to a vector input. It depends on the model formulation whether the link function is applied element-wise or has dependencies across dimensions. Let's examine the case of the latter below.

Example: Categorical Labels One of the most common tasks in machine learning (and therefore in deep learning as well) is *classification*: categorizing a given set of input features

into one of K discrete, disjoint groups. We have already seen an example of a binary classifier (i.e. two groups), which was logistic regression, and so this can be thought of as a higher dimensional generalization. The usual way to represent this encoding is via a so-called 'one hot' vector representation, meaning that one element takes on value 1 and the rest are 0. For example, $\mathbf{y}^T = [0, 0, 1, 0]$ means that there are $K = 4$ total categories, and this label is denoting membership in the third category. The natural choice of distribution for data of this type is the *categorical distribution* (a.k.a. the multinoulli):

$$p(\mathbf{y}; \boldsymbol{\pi}) \;=\; \text{Categorical}(\mathbf{y}; \boldsymbol{\pi}) \;\triangleq\; \prod_{k=1}^{K} \pi_k^{\mathrm{y}_k},$$

where the distribution's parameters are represented by the K-dimensional vector $\boldsymbol{\pi}$, which has the constraints $\pi_k \in [0, 1]$ and $\sum_k \pi_k = 1$. We can interpret these parameters as the probability of each class / category, and therefore the probabilities must sum to one to be well-defined.

Now defining a GLM with the categorical distribution, we have:

$$\mathbb{E}\left[\mathbf{y}|\mathbf{x}\right] \;=\; \boldsymbol{\pi} \;=\; g^{-1}\left(\mathbf{W}^T\mathbf{x}\right)$$

where, like with the Bernoulli, the success probabilities of the classes are the distribution's mean. And as above, we will use a $(D \times K)$-dimensional matrix of parameters. The canonical inverse link function for this setting is known as the *softmax* function:

$$\texttt{softmax}_j(\mathbf{z}) \;\triangleq\; \frac{\exp\{z_j\}}{\sum_{k=1}^{K} \exp\{z_k\}}, \quad \text{where} \;\; \mathbf{z} \in \mathbb{R}^K \tag{18}$$

and where the subscript j in $\texttt{softmax}_j(\mathbf{z})$ denotes the function's jth output dimension. Summing over all dimensions in the denominator is what enforces the constraints imposed by interpreting the output as probabilities. In general, we have $\texttt{softmax} : \mathbb{R}^K \mapsto \Delta_K$ where Δ_K denotes the K-dimensional *simplex*, the space of all positive K-dimensional vectors that sum to one. Note that the 'softmax' function is mis-named, as it really is performing a soft 'argmax.'

Putting this all together, we can derive the *categorical cross-entropy loss function* as:

$$
\begin{aligned}
\boldsymbol{W}^* &= \arg\min_{\mathbf{W}} \; \mathbb{E}_{\mathbb{P}(\mathbf{x})} \mathbb{KLD}\left[\, \mathbb{P}(\mathbf{y}|\mathbf{x}) \;||\; \text{Categorical}\left(\mathbf{y}; \boldsymbol{\pi} = \texttt{softmax}(f(\mathbf{x}; \mathbf{W}))\right) \,\right] \\
&= \arg\min_{\mathbf{W}} \; \mathbb{E}_{\mathbb{P}(\mathbf{x})} \mathbb{E}_{\mathbb{P}(\mathbf{y}|\mathbf{x})} \left[-\log \text{Categorical}\left(\mathbf{y}; \boldsymbol{\pi} = \texttt{softmax}(f(\mathbf{x}; \mathbf{W}))\right)\right] \quad \text{(drop entropy term)} \\
&\approx \arg\min_{\mathbf{W}} \; \frac{1}{N} \sum_{n=1}^{N} -\log \text{Categorical}\left(\boldsymbol{y}_n; \boldsymbol{\pi}_n = \texttt{softmax}(f(\boldsymbol{x}_n; \mathbf{W}))\right) \quad \text{(Monte Carlo approximation)} \\
&= \arg\min_{\mathbf{W}} \; \frac{1}{N} \sum_{n=1}^{N} -\log \left\{ \prod_{k=1}^{K} \pi_{n,k}^{y_{n,k}} \right\} \\
&= \arg\min_{\mathbf{W}} \; \frac{1}{N} \sum_{n=1}^{N} \sum_{k=1}^{K} -y_{n,k} \cdot \log \pi_{n,k} \\
&= \arg\min_{\mathbf{W}} \; \frac{1}{N} \sum_{n=1}^{N} \sum_{k=1}^{K} -y_{n,k} \cdot \log \texttt{softmax}_k(f(\boldsymbol{x}_n; \mathbf{W})) \\
&= \arg\min_{\mathbf{W}} \; \frac{1}{N} \sum_{n=1}^{N} \sum_{k=1}^{K} -y_{n,k} \cdot \log \frac{\exp\{\mathbf{w}_k^T \boldsymbol{x}_n\}}{\sum_{j=1}^{K} \exp\{\mathbf{w}_j^T \boldsymbol{x}_n\}} \\
&= \arg\min_{\mathbf{W}} \; \frac{1}{N} \sum_{n=1}^{N} \sum_{k=1}^{K} -y_{n,k} \cdot \left[\mathbf{w}_k^T \boldsymbol{x}_n - \log \left\{ \sum_{j=1}^{K} \exp\{\mathbf{w}_j^T \boldsymbol{x}_n\} \right\} \right].
\end{aligned}
$$

We will need to use gradient descent to perform this optimization. We'll start by deriving the gradient for the one particular column of $\mathbf{W}$ that corresponds to the dimension for which $\mathrm{y}_k = 1$; call it $\mathbf{w}_k$:

$$
\begin{aligned}
\nabla_{\mathbf{w}_k} \, \ell(\mathbf{W}; \mathcal{D}) &= \frac{1}{N} \sum_{n=1}^{N} \nabla_{\mathbf{w}_k} \sum_{k=1}^{K} -y_{n,k} \cdot \left[\mathbf{w}_k^T \boldsymbol{x}_n - \log \left\{ \sum_{j=1}^{K} \exp\{\mathbf{w}_j^T \boldsymbol{x}_n\} \right\} \right] \\
&= \frac{1}{N} \sum_{n=1}^{N} -\nabla_{\mathbf{w}_k} \left[\mathbf{w}_k^T \boldsymbol{x}_n - \log \left\{ \sum_{j=1}^{K} \exp\{\mathbf{w}_j^T \boldsymbol{x}_n\} \right\} \right] \quad (y_{n,k} = 1 \text{ so we can drop it}) \\
&= \frac{1}{N} \sum_{n=1}^{N} - \left[\nabla_{\mathbf{w}_k} \left[\mathbf{w}_k^T \boldsymbol{x}_n \right] - \frac{1}{\sum_{j=1}^{K} \exp\{\mathbf{w}_j^T \boldsymbol{x}_n\}} \nabla_{\mathbf{w}_k} \left[\sum_{j=1}^{K} \exp\{\mathbf{w}_j^T \boldsymbol{x}_n\} \right] \right] \\
&= \frac{1}{N} \sum_{n=1}^{N} - \left[\boldsymbol{x}_n^T - \frac{\exp\{\mathbf{w}_k^T \boldsymbol{x}_n\}}{\sum_{j=1}^{K} \exp\{\mathbf{w}_j^T \boldsymbol{x}_n\}} \nabla_{\mathbf{w}_k} \left[\mathbf{w}_k^T \boldsymbol{x}_n \right] \right] \\
&= \frac{1}{N} \sum_{n=1}^{N} - \left[\boldsymbol{x}_n^T - \pi_{n,k} \cdot \boldsymbol{x}_n^T \right] \\
&= \frac{1}{N} \sum_{n=1}^{N} - \left(1 - \pi_{n,k}\right) \cdot \boldsymbol{x}_n^T \\
&= \frac{1}{N} \sum_{n=1}^{N} \left(\pi_{n,k} - 1\right) \cdot \boldsymbol{x}_n^T.
\end{aligned}
$$

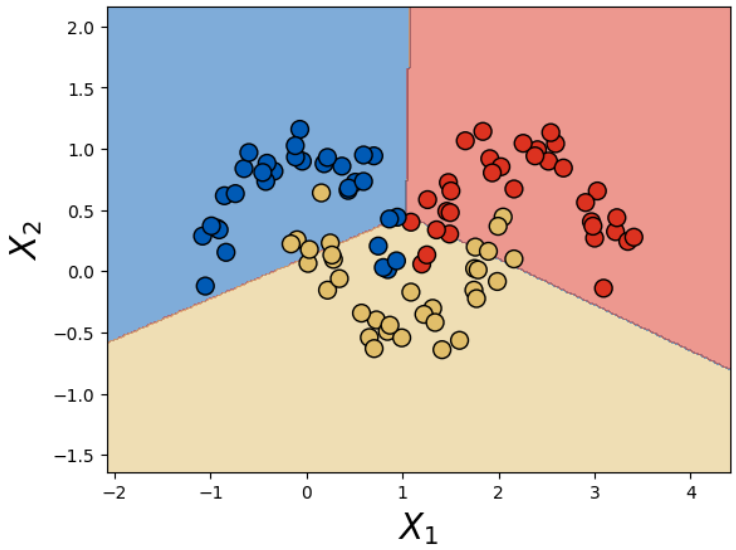


Figure 5: *Example of a Multi-Class Linear Classifier (i.e. Categorical Regression).* The plots shows the feature space; training data points are shown by the scatter plot and colored according to their true class assignment. The background is shaded according to how a 3-class linear classifier would categorize the data points.

Now turn to the dimensions for which $\mathrm{y}_j = 0$; call one of them $\mathbf{w}_i$:

$$
\begin{aligned}
\nabla_{\mathbf{w}_i}\, \ell(\mathbf{W}; \mathcal{D}) &= \frac{1}{N}\sum_{n=1}^{N} \nabla_{\mathbf{w}_i} \sum_{k=1}^{K} -y_{n,k} \cdot \left[\mathbf{w}_k^T \boldsymbol{x}_n - \log\left\{ \sum_{j=1}^{K} \exp\{\mathbf{w}_j^T \boldsymbol{x}_n\} \right\} \right] \\
&= \frac{1}{N}\sum_{n=1}^{N} -\nabla_{\mathbf{w}_i} \left[\mathbf{w}_k^T \boldsymbol{x}_n - \log\left\{ \sum_{j=1}^{K} \exp\{\mathbf{w}_j^T \boldsymbol{x}_n\} \right\} \right] \quad (\nabla_{\mathbf{w}_i} \mathbf{w}_k^T \boldsymbol{x}_n = 0) \\
&= \frac{1}{N}\sum_{n=1}^{N} \frac{1}{\sum_{j=1}^{K} \exp\{\mathbf{w}_j^T \boldsymbol{x}_n\}} \nabla_{\mathbf{w}_i} \left[\sum_{j=1}^{K} \exp\{\mathbf{w}_j^T \boldsymbol{x}_n\} \right] \\
&= \frac{1}{N}\sum_{n=1}^{N} \frac{\exp\{\mathbf{w}_i^T \boldsymbol{x}_n\}}{\sum_{j=1}^{K} \exp\{\mathbf{w}_j^T \boldsymbol{x}_n\}} \nabla_{\mathbf{w}_i} \left[\mathbf{w}_i^T \boldsymbol{x}_n \right] \\
&= \frac{1}{N}\sum_{n=1}^{N} \pi_{n,i} \cdot \boldsymbol{x}_n^T.
\end{aligned}
$$

Lastly, for some nicer book keeping, we can combine the equations to get a unified update rule:

$$
\nabla_{\mathbf{w}_j}\, \ell(\mathcal{D}; \mathbf{W}) = \frac{1}{N}\sum_{n=1}^{N} (\pi_{n,j} - y_{n,j}) \cdot \boldsymbol{x}_n^T, \tag{19}
$$

which recovers the first gradient when $y_{n,j} = 1$ and the second when $y_{n,j} = 0$. Finally, the gradient descent update for the jth weight vector is:

$$
\boldsymbol{w}_{j,t+1} = \boldsymbol{w}_{j,t} - \alpha \cdot \nabla_{\boldsymbol{w}_{j,t}}\, \ell(\boldsymbol{W}_t; \mathcal{D}),
$$

and we would perform that update for all of the $j \in [1, K]$ weights vectors.

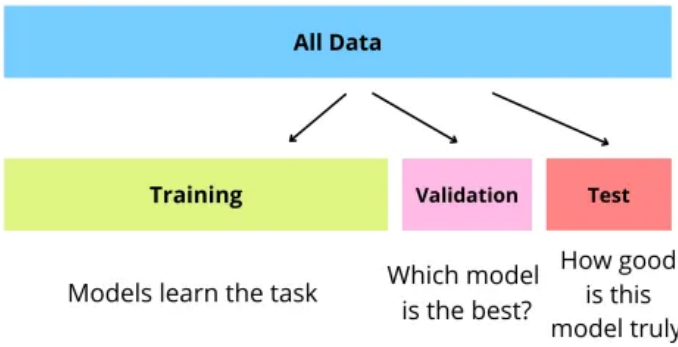


Figure 6: *Splitting the Available Data for Training, Validation, and Testing*

3 Model Evaluation, Model Selection, and Capacity Control

Until now, we have been primarily concerned with *training* predictive models. But there are other important issues that we so far ignored: the related topics of model evaluation, model selection, and regularization. These pertain to the issues of *how well do we expect the model to perform when deployed to the real world?* And, *when we have multiple models, which one should we deploy?*

3.1 Model Evaluation

Recall that the goal of predictive modeling is to approximate the true, data-generating process: $p(\mathrm{y}; f(\mathbf{x})) \approx \mathbb{P}(\mathrm{y}|\mathbf{x})$. Yet we don't have direct access to $\mathbb{P}(\mathrm{y}|\mathbf{x})$; thus, how should we evaluate if our model will work when we deploy it to the real world (e.g. embed it within a self-driving car, integrate it into our web application, etc)? Recall that we only see samples from our generative process, which we call 'data': $\mathcal{D} \sim \mathbb{P}(\mathrm{y}, \mathbf{x})$. If $\mathcal{D}$ is all of the data that we have access to during model development, and we use all of $\mathcal{D}$ to train the model, then assessing the model's performance on $\mathcal{D}$ again isn't ideal. This is like, given some practice problems for which you already know the answers, using those same practice problems to study for the exam. While performance on those known problems tells you something about test-time performance, it does not simulate the actual process of having to answer truly never-before-seen questions. Similarly, we want to test our models by having them make predictions on data points that they have never seen before. Thus, when evaluating models, it's of the utmost importance that we evaluate them on 'held-out' data—that is, data that was not used for training. One simple quantity to compute is the loss function used for training but with the held-out data plugged in for the training data:

$$\ell\left(\mathbf{W}^*; \mathcal{D}_{\text{held-out}}\right)$$

where $\mathbf{W}^*$ are the parameters that were found by training and $\mathcal{D}_{\text{held-out}}$ is the never-before-seen data.

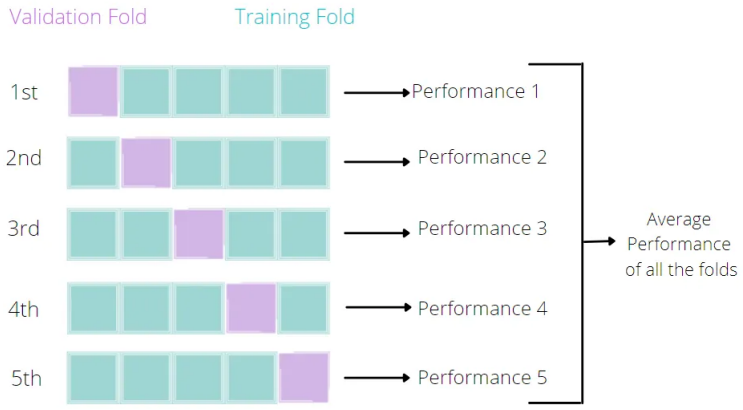


Figure 7: *An Example of 5-Fold Cross Validation.*

3.1.1 Train-Validation-Test Splits

Yet we are often limited in the amount of data that we have and thus must use it sparingly to perform the aforementioned held-out evaluation. One common way to do this is to split the finite data set that we are given into three pieces, with the largest piece used for training ($60\% - 80\%$), the next biggest used for validation ($30\% - 10\%$), and the final part used for testing ($20\% - 10\%$). See Figure 6 for a diagram. As the name implies, the training split is used for training the model. The 'validation' split is used for selecting among multiple models: for example, if you fit multiple models, each with different hyperparameter settings such as step size or polynomial degree. Every time the validation split is used to revise the model and improve the parameter settings, it loses its value. When you believe you have finally selected the best model, you can use the test split to calculate the final performance that is expected on the data that will be seen when the model is deployed and receives data from the real world.

If there is not enough available data to make three sufficiently large subsets, one strategy is to use the training set for both training and validation by creating multiple 'folds.' The procedure is visualized in Figure 7 for 5 folds. The training data is split into five subsets, with four being used for training and one for validation. The subset that is used for validation is varied across all five partitions. Note that this then *requires re-training the model five times!* Thus, while the evaluation procedure is more data efficient, we must pay in the computational cost of re-training the model. The most extreme form of this procedure is *leave-one-out cross validation*, where all but one data points are used for training and the remaining point is used for validation. Thus this procedure is extremely expensive since it requires re-training the model as many times as there are data points. Generally, using more folds gives better estimates of the validation error, but the choice must be balanced with the computational considerations.

3.1.2 Example Evaluation Metrics

Now let's examine some example evaluation metrics for real-valued regression and classification.

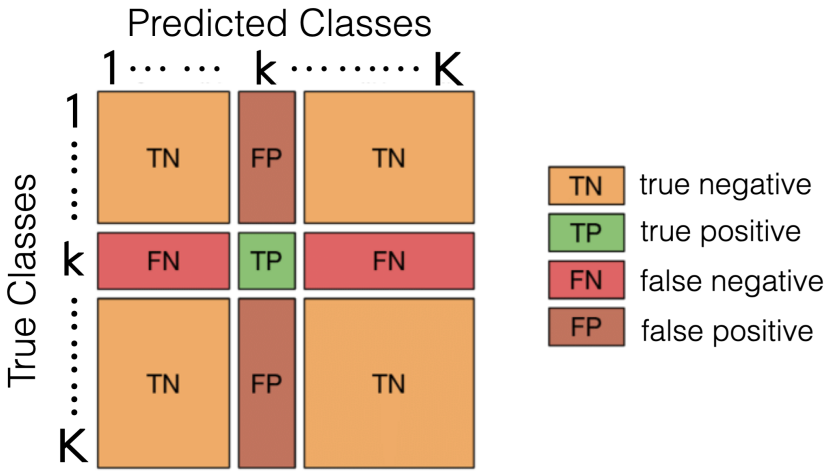


Figure 8: *Confusion Matrix.* The entries of a confusion matrix are the counts of the number of times an instance of the ground-truth class denoted by the row indices is classified as the class denoted by the column indices. Hence, the confusion matrix of a classifier that never makes a mistake on the tested data has zero entries for all off-diagonal elements.

Real-Valued Regression Recall that when training a model for real-valued regression, we usually use the squared error function. When evaluating these models, we usually use a slightly different quantity called the *root-mean-squared error* (RMSE):

$$\texttt{RMSE}\left(\mathbf{w}^*; \mathcal{D}_{\text{held-out}}\right) = \sqrt{\frac{1}{M}\sum_{m=1}^{M}\left(\mathbb{E}\left[y_m | f\left(\boldsymbol{x}_m; \mathbf{w}^*\right)\right] - y_m\right)^2}$$

where M is the number of evaluation points and $\mathbb{E}\left[y_m | f\left(\boldsymbol{x}_m; \mathbf{w}^*\right)\right] = \left(\mathbf{w}^*\right)^T \mathbf{x}$ for real-valued regression. The reason why the square root operation is added is so that the units of the error are on the same scale as the label's.

Classification, a.k.a. Categorical Regression For classification tasks, we can also use the training loss evaluated (i.e. cross-entropy error) on held-out data to evaluate the model. However, this doesn't fully capture how classifiers are used when they are deployed. The cross-entropy loss will consider the precise value of $\mathbb{E}\left[y_m | f\left(\boldsymbol{x}_m; \mathbf{w}^*\right)\right]$, but in practice, we often need to make a discrete choice. *Which class / category does this instance before too?* If the goal of the classifier is to filter out spam, we need it to make a clear decision whether to let the incoming email go into the inbox or into the junk folder. One metric that directly measure this decision-making is *accuracy*:

$$\texttt{Accuracy}(\mathbf{w}^*; \mathcal{D}_{\text{held-out}}) = \frac{1}{M}\sum_{m=1}^{M}\mathbb{I}\left[\texttt{round}\left(\mathbb{E}[y_m | f(\boldsymbol{x}_m; \mathbf{w}^*)]\right) = y_m\right],$$

where, if $y_m \in \{0, 1\}$, $\texttt{round}\left(\mathbb{E}[y_m | f(\boldsymbol{x}_m; \mathbf{w}^*)]\right)$ simply means that we return 1 if $\mathbb{E}[y_m | f(\boldsymbol{x}_m; \mathbf{w}^*)] \geq 0.5$ and 0 otherwise. If $y_m \in \{1, \ldots, K\}$, then $\texttt{round}\left(\mathbb{E}[y_{m,k} = 1 | f(\boldsymbol{x}_m; \mathbf{w}^*)]\right) = \arg\max_k \mathbb{E}[y_{m,k} = 1 | f(\boldsymbol{x}_m; \mathbf{w}^*)]$. Usually this is implemented by taking the maximum dimension of the probability vector that is produced by the softmax transformation.

Accuracy is an aggregate metric and therefore can hide disparate performance across subclasses. That is, maybe the classifier identifies two out of three classes well but does poorly discriminating the third one. This might result in an accuracy that is indistinguishable from doing moderately well across all classes. If this is the fear, then we can use a *confusion matrix* to better assess per-class performance. See Figure 8 for a visualization. The entries of a confusion matrix are the counts of the number of times an instance of the ground-truth class denoted by the row indices is classified as the class denoted by the column indices. Mathematically, we can write this as:

$$C_{i,j} \;=\; \sum_{m=1}^{M} \mathbb{I}[y_m = i] \cdot \mathbb{I}[\hat{y}_m = j]$$

where y_m is the true class of the mth instance and $\hat{y}_m$ is the predicted class of the mth instance. Thus, the confusion matrix of a classifier that never makes a mistake on the tested data has zero entries for all off-diagonal elements. Then entry $C_{k,k}$ is the count of the number of times an instance of class k was correctly classified as such.

If there is an imbalanced number of classes, then accuracy may not be a good metric. Consider the case of identifying credit card fraud. Fraud only happens in a small minority of cases—say, 1 out of 1000—and therefore a classifier that always predicts 'no fraud' will have an accuracy of about 99.9%. This classifier, of course, would actually be useless because we presumably built it with the motivation of detecting fraud.

Two metrics that are useful when there is class imbalance are *precision* and *recall*. They can easily be defined by considering the columns and rows of the confusion matrix, respectively:

$$\texttt{Precision}_k(\boldsymbol{C}) \;=\; \frac{C_{k,k}}{\sum_{j=1}^{K} C_{j,k}}, \quad \texttt{Recall}_k(\boldsymbol{C}) \;=\; \frac{C_{k,k}}{\sum_{j=1}^{K} C_{k,j}}.$$

The precision for the kth class is the number of times class k was predicted correctly ($C_{k,k}$) divided by the total number of times class k was predicted, i.e. the sum of the elements in the kth column $\left(\sum_{j=1}^{K} C_{j,k}\right)$. On the other hand, *recall* is the fraction of times the classifier successfully identified class k ($C_{k,k}$) out of all of the instances of class k, i.e. the sum of the elements in the kth row $\left(\sum_{j=1}^{K} C_{k,j}\right)$. Returning to the example of detecting credit card fraud, while that classifier would have a high accuracy, it's precision and recall for the 'fraud' class would be zero. If one wishes to simultaneously quantify precision and recall, all within one value, the *F1* score is the harmonic mean of precision and recall:

$$\texttt{F1}_k(\boldsymbol{C}) \;=\; 2 \cdot \frac{\texttt{Precision}_k(\boldsymbol{C}) \cdot \texttt{Recall}_k(\boldsymbol{C})}{\texttt{Precision}_k(\boldsymbol{C}) + \texttt{Recall}_k(\boldsymbol{C})}.$$

The harmonic mean is used so that the smaller values are emphasized in the aggregation. All of the above metrics are defined class-wise, but the 'macro' versions can be computed by averaging the values over all classes.

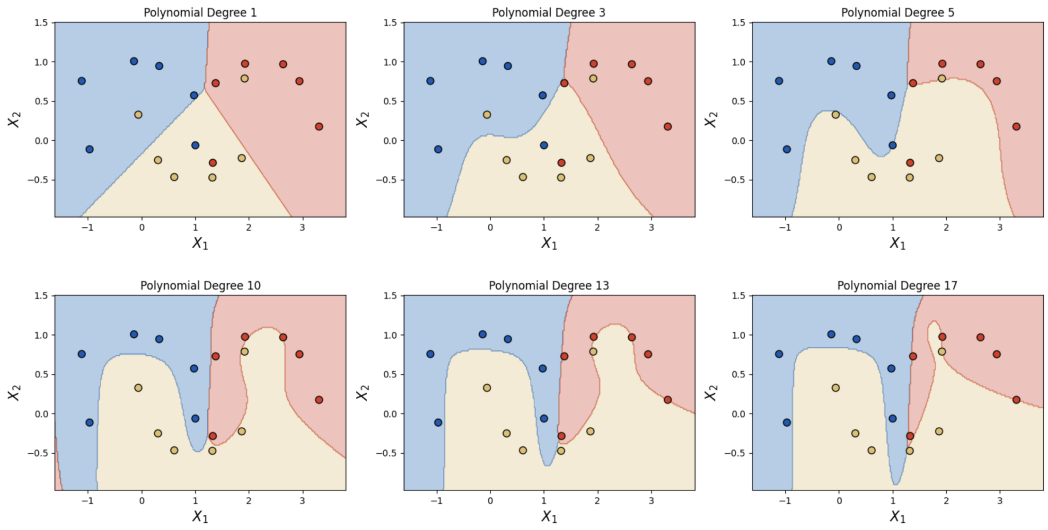


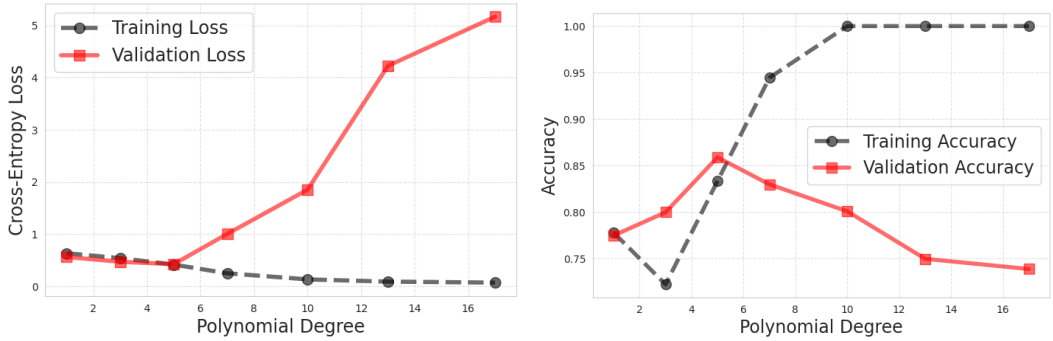
Figure 9: *Classifiers with a Polynomial Basis of Increasing Degree.* The sequence of plots shows the resulting decision boundaries when we increase the degree of the polynomial basis function.

3.2 Model Selection

Having the held-out validation set is integral for performing *model selection*: choosing the best out of a candidate set of models. This is especially important to do when the candidate models vary in their complexity and expressive power. Examine Figure 9; it shows the classification boundaries obtained via categorical regression with a polynomial basis of increasing degree. The classification boundaries between categories are linear in the case of the top left figure and become highly non-linear, culminating in the 17-degree basis shown in the bottom right. *Which of these models is the best classifier to deploy to the real world?*

We can answer this question by plotting the validation loss on the y-axis and the polynomial degree on the x-axis. See Figure 10a for this plot, showing the cross-entropy loss for the validation data in red and for the training data in black. As the polynomial degree increases, the training loss strictly decreases, as the model can fit the data better and better. However, the validation loss (red) does not follow the same monotonic pattern. Instead, the validation loss decreases for degrees 1, 3, and 5, but it increases for all degrees thereafter. This is because the extra model capacity afforded by the higher degrees is only useful for modeling noise in the training data, not for exactly signals that will *generalize* to other data sets. Thus, we would choose the degree 5 polynomial model to deploy to our application. Figure 10 is the same plot but with accuracy as the quality metric instead of cross-entropy loss. Here we see that the same polynomial degree is identified as best (a degree of 5). Yet, interestingly, accuracy is not monotonic and actually decreases from degree 1 to degree 3 and then continues upward from there, finally reaching an accuracy of 100%. An accuracy of 100% should always bring suspicious: either you have an extremely easy problem, a very small dataset, have overfit the model, or otherwise have a bug in your code.

Fortunately, the magic of cross validation (i.e. evaluation on a held-out set) is designed exactly for these situations. The model builder has the natural desire to define more and more expressive models, hoping that the additional flexibility translates into better predic-



(a) Cross-Entropy Loss (b) Classification Accuracy

Figure 10: *Performance on Training vs Held-Out Validation Data.* Cross-entropy loss vs polynomial degree of the classifier is shown in (a); accuracy vs polynomial degree is shown in (b).

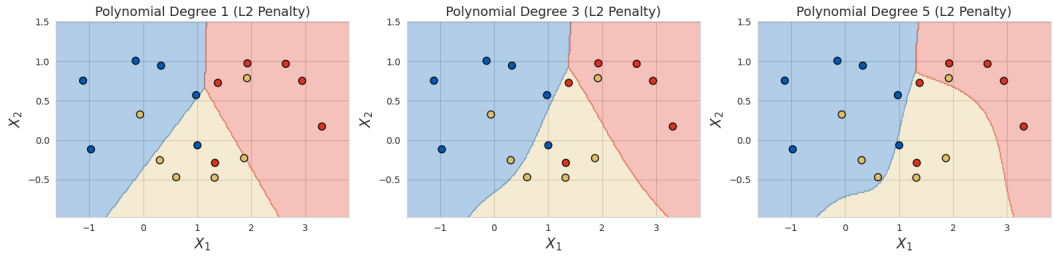


Figure 11: *Classifiers with Polynomial Basis and L2-Regularized Weights.* The sequence of plots shows the resulting decision boundaries when we increase the degree of the polynomial basis function.

tive performance. Yet this desire is counter balanced by cross validation, as it allows us to see if that extra capaticity is being used to model signal or just noise that is specific to the training set.

3.3 Capacity Control

While evaluating on held-out data is perhaps the best method for controlling model capacity and selecting the appropriately powerful model, it is expensive since it is data driven, and high-quality data is all but always expensive. There exists some general techniques for controlling model capacity without data, and I will give an brief overview of two here: regularization penalties and ensembling.

Regularization Penalty Regularization penalties are an additional term that is added to the loss function that penalizes more complex models:

$$\widetilde{\ell}(\mathbf{w}; \mathcal{D}, \lambda) \;=\; \ell(\mathbf{w}; \mathcal{D}) \;+\; \lambda \cdot \mathcal{R}(\mathbf{w})$$

where $\ell(\mathbf{w}; \mathcal{D})$ is the original training loss function, which will purely reward the model's fit to the training data, $\lambda \in \mathbb{R}^+$ is a hyperparameter that controls the strength of the regularization, or in other words, the relative trade-off between data fit and complexity

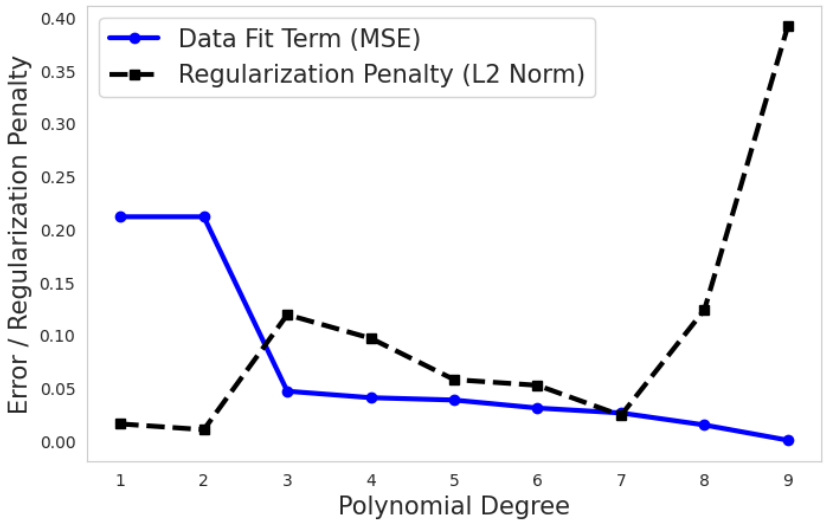


Figure 12: *Data Fit vs Complexity Penalty.* The sequence of plots shows the resulting decision boundaries when we increase the degree of the polynomial basis function.

control, and $\mathcal{R}(\mathbf{w})$ is the regularization penalty that rewards simpler models. One very common and easy way to control a model's capacity is simply by penalizing the weights from moving away from zero. A common implementation is to choose the $L2$ norm:

$$\widetilde{\ell}(\mathbf{w}; \mathcal{D}, \lambda) \;=\; \ell(\mathbf{w}; \mathcal{D}) \;+\; \lambda \cdot \sum_{d=1}^{D} \mathrm{w}_d^2,$$

where the square is applied so that we equally penalize large positive and negative weights. Figure 12 shows the same example of categorical regression with a polynomial basis, but now an L2 penalty has been applied to the weights. Comparing these decision boundaries to the ones in the first row of Figure 5, we see that the decision boundaries are relatively smoother and more closely resemble that of the first-order model's straight-line boundaries. Figure 12 shows how the two terms interact in a real-valued regression example; the data-fit term is in blue, adnd the regularization penalty (without λ) is the black dotted line. We see that at first, the regularization penalty is low as the data fit is poor. Then at degree three, the data fit improves dramatically yet the regularization penalty jumps up as well, indicating a more complex model has been found. The two stay relatively level until degree seven when the data fit makes modest improvements but the regularization penalty skyrockets, indicating that the gains in data fit are only coming at the 'cost' of using increasingly complex models—which is likely a sign of overfitting and poor model generalization. This information is nice in that we can see some hints at how well-fit the model is without using held-out validation data. However, these insights are not as robust as looking at the error on held-out data and also comes at the price of having another hyperparameter to choose (λ). Fortunately, the model is usually less sensitive to the setting of λ than the polynomial degree. If this were not the case, then one would need to intensively cross-validate λ, and if one needs to do that, that data might just as well be used to tune the polynomial degree directly.

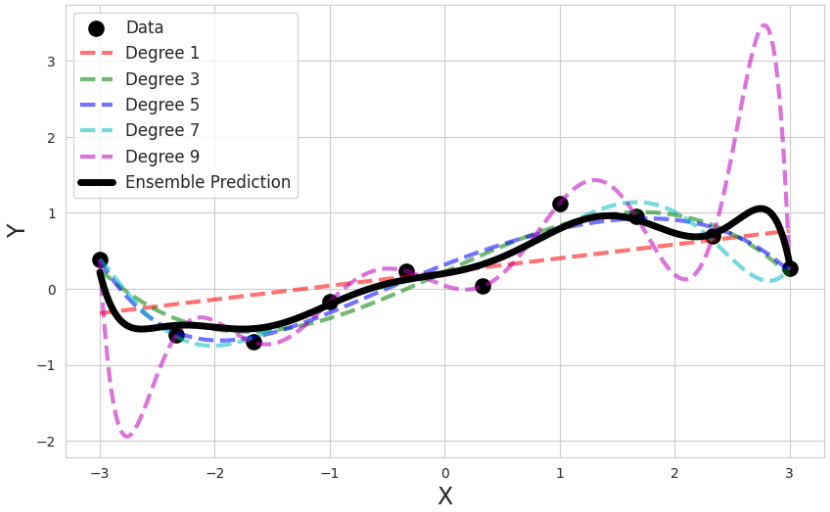


Figure 13: *Ensemble of Polynomial Regressors of Varying Degrees.*

Ensembling Another technique that is easy to implement and controls complexity is *ensembling* a collection of models. This is where we train several models, and instead of using just one to make predictions, we combine the predictions of all models. The intuition behind why this is beneficial is that 'overfit models often overfit in different ways,' and therefore, the differences across models cancel out, leaving only the similarities that correspond to true signal. For real-valued regression, models can be ensembled simply by averaging their mean functions:

$$\bar{f}(\mathbf{x}; \mathbf{w}_1, \ldots, \mathbf{w}_J) \;=\; \frac{1}{J}\sum_{j=1}^{J} \mathbb{E}\left[\mathrm{y}|f(\mathbf{x}; \mathbf{w}_j)\right] \;=\; \frac{1}{J}\sum_{j=1}^{J} \mathbf{w}_j^T\mathbf{x}$$

where J is the total number of models in the ensemble. Notice that for linear models, averaging the mean functions is equivalent to averaging the model parameters: $\frac{1}{J}\sum_{j=1}^{J}\mathbf{w}_j^T\mathbf{x} = \left(\frac{1}{J}\sum_{j=1}^{J}\mathbf{w}_j^T\right)\mathbf{x}$; though this is not true in general (e.g. for logistic regression). Figure 13 shows an ensemble of real-value regressors using a polynomial basis that ranges over degrees 1, 3, 5, 7, and 9 (colored dashed lines). The ensemble output is shown by the black solid line. Note how the black line represents a flexible but restrained model fit—going through more data points than most of the simple models but doesn't have the drastic turns of the 9-degree polynomial.

For categorical regression models, there are two ways to ensemble them. The first is called *voting*: each model makes a prediction, and the class that was predicted most frequently across the ensemble is the aggregate prediction:

$$\bar{f}_{\text{vote}}(\mathbf{x}; \mathbf{w}_1, \ldots, \mathbf{w}_J) \;=\; \underset{k\in[1,K]}{\arg\max} \sum_{j=1}^{J} \hat{y}_{j,k}$$

where $\hat{y}_{j,k}$ is the one-hot-encoded prediction for label k from model j. Another technique is to aggregate the outputs of the inverse link functions directly. For example, for the softmax

we have:

$$\bar{f}_{\text{soft}}(\mathbf{x}; \mathbf{w}_1, \ldots, \mathbf{w}_J) \;=\; \underset{k \in [1,K]}{\arg\max} \sum_{j=1}^{J} \texttt{softmax}_k \left(f(\mathbf{x}; \mathbf{w}_j) \right)$$

where $\texttt{softmax}_k(\cdot)$ denotes the softmax's output for class k. Ensembling is attractive since it doesn't require too much thought: as long as we can train a sufficiently diverse set of models, then it should would and not be too sensitive to the settings of any one particular model. However, the drawback comes at the cost of computation. Not only do we have to train J models, we usually need to deploy all J models to the real world as well, since unlike with linear models for real-valued regression, there is usually not a way to combine all J sets of parameters into one object while maintaining the full benefits of the ensemble.

4 Feedforward Neural Networks

We have so far only considered 'shallow' models—namely, linear models. We have worked with shallow and 'narrow' models defined by have a fixed, given feature set. Yet we have also considered shallow and 'wide' models obtained by having a dynamic feature set, e.g. a polynomial basis of arbitrary degree. Now we will consider 'deep' models that will adapt and change the feature basis via learning. Or in other words, the feature basis itself will only have a very generic form, and the data itself will guide how the features are transformed into different representations. While we saw that polynomial basis functions can represent very complex functions, the degree of the polynomial is determined by us and has a very specific influence on the model. With deep models, we will not have to make such strong choices about what is the best representation space of the features.

4.1 Adaptive Features and the Importance of Non-Linearities

The key idea behind *deep learning* is to have a feature representation (or basis) that is flexible and determined by the data, not by the model builder's assumptions. Thus, in general, we can think of deep models as having the general form:

$$\mathbb{E}[\mathrm{y}|\mathbf{x}] \;=\; g^{-1}\left(\mathbf{w}^T \psi(\mathbf{x})\right) \tag{20}$$

where g is the link function (as usual), $\mathbf{w}$ are parameters to be learned, and $\psi(\mathbf{x})$ is a new representation of $\mathbf{x}$ that *depends on* $\mathbf{x}$ *itself and will have its own parameters.* This is unlike in polynomial regression in that the feature basis was fixed across all possible feature vectors, though would change with the specifics of the features themselves.

So how should we build this adaptive basis $\psi(\mathbf{x})$? The first idea one might have is to start simple and make $\psi(\mathbf{x})$ a linear model as well:

$$\psi(\mathbf{x}; \mathbf{U}) \;=\; \mathbf{U}^T \mathbf{x}$$

where $\mathbf{U} \in \mathbb{R}^{D \times D'}$ is a matrix of real-values that transforms $\mathbf{x}$ from a D-dimensional representation into a new representation of size D'. Plugging this expression into the model above, we have:

$$\mathbb{E}[\mathrm{y}|\mathbf{x}] \;=\; g^{-1}\left(\mathbf{w}^T \psi(\mathbf{x}; \mathbf{U})\right) \;=\; \mathbf{w}^T \left(\mathbf{U}^T \mathbf{x}\right),$$

which means that we would first transform the feature vector with $\mathbf{U}$ and then multiply it with $\mathbf{w}$.

However, there is a problem with the above construction: this model is no more flexible and expressive than the original model $g^{-1}\left(\mathbf{w}^T\mathbf{x}\right)$! This is because a composition of linear transformations is no more expressive than one linear transformation, i.e. $\mathbf{w}^T\left(\mathbf{U}^T\mathbf{x}\right) = \left(\mathbf{w}^T\mathbf{U}^T\right)\mathbf{x}$, which could be equivalently represented just with a linear transformation of the same shape / size as $\mathbf{w}$. Thus, to break out of the space of linear transformations, we need to introduce some non-linear transform. We'll call it $\phi(\mathbf{z})$, which usually acts element-wise:

$$\phi(\mathbf{z}) \;=\; [\phi(z_1), \ldots, \phi(z_D)]\,, \qquad \mathbf{z} \in \mathbb{R}^D.$$

Now introducing this non-linear function into the model we had before, we have:

$$\mathbb{E}[\mathrm{y}|\mathbf{x}] \;=\; g^{-1}\left(\mathbf{w}^T\psi(\mathbf{x}; \mathbf{U}, \phi(\cdot))\right) \;=\; \mathbf{w}^T\phi\left(\mathbf{U}^T\mathbf{x}\right).$$

By definition of ϕ being non-linear, we have successfully made it so that $\mathbf{w}^T\phi\left(\mathbf{U}^T\mathbf{x}\right) \neq \phi\left(\mathbf{w}^T\mathbf{U}^T\right)\mathbf{x}$. Of course, there are many, many choices of $\phi(\cdot)$ for which the above holds. Next we will see some specific implementations.

4.2 Neural Networks

Neural networks (NNs) (a.k.a. *artificial neural networks*) are a specific implementation of the adaptive-basis-function framework described above. Really, the only thing left to introduce is the specific terminology that has become attached to them due to the influences and inspirations of biological intelligence.

Terminology Keeping the same notation as above, both $\mathbf{w}$ and $\mathbf{U}$ are called 'weights', just like in regular linear regression, and are parameters that need to be learned from data. Again, this is why NNs can potentially be very powerful: they learn an adaptive representation from the data. The non-linear transform $\phi(\cdot)$ is called the 'activation function' or 'transfer function,' with the former name arising from the idea of biological neuron 'activating' or 'firing'. Thus, one element of the new representation $\phi_d(\mathbf{U}^T\mathbf{x})$ is usually called a 'neuron' or 'hidden unit.' The collection of all elements $\phi(\mathbf{U}^T\mathbf{x}) = \left[\phi_1(\mathbf{U}^T\mathbf{x}), \ldots, \phi_D(\mathbf{U}^T\mathbf{x})\right]$ is called a 'hidden layer.' The description 'hidden' emphasizes that (i) the representation is neither directly the input or output variables but rather some intermediate variables, and (ii) we do not exactly know how to interpret $\phi(\mathbf{U}^T\mathbf{x})$ since the representation is more free-form than, say, a polynomial basis. Another common term is to call the value $\mathbf{U}^T\mathbf{x}$ the pre-activation (vector) since it serves as the input to the activation function ϕ. To make things a bit more confusing, NNs are sometimes referred to as *multilayer perceptrons* (MLPs) as they can be seen as stacking (or more precisely, composing) multiple perceptron architectures. Lastly, we call a NN of this form 'fully connected' since all input values can influence all of the hidden units, and all hidden units can influence the output(s).

Activation Functions The choice of activation function ϕ is actually quite crucial to the success of NNs. We will briefly introduce some popular choices here and revisit the topic later, after talking about how to train NNs. See Figure for an overview of four activation

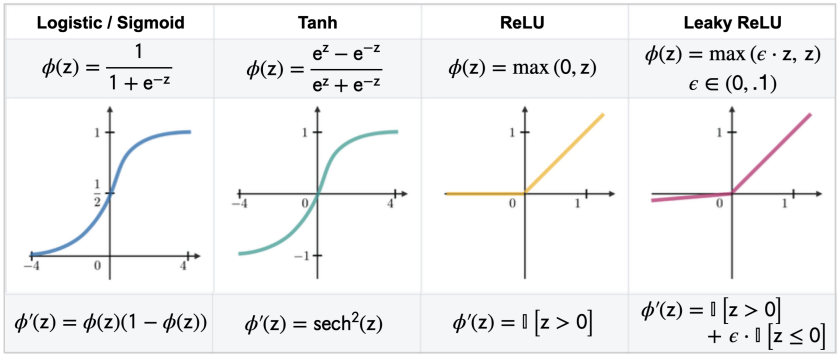


Figure 14: *Common Activation Functions.* Four functions that are commonly used for the activation functions of a NN's hidden layers. Their derivatives are given in the bottom row.

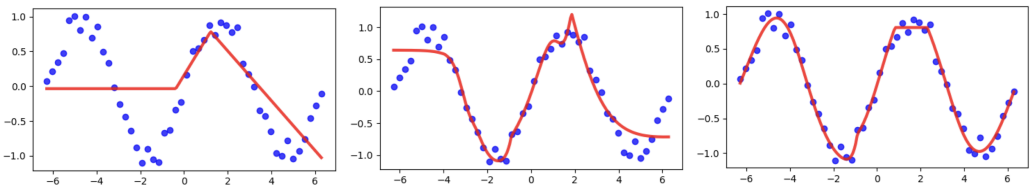
functions. One we have already seen before: the logistic (sigmoid) function. The second one from the left, the hyperbolic tangent function (tanh), looks similar to the logistic in shape, but notice that it's range is from $(-1, 1)$, being symmetrical about the x-axis, whereas the logistic's output is never negative. The last two are variants of the *rectified linear unit* (ReLU). In the simplest implementation, the ReLU simply preserves the pre-activation value when it is positive sets it to zero when negative. The Leaky ReLU behaves similarly, except that it allows some information to 'leak' to the left of zero. As we will discuss more later, this is to make gradient-based training a bit easier. Note that, though the ReLU and Leaky RelU are piecewise linear functions with just two pieces (meaning that they are linear functions one one side of the origin), this rather small amount of non-linearity is still sufficient to prevent NN's from collapsing back to linear functions, as would happen if we used the identity function as the activation.

4.3 Deep Neural Networks

In the preceding subsection, we introduced a NN with one hidden layer. But why stop there? One can simply repeat the process of applying a linear transformation followed by a non-linear activation function to create NNs of arbitrary 'depth'. We can then define *deep NNs* (DNNs) of depth $(L+2)$—with L of the layers being hidden—via the following two equations:

$$\mathbb{E}[\mathrm{y}|\mathbf{x}] \;=\; g^{-1}\left(\mathbf{w}_L^T \mathbf{h}_L\right), \quad \mathbf{h}_l \;=\; \phi\left(\mathbf{W}_{l-1}^T \mathbf{h}_{l-1}\right), \quad \text{where} \;\; \mathbf{h}_0 = \mathbf{x} \in \mathbb{R}^{D_0}, \tag{21}$$

$\mathbf{w}_L \in \mathbb{R}^{D_L}$ (or a $(D_L \times K)$-matrix for K dimensional outputs), $\mathbf{h}_l$ is of dimension D_l, and $\mathbf{W}_{l-1} \in \mathbb{R}^{D_{(l-1)} \times D_l}$. Yet note that the variable $\mathbf{h}_l$ is really just notation for the function composition. This goes back to the above point: the hidden layers are really just computational operations that allow us to define a more expressive model, and they do not have the strong semantic interpretations that the input and output 'layers' have. A DNN



(a) 1 Hidden Layer (b) 2 Hidden Layers (c) 3 Hidden Layers

Figure 15: *Increasing the Depth of a Neural Network.* The three plots above show when neural networks of increasing depth are fit to the sine function. All hidden layers have three hidden units that use Tanh activation functions.

can be equivalently written just in terms of the function compositions:

$$\begin{aligned}
\mathbb{E}[\mathrm{y}|\mathbf{x}] &= g^{-1}\left(\mathbf{w}_L^T \mathbf{h}_L\right) \\
&= g^{-1}\left(\mathbf{w}_L^T \phi\left(\mathbf{W}_{L-1}^T \mathbf{h}_{L-1}\right)\right) \\
&= g^{-1}\left(\mathbf{w}_L^T \phi\left(\mathbf{W}_{L-1}^T \phi\left(\mathbf{W}_{L-2}^T \mathbf{h}_{L-2}\right)\right)\right) \\
&= g^{-1}\left(\mathbf{w}_L^T \phi\left(\mathbf{W}_{L-1}^T \phi\left(\mathbf{W}_{L-2}^T \phi\left(\mathbf{W}_{L-3}^T \mathbf{h}_{L-3}\right)\right)\right)\right) \\
&\vdots
\end{aligned} \tag{22}$$

and so on, continuing to write each hidden layer in terms of the previous one until we stop upon reaching $\mathbf{h}_0 = \mathbf{x}$. Starting from the input layer and evaluating all hidden layers to produce an output is called *forward propagation*. I have written the above expression as having the same activation function applied at every depth in the NN. This is usually the case, but nothing is preventing us from having different activation functions at different layer (or even within a layer). Doing that could possibly improve the model in cases, but cross-validating all of those choices / configurations would probably not be worth the effort. Choosing the depth and width of the layers is usually a more critical hyperparameter to tune. Figure 15 demonstrates the power of adding more hidden layers, as the harmonic function can be better and better approximated as the NN goes from one to three hidden layers.

Incorporating the Offset Parameter Recall that for linear models, we assumed that the input feature vector always had a constant element of one so that we could implement the offset / bias / intercept parameter just by multiplication. We want to do the same thing for DNNs, and therefore at every hidden layer, we need to assume a constant value of one is appended to the hidden layer:

$$\mathbf{h}_l = \left[\phi\left(\mathbf{W}_{l-1}^T \mathbf{h}_{l-1}\right),\ 1\right].$$

Going forward, we will assume the hidden layers are always implemented using this definition unless otherwise stated.

Why depth vs width? Before we get too excited about making our NNs as deep as possible, it is worth considering: why can't we making our NNs powerful just by making one or two hidden layers really wide? Of course, we are allowed to pick the dimensionality

of the hidden layers, so nothing is stopping us from making them on the order of a thousand or even a million units wide. In fact, there is actually a theorem called the *Universal Approximation Theorem*[1] that says that, if you can make the NN wide enough, it can approximate just about any function you would possibly ever need in practice. Thus, at least in theory, depth is not necessary to define expressive NNs. *However*, the theorem does not prescribe how wide you need to make the NN, and in fact, you may need exponentially more hidden units than you have feature dimensions. There has also been theoretical work showing that, for a fixed number of parameters, a deeper network can represent functions that the corresponding wide network cannot (Telgarsky, 2016). Yet the simple answer is that people have found that increasing an NN's depth is a much more effective way to use parameters and just generally works better in practice. In the future, perhaps we will have new architectures or learning algorithms that better take advantage of width than depth.

4.4 Gradient-Based Learning of Neural Networks

We can learn the parameters of a neural network via gradient descent, following a similar recipe to how we implemented it for linear models. We will use the same loss functions as considered earlier—such as squared error or cross-entropy error. For example, if we are using a DNN with L hidden layers for a real-valued regression task $\mathrm{y} \in \mathbb{R}$, we would have:

$$\begin{aligned}
\ell\left(\mathbf{W}_0, \ldots, \mathbf{W}_l, \ldots, \mathbf{w}_L; \mathcal{D}\right) &= \frac{1}{N} \sum_{n=1}^{N} \left(\mathbb{E}[y_n | \boldsymbol{x}_n] - y_n\right)^2 \\
&= \frac{1}{N} \sum_{n=1}^{N} \left(\mathbf{w}_L^T \boldsymbol{h}_{n,L} - y_n\right)^2 \\
&= \frac{1}{N} \sum_{n=1}^{N} \left(\mathbf{w}_L^T \phi\left(\mathbf{W}_{L-1} \boldsymbol{h}_{n,L-1}\right) - y_n\right)^2 \\
&\vdots
\end{aligned} \tag{23}$$

and so one, writing each hidden layer in terms of the previous one. $\mathbf{W}_0, \ldots, \mathbf{W}_l, \ldots, \mathbf{w}_L$ are all parameters in the DNN. Given a loss function, we then need to compute the gradient of the weights at all layers ($\mathbf{W}_l \ \forall l$):

$$\mathbf{W}_l^{t+1} \ = \ \mathbf{W}_l^t \ - \ \alpha \cdot \nabla_{\mathbf{W}_l^t} \, \ell\left(\mathbf{W}_0^t, \ldots, \mathbf{W}_l^t, \ldots, \mathbf{w}_L^t; \mathcal{D}\right) \tag{24}$$

where t indexes the gradient descent iteration.

Example: Scalar NN Let's build our intuition by working out the gradient equation for a simple NN in which the weights are all scalars. Moreover, let's assume the task is real-valued regression, and the NN has two hidden layers, no offset parameters, scalar inputs and outputs, and logistic activations. It can be implemented like so:

$$\mathbb{E}[\mathrm{y}|\mathrm{x}] = \mathrm{w}_2 \cdot \mathrm{h}_2, \quad \mathrm{h}_2 = \phi(\underbrace{\mathrm{w}_1 \cdot \mathrm{h}_1}_{\mathrm{a}_2}), \quad \mathrm{h}_1 = \phi(\underbrace{\mathrm{w}_0 \cdot \mathrm{x}}_{\mathrm{a}_1}),$$

[1] https://en.wikipedia.org/wiki/Universal_approximation_theorem

where a is an intermediate variable we will use to denote the pre-activation. Starting with the derivative for w_2, we have:

$$\begin{aligned}\frac{d}{d\mathrm{w}_2}\,\ell(\mathrm{w}_2,\mathrm{w}_1,\mathrm{w}_0;\mathcal{D}) &= \frac{1}{N}\sum_{n=1}^{N}\frac{d\,\ell_n}{d\,\mathbb{E}[y_n|x_n]}\frac{d\,\mathbb{E}[y_n|x_n]}{d\,\mathrm{w}_2}\\ &= \frac{1}{N}\sum_{n=1}^{N}\underbrace{2\cdot((\mathrm{w}_2\cdot h_{n,2})-y_n)}_{\frac{d\,\ell_n}{d\,\mathbb{E}[y_n|x_n]}}\underbrace{h_{n,2}}_{\frac{d\,\mathbb{E}[y_n|x_n]}{d\,\mathrm{w}_2}}\end{aligned}$$

Moving on to the derivative for w_1, we have:

$$\begin{aligned}\frac{d}{d\mathrm{w}_1}\,\ell(\mathrm{w}_2,\mathrm{w}_1,\mathrm{w}_0;\mathcal{D}) &= \frac{1}{N}\sum_{n=1}^{N}\frac{d\,\ell_n}{d\,\mathbb{E}[y_n|x_n]}\frac{d\,\mathbb{E}[y_n|x_n]}{d\,h_{n,2}}\frac{d\,h_{n,2}}{d\,a_{n,2}}\frac{d\,a_{n,2}}{d\,\mathrm{w}_1}\\ &= \frac{1}{N}\sum_{n=1}^{N}\underbrace{2\cdot((\mathrm{w}_2\cdot h_{n,2})-y_n)}_{\frac{d\,\ell_n}{d\,\mathbb{E}[y_n|x_n]}}\underbrace{\mathrm{w}_2}_{\frac{d\,\mathbb{E}[y_n|x_n]}{d\,h_{n,2}}}\underbrace{h_{n,2}\cdot(1-h_{n,2})}_{\frac{d\,h_{n,2}}{d\,a_{n,2}}}\underbrace{h_{n,1}}_{\frac{d\,a_{n,2}}{d\,\mathrm{w}_1}}\end{aligned}$$

Then finally for w_0, we have:

$$\begin{aligned}&\frac{d}{d\mathrm{w}_0}\,\ell(\mathrm{w}_2,\mathrm{w}_1,\mathrm{w}_0;\mathcal{D})\\ &\quad= \frac{1}{N}\sum_{n=1}^{N}\frac{d\,\ell_n}{d\,\mathbb{E}[y_n|x_n]}\frac{d\,\mathbb{E}[y_n|x_n]}{d\,h_{n,2}}\frac{d\,h_{n,2}}{d\,a_{n,2}}\frac{d\,a_{n,2}}{d\,h_{n,1}}\frac{d\,h_{n,1}}{d\,a_{n,1}}\frac{d\,a_{n,1}}{d\,\mathrm{w}_0}\\ &\quad= \frac{1}{N}\sum_{n=1}^{N}\underbrace{2\cdot((\mathrm{w}_2\cdot h_{n,2})-y_n)}_{\frac{d\,\ell_n}{d\,\mathbb{E}[y_n|x_n]}}\underbrace{\mathrm{w}_2}_{\frac{d\,\mathbb{E}[y_n|x_n]}{d\,h_{n,2}}}\underbrace{h_{n,2}\cdot(1-h_{n,2})}_{\frac{d\,h_{n,2}}{d\,a_{n,2}}}\underbrace{\mathrm{w}_1}_{\frac{d\,a_{n,2}}{d\,h_{n,1}}}\underbrace{h_{n,1}\cdot(1-h_{n,1})}_{\frac{d\,h_{n,1}}{d\,a_{n,1}}}\underbrace{x}_{\frac{d\,a_{n,1}}{d\,\mathrm{w}_0}}\end{aligned}$$

There are a few observations to make here. The first is that the $\frac{d\,\ell_n}{d\,\mathbb{E}[y_n|x_n]}$ partial derivative is present in all of the calculations, which makes sense since this is what connects the NN to the label y_n to provide the supervision signal. From there, the signal flows backward into one or more hidden layers. Next we see that the partial derivatives $\frac{d\,\mathbb{E}[y_n|x_n]}{d\,h_{n,2}}$ and $\frac{d\,h_{n,2}}{d\,a_{n,2}}$ are present in the total derivatives for w_1 and w_0. Again, this makes sense since these parameters reside earlier in the model than both $\mathbb{E}[y_n|x_n]$ and $h_{n,2}$. This suggests that, in our computational implementations, there are savings to be had by caching these partial derivatives for multiple use.

4.5 Backpropagation

As we saw with the example of a scalar NN, we can think of the prediction error signal as flowing backwards from the loss function towards the input layer. This is intuitive: if *forward propagation* sends the signal from the input to the DNN output, then *backpropagation* sends the signal from the output (i.e. the deepest part of the network) towards to shallow layers. This can be seen in the general form of the chain rule as applied above:

$$\frac{d}{d\mathbf{W}_l}\ell\left(\mathbf{W}_0,\ldots,\mathbf{W}_l,\ldots,\mathbf{w}_L;\mathcal{D}\right) = \frac{d\ell}{d\mathbb{E}[\mathrm{y}|\mathbf{x}]}\frac{d\mathbb{E}[\mathrm{y}|\mathbf{x}]}{d\mathbf{h}_L}\frac{d\mathbf{h}_L}{d\mathbf{h}_{L-1}}\frac{d\mathbf{h}_{L-1}}{d\mathbf{h}_{L-2}}\cdots\frac{d\mathbf{h}_{l+2}}{d\mathbf{h}_{l+1}}\frac{d\mathbf{h}_{l+1}}{d\mathbf{W}_l}. \tag{25}$$

Notice the repeated structure of $(d\mathbf{h}_{l+1}/d\mathbf{h}_l)$, which means that all parameter involved in computation of $\mathbf{h}_l$ share all the same partial derivatives required to compute $(d\ell/d\mathbf{h}_l)$! Thus we trade off computational complexity for memory.

Forward vs Reverse Mode Give the sequence of partial derivatives above, you can evaluate them in two different orders: left-to-right or right-to-left, i.e.:

$$\left(\left(\frac{d\ell}{d\mathbf{h}_L}\,\frac{d\mathbf{h}_L}{d\mathbf{h}_{L-1}}\right)\frac{d\mathbf{h}_{L-1}}{d\mathbf{h}_{l+1}}\right)\frac{d\mathbf{h}_{l+1}}{d\mathbf{W}_l} \quad \text{vs} \quad \frac{d\ell}{d\mathbf{h}_L}\left(\frac{d\mathbf{h}_L}{d\mathbf{h}_{L-1}}\left(\frac{d\mathbf{h}_{L-1}}{d\mathbf{h}_{l+1}}\,\frac{d\mathbf{h}_{l+1}}{d\mathbf{W}_l}\right)\right).$$

The left-to-right version starts the computation at the last layer and therefore is known as *reverse mode*. This is the typical way that backpropagation is implemented since, again, the partial derivatives are computed starting at the deepest layers. This is most efficient when the output dimension is smaller than the input dimension—which is usually the case—since the highest-dimensional (sub)gradients will be computed last. *Forward mode* is the right-to-left version, and it is more computationally efficient when the input dimension is smaller than the output dimension, which follows the same logic that motivates reverse mode.

Exploding and Vanishing Gradients The shared partial derivatives that makes backpropagation computationally efficient also has a downside: if any of the intermediate derivatives $(d\mathbf{h}_{l+1}/d\mathbf{h}_l)$ are problematic, then this problem will propagate to all parameters that reside at all earlier layers. Due to the multiplicative structure of the chain rule, by 'problematic' we usually mean we are worried about derivatives that are very large or very small. The former is called the *exploding gradient* problem, and the latter is called the *vanishing gradient* problem. To make the vanishing gradient version explicit, let's assume that partial derivative $(d\mathbf{h}_{l+1}/d\mathbf{h}_l) \approx 0$. We then have that:

$$\frac{d\ell}{d\mathbf{h}_L}\,\frac{d\mathbf{h}_L}{d\mathbf{h}_{l+1}}\,\cancel{\frac{\mathbf{h}_{l+1}}{\mathbf{h}_l}}^{\approx 0}\,\frac{\mathbf{h}_l}{\mathbf{h}_{l-1}}\,\frac{\mathbf{h}_{l-1}}{\mathbf{h}_{l-2}}\cdots \approx 0$$

for any parameter at a layer earlier than $\mathbf{h}_l$. The same logic would follow for exploding gradients: one really large partial derivative can blow up the whole chain. For many years, it was difficult to train DNNs for this very problem, as we did not yet have the tools and tricks to effectively stabilize the gradients across multiple hidden layers.

Saturating Activation Functions One common cause of vanishing gradients is what's call 'saturating' activation functions. This means that the input to the activation function is in a regime in which the output of the function is relatively flat and therefore has a derivative near zero. Recall the logistic activation function's derivative: $\phi'(\mathrm{z}) = \phi(\mathrm{z})(1-\phi(\mathrm{z}))$. How can the logistic saturate and cause the gradient signal to vanish? This would happen when $\phi(\mathrm{z}) \approx 0$ or $\phi(\mathrm{z}) \approx 1$. Next consider the ReLU activation; when would it saturate? As it's derivative is $\phi'(\mathrm{z}) = \mathbb{I}[\mathrm{z} > 0]$, the function can saturate when the input is negative. In fact, when many of a DNN's ReLUs are evaluating to zero, this is called having 'dead ReLUs.' However, unlike the logistic, the ReLU only saturates to one side of the origin, not both (like the logistic and tanh). Lastly, consider the Leaky RelU. The hyperparameter ϵ prevents it from ever having a derivative of zero. Rather, for negative inputs the derivative is simply

'small.' Thus, the Leaky ReLU can never fully saturate.

4.5.1 Vectorized Implementation

When implementing DNNs in practice, we often want to make use of matrix / vector products as much as possible, since these are fast to implement with modern software and on graphics processing units. The implementation I have given above aims to follow the model definition given for linear models, but to reduce computational overhead, we'd like to make some modest changes in practice. Firstly, let's assume that the feature vectors are D-dimensional and the labels are one-hot-encoded K-dimensional vectors. We will then assume we see N training points, which we can aggregate into matrices $\boldsymbol{X}$ and $\boldsymbol{Y}$ with both have N rows, i.e.:

$$\boldsymbol{X} = \begin{bmatrix} \boldsymbol{x}_1^T \\ \vdots \\ \boldsymbol{x}_N^T \end{bmatrix}, \qquad \boldsymbol{Y} = \begin{bmatrix} \boldsymbol{y}_1^T \\ \vdots \\ \boldsymbol{y}_N^T \end{bmatrix}.$$

Moving on to the weight parameters $\mathbf{W}_l$, originally we defined the NN as requiring $\mathbf{W}_l$ to undergo a transpose operation—again, to follow the GLM implementation. But in practice, we can simply implement the hidden layers using the product in the other direction in order to keep the data points as the rows and the hidden units as the columns:

$$\boldsymbol{H}_l \;=\; \phi\left(\boldsymbol{H}_{l-1}\mathbf{W}_{l-1}\right)$$

where $\boldsymbol{H}_{l-1}$ is of size $(N \times D_{l-1})$, $\mathbf{W}_{l-1}$ is of size $(D_{l-1} \times D_l)$, and $\boldsymbol{H}_l$ is of size $(N \times D_l)$. Finally, for the output layer and loss, we have:

$$\ell\left(\mathbf{W}_0, \ldots, \mathbf{W}_L; \boldsymbol{X}, \boldsymbol{Y}\right) \;=\; -\frac{1}{N} \cdot \mathbf{1}^T \left(\boldsymbol{Y} \odot \log \texttt{softmax}\left(\boldsymbol{H}_L\mathbf{W}_L\right)\right)\mathbf{1}$$

where $\odot$ denotes the element-wise product and $\mathbf{1}^T(\cdot)\mathbf{1}$ is just a fancy way of writing that we are going to sum over all $N \times K$ elements of $\boldsymbol{Y} \odot \log \texttt{softmax}\left(\boldsymbol{H}_L\mathbf{W}_L\right)$. While this product has $N \times K$ elements, only N are active since there is only one 1 in each row of $\boldsymbol{Y}$.

Binary classification and real-valued regression would be setup in similar ways. For example, for real-valued regression, the loss function would be

$$\ell\left(\mathbf{W}_0, \ldots, \mathbf{W}_L; \boldsymbol{X}, \boldsymbol{Y}\right) \;=\; -\frac{1}{N} \cdot \mathbf{1}^T \left(\boldsymbol{H}_L\mathbf{W}_L - \mathbf{Y}\right)^2 \mathbf{1}$$

where $\boldsymbol{Y} \in \mathbb{R}^{N \times K}$ for K output dimensions and when assuming that the underlying Normal distribution has a diagonal covariance matrix and $(\cdot)^2$ acts element-wise.

General Form for Backpropagation We can derive a general form for the vectorized version of backpropagation's gradient equation. Recall that for many of the nice choices of inverse link functions discussed in this course (e.g. logistic for binary classification, softmax for multi-class classification, identity for real-valued regression), the gradient w.r.t. the model output $\mathbf{H}_L\mathbf{W}_L$ (i.e. the output before the link is applied) has the nice form:

$$\nabla_{\mathbf{H}_L\mathbf{W}_L}\, \ell\left(\mathbf{W}_0, \ldots, \mathbf{W}_L; \boldsymbol{X}, \boldsymbol{Y}\right) \;=\; \frac{1}{N}\left(\mathbb{E}\left[\mathbf{Y}|\mathbf{X}\right] - \boldsymbol{Y}\right)^T$$

where the transpose is taken because we define the gradient to be in row orientation. Taking the gradient one-step further into the hidden units, we have in column-form:

$$\frac{1}{N}\left(\mathbb{E}\left[\mathbf{Y}|\mathbf{X}\right]-\boldsymbol{Y}\right)\mathbf{W}_L^T \ \in \mathbb{R}^{N\times D_L}.$$

Then the activation function is applied element-wise, which means the gradient has an element-wise product:

$$\frac{1}{N}\left(\mathbb{E}\left[\mathbf{Y}|\mathbf{X}\right]-\boldsymbol{Y}\right)\mathbf{W}_L^T \odot \phi'(\mathbf{A}_L) \ \in \mathbb{R}^{N\times D_L}.$$

The backpropagating one step further into $\mathbf{H}_{L-1}$, we have:

$$\frac{1}{N}\left(\mathbb{E}\left[\mathbf{Y}|\mathbf{X}\right]-\boldsymbol{Y}\right)\mathbf{W}_L^T \odot \phi'(\mathbf{A}_L)\mathbf{W}_{L-1}^T \ \in \mathbb{R}^{N\times D_{L-1}}.$$

Then once again through the hidden units we have:

$$\frac{1}{N}\left(\mathbb{E}\left[\mathbf{Y}|\mathbf{X}\right]-\boldsymbol{Y}\right)\mathbf{W}_L^T \odot \phi'(\mathbf{A}_L)\mathbf{W}_{L-1}^T \odot \phi'(\mathbf{A}_{L-1}) \ \in \mathbb{R}^{N\times D_{L-1}}.$$

Notice how the $\mathbf{W}^T \ \phi'(\mathbf{A})$ structure is repeated. This means that for a weight matrix at arbitrary depth l, we have the general form:

$$\nabla_{\mathbf{W}_l}\,\ell\left(\mathbf{W}_0,\ldots,\mathbf{W}_L;\boldsymbol{X},\boldsymbol{Y}\right) \;=\; \frac{1}{N}\left[\left(\mathbb{E}\left[\mathbf{Y}|\mathbf{X}\right]-\boldsymbol{Y}\right)\prod_{j=L}^{l+1}\mathbf{W}_j^T\odot\phi'(\mathbf{A}_j)\right]^T\mathbf{H}_l, \tag{26}$$

where switching back to row form allows the chain rule to be expressed just as left-to-right matrix multiplication. The term $\left[\left(\mathbb{E}\left[\mathbf{Y}|\mathbf{X}\right]-\boldsymbol{Y}\right)\prod_{j=L}^{l+1}\mathbf{W}_j^T\odot\phi'(\mathbf{A}_j)\right]$ is of size $N\times D_{l+1}$. Transposing it gives a matrix of size $D_{l+1}\times N$. $\mathbf{H}_l$ is of size $N\times D_l$. Thus the resulting is product is of size $D_{l+1}\times D_l$, which exactly the size of $\mathbf{W}_l^T$, which again is what we should expect by assuming the row-form of the gradient.

4.5.2 Skip Connections

While using (Leaky) ReLU activations helps with the vanishing gradient problem, it is not a foolproof solution. A simple yet robust solution called *skip connections* (a.k.a. *residual connections*) that is now ubiquitous was proposed and popularized by He et al. (2016). The idea is that the non-linear transformation should not directly parameterize the hidden units but rather an additive change from the previous hidden units:

$$\mathbf{h}_l \;=\; \phi\left(\mathbf{W}_{l-1}^T\mathbf{h}_{l-1}\right) \;+\; \mathbf{h}_{l-1}. \tag{27}$$

We can see the non-linear transformation parameterizes the *residual* simply by moving the previous hidden units to the left-hand-side:

$$\mathbf{h}_l \;-\; \mathbf{h}_{l-1} \;=\; \phi\left(\mathbf{W}_{l-1}^T\mathbf{h}_{l-1}\right).$$

Parameterizing the hidden units in this way should ensure that gradient information can 'backpropagate' into previous hidden units even if $\phi\left(\mathbf{W}_{l-1}^T\mathbf{h}_{l-1}\right)$ is 'dead' or saturates to

zero because then we will just have an identity transformation: $\mathbf{h}_l \approx \mathbf{h}_{l-1}$.

Example: Scalar NN Let's again return to our scalar NN to build intuition. Still assume the task is real-valued regression, and the NN has two hidden layers, no offset parameters, scalar inputs and outputs, and logistic activations. Yet now the NN has one skip connection from h_1 to h_2:

$$\mathbb{E}[\mathrm{y}|\mathrm{x}] = \mathrm{w}_2 \cdot \mathrm{h}_2, \quad \mathrm{h}_2 = \phi(\underbrace{\mathrm{w}_1 \cdot \mathrm{h}_1}_{\mathrm{a}_2}) \; + \; \mathrm{h}_1, \quad \mathrm{h}_1 = \phi(\underbrace{\mathrm{w}_0 \cdot \mathrm{x}}_{\mathrm{a}_1}),$$

where a is an intermediate variable we will use to denote the pre-activation. The derivatives w.r.t. w_2 and w_1 stay exactly the same in their form as given above, though the underlying semantics have changed since h_2 is defined differently. The derivative w.r.t. w_0 changes to:

$$\begin{aligned}
&\frac{d}{d\mathrm{w}_0} \; \ell(\mathrm{w}_2, \mathrm{w}_1, \mathrm{w}_0; \mathcal{D}) \\
&= \; \frac{1}{N} \sum_{n=1}^{N} \; \frac{d \; \ell_n}{d \; \mathbb{E}[y_n|x_n]} \frac{d \; \mathbb{E}[y_n|x_n]}{d \; h_{n,2}} \; \left(\frac{d \; h_{n,2}}{d \; a_{n,2}} \; \frac{d \; a_{n,2}}{d \; h_{n,1}} + \frac{d \; h_{n,2}}{d \; h_{n,1}} \right) \; \frac{d \; h_{n,1}}{d \; a_{n,1}} \frac{d \; a_{n,1}}{d \; \mathrm{w}_0} \\
&= \; \frac{1}{N} \sum_{n=1}^{N} \; \frac{d \; \ell_n}{d \; \mathbb{E}[y_n|x_n]} \frac{d \; \mathbb{E}[y_n|x_n]}{d \; h_{n,2}} \; \left(\frac{d \; h_{n,2}}{d \; a_{n,2}} \; \frac{d \; a_{n,2}}{d \; h_{n,1}} + 1 \right) \; \frac{d \; h_{n,1}}{d \; a_{n,1}} \frac{d \; a_{n,1}}{d \; \mathrm{w}_0} \\
&= \; \frac{1}{N} \sum_{n=1}^{N} \underbrace{2 \cdot ((\mathrm{w}_2 \cdot h_{n,2}) - y_n)}_{\frac{d \; \ell_n}{d \; \mathbb{E}[y_n|x_n]}} \underbrace{\mathrm{w}_2}_{\frac{d \; \mathbb{E}[y_n|x_n]}{d \; h_{n,2}}} \left(\underbrace{h_{n,2} \cdot (1 - h_{n,2})}_{\frac{d \; h_{n,2}}{d \; a_{n,2}}} \underbrace{\mathrm{w}_1}_{\frac{d \; a_{n,2}}{d \; h_{n,1}}} + 1 \right) \underbrace{h_{n,1} \cdot (1 - h_{n,1})}_{\frac{d \; h_{n,1}}{d \; a_{n,1}}} \underbrace{x}_{\frac{d \; a_{n,1}}{d \; \mathrm{w}_0}} .
\end{aligned}$$

The crucial 'link' in the chain rule is the term:

$$\frac{d}{dh_{n,1}} \; [\phi(\mathrm{w}_1 \cdot h_{n,1}) \; + \; h_{n,1}] \; = \; \frac{d \; h_{n,2}}{d \; a_{n,2}} \; \frac{d \; a_{n,2}}{d \; h_{n,1}} + \frac{d \; h_{n,2}}{d \; h_{n,1}} \; = \; \frac{d \; h_{n,2}}{d \; a_{n,2}} \; \frac{d \; a_{n,2}}{d \; h_{n,1}} + 1,$$

which means that even if $\phi'(\mathrm{w}_1 \cdot h_{n,1})$ vanishes, then $(d\mathrm{h}_2/d\mathrm{h}1) = 1$, which still allows backpropagation back to the earlier layers.

Changing Dimensions The above implementation of the skip connection requires that $\mathbf{h}_l$ and $\mathbf{h}_{l-1}$ be the same dimensionality. If we wish to change dimensions from layer l to $l+1$, we need to incorporate another matrix of parameters; let's call them $\mathbf{U}$:

$$\mathbf{h}_l \; = \; \phi\left(\mathbf{W}_{l-1}^T \mathbf{h}_{l-1}\right) \; + \; \mathbf{U}^T \mathbf{h}_{l-1}$$

where $\mathbf{U} \in \mathbb{R}^{D_{l-1} \times D_l}$, which should be the same size as $\mathbf{W}_{l-1}$. The parameters $\mathbf{U}$ would also need trained with gradient descent. The skip connection should still prevent vanishing gradients just so long as $\mathbf{U} \neq \mathbf{0}$.

4.5.3 Initializations

Skip connections allow backpropagation signals to bypass hidden layers whose gradient has vanished, but that does not stop the gradient from vanishing within a hidden layer. And of course, if many of the layers' gradients vanish, we would be using an effectively shallower

DNN. To make sure the gradient computation through a particular layer does not vanish / explode, it is very important that the DNN's weights are initialized sensibly. However, the particular initialization strategy depends on choice of activation functions. We give two below: one for sigmoidal (S-shaped) activations, one for ReLUs. In both cases, the goal is to have the initialization of the weights to automatically adjust to the size of the architecture. The more parameters we have, the more likely they are to generate large values during the forward pass (especially during the early iterations of gradient descent).

Xavier Initialization For sigmoidal activations such as logistic and tanh, the following *Xavier* initialization (Glorot and Bengio, 2010) scheme is effective:

$$\mathbf{W}_l \sim \texttt{Uniform}\left(\frac{-\sqrt{6}}{\sqrt{D_l + D_{l+1}}}, \frac{\sqrt{6}}{\sqrt{D_l + D_{l+1}}}\right) \tag{28}$$

where D_l and D_{l+1} are the number of hidden units at the current and next layers. The offset / bias / intercept parameters should be initialized all to zero.

He Initialization For ReLU activations, the *He* initialization (He et al., 2016) is suitable:

$$\mathbf{W}_l \sim \texttt{Normal}\left(0, \frac{2}{D_l}\right) \tag{29}$$

where D_l are the number of units at the current hidden layer. Again, the bias / offset / intercept parameters should be set to zero.

4.5.4 Normalization Layers

While proper initialization ensures the weights are at a good setting at the start of gradient descent, nothing is stopping them from becoming numerically problematic during training. This is the problem *normalization* techniques such as *Batch Normalization* (BatchNorm) and *Layer Normalization* (LayerNorm) attempt to solve. These are typically applied before the activation function so that the input to the activation is scaled appropriately and does not saturate. Although with ReLU activations, BatchNorm has been found to be slightly more effective if applied after the activation. For both techniques defined below, assume we have a matrix of pre-activations $\mathbf{A} \in \mathbb{R}^{N \times D}$, with N being the number of instances / data points and D being the number of dimensions at this hidden layer.

BatchNorm BatchNorm applies the following normaliziation to each element $\mathrm{a}_{n,d}$ of $\mathbf{A}$:

$$\tilde{\mathrm{a}}_{n,d} = \texttt{BatchNorm}\left(\mathrm{a}_{n,d}; \mathbf{A}, \gamma_d, \beta_d\right) = \beta_d + \gamma_d \cdot \frac{a_{n,d} - \mathbb{E}[\mathrm{a}_{\cdot,d}]}{\sqrt{\mathrm{Var}[\mathrm{a}_{\cdot,d}] + \epsilon}}$$

where $\beta_d \in \mathbb{R}$ and $\gamma_d \in \mathbb{R}$, $d \in [1, D]$, are per-dimension parameters to be learned, $\epsilon \in \mathbb{R}^+$ is a small positive constant for numerical stability, $\mathbb{E}[\mathrm{a}_{\cdot,d}]$ is the mean (first moment) of the d-th dimension as computed empirically over the N samples contained within $\mathbf{A}$, and $\mathrm{Var}[\mathrm{a}_{\cdot,d}]$ is the variance (second moment) of the d-th dimension as computed empirically over the N samples contained within $\mathbf{A}$. In other words, BatchNorm performs its normalization by computing the mean and variance statistics for each column of $\mathbf{A}$. At test time, BatchNorm

is 'turned off,' meaning that the γ and β parameters are no longer updated and $\mathbb{E}[\mathrm{a}_{\cdot,d}]$ and $\mathrm{Var}[\mathrm{a}_{\cdot,d}]$ are fixed to whatever their values are for the final training iteration.

LayerNorm LayerNorm, on the other hand, computes the mean and variance parameters across *the rows of* $\mathbf{A}$. Its transformation can be written as:

$$\bar{\mathrm{a}}_{n,d} \;=\; \texttt{LayerNorm}\left(\mathrm{a}_{n,d}; \mathbf{A}, \gamma_d, \beta_d\right) \;=\; \beta_d \;+\; \gamma_d \;\cdot\; \frac{a_{n,d} \;-\; \mathbb{E}[\mathrm{a}_{n,\cdot}]}{\sqrt{\mathrm{Var}[\mathrm{a}_{n,\cdot}] + \epsilon}},$$

where $\beta_d \in \mathbb{R}$ and $\gamma_d \in \mathbb{R}$, $d \in [1, D]$, are per-dimension parameters to be learned, $\epsilon \in \mathbb{R}^+$ is (again) a small positive constant for numerical stability, $\mathbb{E}[\mathrm{a}_{n,\cdot}]$ is the mean (first moment) of the n pre-activation vector as computed empirically over the D *dimensions* contained within $\mathbf{a}_n$, and $\mathrm{Var}[\mathrm{a}_{n,\cdot}]$ is the variance (second moment) of the n-th per-activation vector as computed empirically over the D dimensions contained within $\mathbf{a}_n$. In other words, LayerNorm performs its normalization by computing the mean and variance statistics for each *row* of $\mathbf{A}$. At test time, the mean and variance of the per-instance pre-activations can be computed just the same as they were during training. The only difference is that the γ and β parameters will no longer be updated.

One may worry that applying these normalization techniques can discard valuable information about the scale of the pre-activations. This is not too much of a worry since the normalization is applied on the hidden units, which don't have a strong interpretation in the way that data does, so we are quite free to change the representations to make computation easier and the DNN will learn to propagate information within these constraints. Moreover, ReLU activations are 'scale free,' meaning that for $\mathrm{z} \in \mathbb{R}^+$, we have $\mathrm{z} \cdot \texttt{ReLU}(\mathrm{a}) = \texttt{ReLU}(\mathrm{z} \cdot \mathrm{a})$. This means that it is more meaningful to the DNN if the ReLU is inactive vs active rather than its precise output value.

4.6 Capacity Control for Deep Neural Networks

Deep neural networks (DNNs) typically have enough parameters to interpolate (nearly) any finite training set. This is a blessing—it lets us fit extremely rich functions—but it is also a curse because it makes *overfitting* easy. In this section, we discuss practical ways to control the *effective capacity* of DNNs so that they generalize better, i.e. so that performance on held-out data matches performance on the training data.

4.6.1 Data Augmentation (Capacity Control by Inductive Bias)

Many deep learning problems have known invariances. For example, for image classification we often want invariance to small translations, left-right flips, and mild color changes. *Data augmentation* enforces such invariances by training on randomly transformed versions of the data. Let $T(\cdot)$ denote a random transformation drawn from some distribution over augmentations. Then the empirical risk becomes:

$$\min_{\boldsymbol{\theta}} \; \frac{1}{N} \sum_{n=1}^{N} \mathbb{E}_T \left[\ell\big(\boldsymbol{\theta}; (T(\boldsymbol{x}_n), y_n)\big) \right], \tag{30}$$

which in practice is approximated by sampling a fresh augmentation each time the example is loaded. Data augmentation can be seen as (i) increasing the effective dataset size and (ii) restricting the class of functions the network can represent *while still fitting the augmented training set*, thereby controlling capacity in a way that is often more aligned with the task than generic norm penalties.

4.6.2 Weight Decay (a.k.a. $L2$ Regularization)

The most common explicit capacity control technique for DNNs is *weight decay*. It augments the training objective with a penalty that discourages large weights:

$$\widetilde{\ell}(\mathbf{W}_0, \ldots, \mathbf{W}_L; \mathcal{D}, \lambda) \;=\; \ell(\mathbf{W}_0, \ldots, \mathbf{W}_L; \mathcal{D}) \;+\; \lambda \cdot \sum_{l=0}^{L} ||\mathbf{W}_l||_2^2 \,, \tag{31}$$

where $\lambda \in \mathbb{R}^+$ controls the trade-off between data fit and complexity control.[2]

Why does penalizing the weight norm reduce capacity? Intuitively, constraining the weights tends to produce smoother, less "wiggly" functions, especially when combined with Lipschitz activation functions. More formally, for linear regression, an $L2$ penalty leads to ridge regression and shrinks the solution toward zero; for deep models, it similarly discourages sharp decision boundaries and extreme logit values.

Gradient descent view (explicit shrinkage) If we optimize equation 31 with (mini-batch) gradient descent, then for a single parameter tensor (e.g. $\mathbf{W}_l$) we have:

$$\mathbf{W}_l^{t+1} \;=\; \mathbf{W}_l^t \;-\; \alpha \cdot \nabla_{\mathbf{W}_l^t} \ell(\mathbf{W}_0^t, \ldots, \mathbf{W}_L^t; \mathcal{B}_t) \;-\; \alpha \cdot 2\lambda \mathbf{W}_l^t. \tag{32}$$

Rearranging yields:

$$\mathbf{W}_l^{t+1} \;=\; (1 - 2\alpha\lambda)\mathbf{W}_l^t \;-\; \alpha \cdot \nabla_{\mathbf{W}_l^t} \ell(\cdot), \tag{33}$$

which shows why it is called *weight decay*: the weights are multiplicatively shrunk at each step.

4.6.3 Dropout: Stochastic Capacity Control via Random Subnetworks

Dropout is a simple and effective technique that injects multiplicative noise into the hidden representations during training (Srivastava et al., 2014). For a hidden layer vector $\mathbf{h}_l \in \mathbb{R}^{D_l}$, dropout samples a random binary mask:

$$\mathbf{m}_l \sim \text{Bernoulli}(p)^{D_l}, \tag{34}$$

and then replaces $\mathbf{h}_l$ by a masked version:

$$\tilde{\mathbf{h}}_l \;=\; \frac{1}{p} \left(\mathbf{h}_l \odot \mathbf{m}_l\right), \tag{35}$$

[2] In many implementations, the bias / offset parameters are excluded from weight decay. In our notation biases are encoded by appending a constant 1 to $\mathbf{h}_l$; equivalently, one can exclude the last row (or column, depending on the convention) of each $\mathbf{W}_l$ from the penalty.

where $\odot$ denotes element-wise multiplication and $p \in (0, 1]$ is the *keep probability*. The factor $1/p$ is called *inverted dropout*; it ensures that

$$\mathbb{E}[\tilde{\mathbf{h}}_l \,|\, \mathbf{h}_l] \;=\; \mathbf{h}_l, \tag{36}$$

so that we do not need to rescale activations at test time.

Test time At test time, we turn off the noise and use the full hidden representation:

$$\tilde{\mathbf{h}}_l \;=\; \mathbf{h}_l, \tag{37}$$

which corresponds (roughly) to averaging over an ensemble of many subnetworks induced by different dropout masks (this is one of the standard intuitions for why dropout works).

Where to apply dropout? In fully connected networks, dropout is often applied to hidden layers (and sometimes the input). In modern architectures with BatchNorm and residual connections, dropout is still used but tends to require more careful tuning. A typical heuristic is to (i) apply dropout to later layers more than early layers, and (ii) avoid very aggressive dropout in networks that already have strong implicit regularization from data augmentation and normalization layers.

4.6.4 Ensembling (Deep Ensembles and Prediction Averaging)

As discussed earlier in the notes, *ensembling* reduces variance by averaging the predictions of multiple models. For DNNs, the standard approach is to train J independent networks (e.g. with different random initializations, data shuffles, or even architectures). For a K-class classifier producing softmax probabilities $\boldsymbol{\pi}_j(\mathbf{x})$, the ensemble predictive distribution is:

$$\bar{\boldsymbol{\pi}}(\mathbf{x}) \;=\; \frac{1}{J}\sum_{j=1}^{J} \boldsymbol{\pi}_j(\mathbf{x}), \qquad \hat{y}(\mathbf{x}) \;=\; \operatorname*{arg\,max}_{k\in[1,K]} \; \bar{\pi}_k(\mathbf{x}). \tag{38}$$

Ensembles often provide some of the most reliable improvements in held-out accuracy (and also uncertainty estimates), but the drawback is computational: training and storing J networks, and evaluating all J at test time.

Dropout vs ensembling Dropout can be interpreted as an inexpensive approximate ensemble of subnetworks; deep ensembles in equation 38 are a more direct (and typically stronger) form of ensembling, but with correspondingly higher computational cost. One can construct a multi-model ensemble via dropout by leaving dropout on at test-time to generate J different model and then explicitly ensembling the J different predictions. This has the same test-time overhead but reduces the need to independently train J models (though, as mentioned above, at the cost of weaker ensembling).

5 Stochastic, Adaptive Optimizers

Thus far, we have described gradient descent as an iterative procedure that uses the full training set $\mathcal{D}$ to compute an update at every step. For modern deep learning models, this

is usually computationally prohibitive: for large N, one full gradient computation can take seconds, minutes, or longer. Moreover, when the data has redundancy (as it often does), using the whole data set at every step is wasteful. These two motivations lead to *stochastic* optimization methods, in which we approximate the full gradient using only a small subset of the data at each iteration.

5.1 Mini-Batch Gradient Descent

Assume the loss decomposes over data points:

$$\ell(\boldsymbol{\theta}; \mathcal{D}) \;=\; \frac{1}{N} \sum_{n=1}^{N} \ell(\boldsymbol{\theta}; (\boldsymbol{x}_n, \boldsymbol{y}_n)). \tag{39}$$

At iteration t, rather than computing $\nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}^t; \mathcal{D})$, we instead sample a *mini-batch* $\mathcal{B}_t \subset \mathcal{D}$ with $|\mathcal{B}_t| = B \ll N$ (typically uniformly at random), and compute:

$$\nabla_{\boldsymbol{\theta}^t} \ell(\boldsymbol{\theta}^t; \mathcal{B}_t) \;\triangleq\; \frac{1}{B} \sum_{b \in \mathcal{B}_t} \nabla_{\boldsymbol{\theta}^t} \ell(\boldsymbol{\theta}^t; (\boldsymbol{x}_b, \boldsymbol{y}_b)). \tag{40}$$

This gives the update:

$$\boldsymbol{\theta}^{t+1} \;=\; \boldsymbol{\theta}^t \;-\; \alpha \cdot \left[\nabla_{\boldsymbol{\theta}^t} \ell(\boldsymbol{\theta}^t; \mathcal{B}_t)\right]^T. \tag{41}$$

When $\boldsymbol{\theta}$ corresponds to a particular layer's parameters $\mathbf{W}_l$, we recover the same form as in Section 4, but with $\mathcal{D}$ replaced by $\mathcal{B}_t$.

Proposition 5.1 (Unbiasedness of the mini-batch gradient)**.** Assume $\mathcal{B}_t$ is formed by sampling B points i.i.d. uniformly from $\mathcal{D}$ (with replacement). Then

$$\mathbb{E}_{\mathcal{B}_t} \left[\nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}; \mathcal{B}_t)\right] \;=\; \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}; \mathcal{D}). \tag{42}$$

Proof. By linearity of expectation and i.i.d. sampling,

$$\begin{aligned}
\mathbb{E}_{\mathcal{B}_t} \left[\nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}; \mathcal{B}_t)\right] &= \mathbb{E}\left[\frac{1}{B} \sum_{i=1}^{B} \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}; (\boldsymbol{x}_{b_i}, \boldsymbol{y}_{b_i}))\right] \\
&= \frac{1}{B} \sum_{i=1}^{B} \mathbb{E}\left[\nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}; (\boldsymbol{x}_{b_i}, \boldsymbol{y}_{b_i}))\right] \\
&= \mathbb{E}_{(\boldsymbol{x}, \boldsymbol{y}) \sim \mathtt{Unif}(\mathcal{D})} \left[\nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}; (\boldsymbol{x}, \boldsymbol{y}))\right] = \frac{1}{N} \sum_{n=1}^{N} \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}; (\boldsymbol{x}_n, \boldsymbol{y}_n)) \\
&= \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}; \mathcal{D}).
\end{aligned}$$

□

The key difference between full-batch and mini-batch training is therefore not bias but *variance*. For small B, the gradient estimator in Eq. equation 40 has high variance, yielding a noisy optimization trajectory. This noise can be beneficial early in training (helping the optimizer explore), but it can also slow convergence and require smaller step sizes.

Terminology: epochs and iterations One *iteration* is one update of Eq. equation 41. One *epoch* is one full pass over the data set. If all data points are visited exactly once per epoch (e.g. by shuffling and then taking contiguous mini-batches), then there are approximately N/B iterations per epoch.

5.2 Momentum

A simple way to stabilize the noise in mini-batch gradients is to smooth them across iterations. *Momentum* maintains a velocity variable (an exponentially weighted moving average of gradients) for each parameter tensor. For a particular layer's weights $\mathbf{W}_l$, we define:

$$\mathbf{V}_l^t \;=\; \beta \cdot \mathbf{V}_l^{t-1} \;+\; \nabla_{\mathbf{W}_l^t}\, \ell\left(\mathbf{W}_0^t, \ldots, \mathbf{W}_l^t, \ldots, \mathbf{W}_L^t; \mathcal{B}_t\right), \tag{43}$$

followed by the update

$$\mathbf{W}_l^{t+1} \;=\; \mathbf{W}_l^t \;-\; \alpha \cdot \mathbf{V}_l^t. \tag{44}$$

Here $\beta \in [0, 1)$ controls the amount of memory. If $\beta = 0$, we recover vanilla mini-batch gradient descent; if β is close to 1, then gradients are heavily smoothed.

Interpretation If we unroll Eq. equation 43, we see that $\mathbf{V}_l^t$ is a weighted average of all past gradients:

$$\mathbf{V}_l^t \;=\; \sum_{j=0}^{t} \beta^j \cdot \nabla_{\mathbf{W}_l^{t-j}}\, \ell\left(\mathbf{W}_0^{t-j}, \ldots, \mathbf{W}_L^{t-j}; \mathcal{B}_{t-j}\right), \tag{45}$$

up to initialization effects (usually $\mathbf{V}_l^0 = \mathbf{0}$). This averaging reduces variance and can dramatically improve optimization speed in deep networks.

5.3 Adaptive Moment Estimation (Adam)

Momentum smooths the *direction* of the update but does not adapt the *step size* per parameter. Adaptive methods do both. The most popular optimizer in deep learning is *Adam* (adaptive moment estimation), which maintains moving averages of both the first moment (mean) and second moment (uncentered variance) of the gradient (Kingma and Ba, 2014).

For each parameter tensor $\mathbf{W}_l$, Adam computes:

$$\mathbf{V}_l^t \;=\; \beta_1 \cdot \mathbf{V}_l^{t-1} \;+\; (1-\beta_1) \cdot \nabla_{\mathbf{W}_l^t}\, \ell\left(\mathbf{W}_0^t, \ldots, \mathbf{W}_L^t; \mathcal{B}_t\right), \tag{46}$$

$$\mathbf{S}_l^t \;=\; \beta_2 \cdot \mathbf{S}_l^{t-1} \;+\; (1-\beta_2) \cdot \left(\nabla_{\mathbf{W}_l^t}\, \ell\left(\mathbf{W}_0^t, \ldots, \mathbf{W}_L^t; \mathcal{B}_t\right)\right)^2, \tag{47}$$

where $(\cdot)^2$ is applied element-wise.

Bias correction Because $\mathbf{V}_l^t$ and $\mathbf{S}_l^t$ are initialized at zero, early iterations are biased toward zero. Adam corrects this via:

$$\hat{\mathbf{V}}_l^t = \frac{\mathbf{V}_l^t}{1-\beta_1^t}, \quad \hat{\mathbf{S}}_l^t = \frac{\mathbf{S}_l^t}{1-\beta_2^t}. \tag{48}$$

Final update The Adam update is then:

$$\mathbf{W}_l^{t+1} \;=\; \mathbf{W}_l^t \;-\; \frac{\alpha}{\sqrt{\hat{\mathbf{S}}_l^t + \epsilon}} \odot \hat{\mathbf{V}}_l^t, \tag{49}$$

where $\epsilon \in \mathbb{R}^+$ is a small constant for numerical stability and $\odot$ denotes element-wise multiplication. The division is also element-wise, meaning that Adam effectively assigns a different step size to each element of $\mathbf{W}_l$.

Default hyperparameters A common choice is $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$. Despite this being a common "default," it is still necessary to tune the learning rate α, and sometimes also β_1 and β_2, for best performance.

Decoupled weight decay (AdamW) Recall from Section 4 that *weight decay* penalizes large weights. In Adam, naively adding an $L2$ penalty to the loss changes the gradient in a way that entangles the adaptive preconditioning in Eq. equation 49. A common alternative is *decoupled* weight decay (often called AdamW), which applies the penalty directly to the parameters:

$$\mathbf{W}_l^{t+1} \;=\; (1 - \alpha\lambda)\mathbf{W}_l^t \;-\; \frac{\alpha}{\sqrt{\hat{\mathbf{S}}_l^t + \epsilon}} \odot \hat{\mathbf{V}}_l^t, \tag{50}$$

where λ is the weight decay coefficient. This is often more stable and can yield better generalization.

5.4 Practical Considerations

Learning rate schedules It is common to vary α across training. Two popular heuristics are (i) *warmup*, where α is increased linearly over the first few epochs, and (ii) *decay*, where α is decreased over time (step decay, exponential decay, or cosine decay). These schedules can improve both stability and final performance.

Gradient clipping Especially for sequential models (e.g. RNNs), gradients can explode. A common fix is *clipping*, e.g. clipping the global gradient norm:

$$\nabla_{\boldsymbol{\theta}} \ell \;\leftarrow\; \nabla_{\boldsymbol{\theta}} \ell \cdot \min\left\{1,\; \frac{\tau}{||\nabla_{\boldsymbol{\theta}} \ell||_2}\right\}, \tag{51}$$

where τ is a chosen threshold.

Batch size The batch size B influences both compute efficiency and the noise scale of the gradient estimator. Small batches lead to noisier updates (sometimes improving generalization), while large batches are more compute-efficient on GPUs but can require careful tuning of the learning rate.

6 Convolutional Neural Networks

Convolutional neural networks (CNNs) are feedforward neural networks designed for data with strong local structure, such as images, audio spectrograms, and other grid-like inputs.

The defining architectural change relative to fully-connected networks is that a CNN replaces dense linear maps with *convolutions*: local, translation-equivariant operations with *shared* parameters. This simultaneously (i) reduces the number of parameters and (ii) induces useful inductive bias, namely that local patterns can appear anywhere in the input. The idea of this operation is that we want to preserve spatial information locally but not globally. Regarding (ii), I want to be able to recognize a pattern of pixels that looks like a dog, but I don't care so much where exactly the dog is in the image. This allows the CNNs to search for patterns that can be translated to different parts of the input space.

6.1 Convolutional Layers

Let the input be a 3D tensor (e.g. an image with channels)

$$\mathbf{X} \in \mathbb{R}^{H \times W \times D}, \tag{52}$$

where H is height, W is width, and D is the number of channels (e.g. $D = 3$ for RGB images). A convolutional layer with K filters (a.k.a. output channels) produces a tensor:

$$\mathbf{A} \in \mathbb{R}^{H' \times W' \times K}. \tag{53}$$

Discrete convolution (cross-correlation) In deep learning libraries, the "convolution" operation is typically implemented as cross-correlation. Let the filter size be $F \times F$, stride be S, and padding be P. For output channel $k \in [1, K]$, the filter weights are

$$\mathbf{W}^{(k)} \in \mathbb{R}^{F \times F \times D}, \quad \text{and bias } \mathrm{b}_k \in \mathbb{R}. \tag{54}$$

Then the pre-activation at spatial location (i, j) is:

$$a_{i,j,k} \;=\; \sum_{u=0}^{F-1} \sum_{v=0}^{F-1} \sum_{d=1}^{D} w_{u,v,d}^{(k)} \cdot x_{iS+u-P,\; jS+v-P,\; d} \;+\; b_k, \tag{55}$$

where indices that fall outside the original image are handled by padding (most commonly, zero padding).

Output size The spatial output dimensions are:

$$H' \;=\; \frac{H - F + 2P}{S} + 1, \quad W' \;=\; \frac{W - F + 2P}{S} + 1, \quad D' \;=\; K, \tag{56}$$

assuming the divisions yield integers.

Parameter count Each filter has $F \cdot F \cdot D$ weights plus one bias. Therefore, the number of trainable parameters in the layer is:

$$K \cdot (F \cdot F \cdot D) \;+\; K. \tag{57}$$

This should be compared to a fully-connected map from $\mathbb{R}^{HWD}$ to $\mathbb{R}^{H'W'K}$, which would require $(HWD) \cdot (H'W'K)$ parameters and thus is typically infeasible.

Activation functions and normalization As with fully-connected networks, we almost always follow the convolution by a non-linearity (e.g. ReLU) and often a normalization layer (e.g. BatchNorm). A typical pattern is:

$$\mathbf{H} \;=\; \phi\left(\texttt{BatchNorm}(\mathbf{A})\right), \tag{58}$$

where ϕ is applied element-wise.

Proposition 6.1 (Translation equivariance of convolution)**.** Let $\mathcal{T}_\delta$ denote a spatial translation operator applied to $\mathbf{X}$ by shifting it by an integer offset δ (with compatible padding). Let $\texttt{Conv}(\cdot;\mathbf{W})$ denote the convolution map in Eq. equation 55 without non-linearities. Then

$$\texttt{Conv}(\mathcal{T}_\delta \mathbf{X};\mathbf{W}) \;=\; \mathcal{T}_\delta \texttt{Conv}(\mathbf{X};\mathbf{W}). \tag{59}$$

Both sides compute the same weighted sums of local neighborhoods, and the neighborhoods are shifted in exactly the same way under $\mathcal{T}_\delta$. Because the filter weights are shared across all spatial locations, the same local computation is applied after translation, yielding an output that is translated by the same offset. **Equivariance is not the same as invariance**: if we want the representation to become less sensitive to small translations (i.e. more invariant), we typically introduce pooling or strided downsampling.

6.2 Pooling Layers

It is common to also pair *pooling layers* with convolutional layers. These help reduce the size and parameter count of the model by periodically pooling information across the activation map while still preserving the spatial structure. Pooling layers aggregate local information and reduce spatial resolution, which (i) lowers compute cost and (ii) increases robustness to small spatial changes.

Max pooling *Max pooling* is the most common form, which returns the maximum element contained in some filter size. Given $\mathbf{X} \in \mathbb{R}^{H\times W\times D}$, max pooling with window size F and stride S produces an output $\mathbf{X}' \in \mathbb{R}^{H'\times W'\times D}$ where:

$$H' \;=\; \frac{H-F}{S}+1, \quad W' \;=\; \frac{W-F}{S}+1, \quad D' \;=\; D, \tag{60}$$

and

$$x'_{i,j,d} \;=\; \max_{0\le u<F,\; 0\le v<F} x_{iS+u,\; jS+v,\; d}. \tag{61}$$

Notice that the depth is preserved: $D' = D$.

Average pooling and global pooling An alternative is average pooling, which replaces the max with an average. A common special case is *global average pooling*, which pools over the full spatial extent and yields one value per channel. This is often used at the end of CNN classifiers to avoid large fully-connected layers.

6.3 Backpropagation for Convolutions (Index Form)

While convolutional backpropagation is typically handled by automatic differentiation, it is helpful to see the structure of the gradients. Let ℓ denote the loss and assume we have computed the gradient at the pre-activations:

$$g_{i,j,k} \triangleq \frac{\partial \ell}{\partial a_{i,j,k}}. \tag{62}$$

Then the gradient for a particular filter weight is:

$$\frac{\partial \ell}{\partial w^{(k)}_{u,v,d}} = \sum_{i=0}^{H'-1} \sum_{j=0}^{W'-1} g_{i,j,k} \cdot x_{iS+u-P,\ jS+v-P,\ d}, \tag{63}$$

and the bias gradient is:

$$\frac{\partial \ell}{\partial b_k} = \sum_{i=0}^{H'-1} \sum_{j=0}^{W'-1} g_{i,j,k}. \tag{64}$$

Thus, like fully-connected layers, gradients have the form "error signal times input," but the input is now a local receptive field and the summation is over all spatial locations.

6.4 Common CNN Design Patterns

Downsampling while increasing channels CNNs typically reduce spatial resolution (via pooling or strided convolutions) while increasing the number of channels. Intuitively, early layers represent local edge- or texture-like features, while deeper layers represent higher-level patterns composed of those local primitives.

Receptive fields Stacking convolutions increases the effective *receptive field* of deeper units: a unit in a deep layer can depend on a larger portion of the input than a unit in a shallow layer. This is one reason that depth is important for CNNs even when each individual filter is small (e.g. 3×3).

Residual blocks As discussed in Section 4, skip connections improve optimization for deep networks. Modern CNNs (e.g. ResNets) are built primarily from residual blocks, where convolutional sub-networks predict residual updates to hidden representations.

From feature maps to predictions For classification with K classes, a CNN typically maps the final representation to logits $\boldsymbol{a}(\boldsymbol{x}) \in \mathbb{R}^K$ and then applies the softmax inverse link:

$$p(\mathrm{y} = k|\mathbf{x}) = \texttt{softmax}_k(\boldsymbol{a}(\boldsymbol{x})). \tag{65}$$

The training objective is then the same categorical cross-entropy loss derived earlier; the key difference is that $\boldsymbol{a}(\boldsymbol{x})$ is produced by convolutional rather than fully-connected feature extractors.

7 Models for Sequential Data

Many applications have an inherent notion of a sequence, with language being the dominant example. The sentence *The dog jumps* can be treated as sequential data of the form $\{\mathrm{x}_1 = \texttt{the}, \mathrm{x}_2 = \texttt{dog}, \mathrm{x}_3 = \texttt{jumps}\}$. Beyond language, sequential structure arises in time series (stock prices, sensor readings), audio (speech waveforms, music), genomics (DNA sequences), and many other domains. The key challenge in modeling sequential data is that, unlike the i.i.d. setting we have mostly considered so far, observations are not independent: the identity and meaning of $\mathrm{x}_3 = \texttt{jumps}$ depends on $\mathrm{x}_2 = \texttt{dog}$ preceding it. In this section, we will discuss recurrent neural networks that are designed to handle sequences as input, output, or both.

7.1 Recurrent Neural Networks

Recurrent neural networks (RNNs) perform a recurrent operation at each time step whose input is the new observation available at that time, x_t, and the previous hidden state h_{t-1}. The output of this operation is a new hidden state that can either be transformed again to make a prediction and/or used again for a recurrent operation. Thus the architecture allows information about the collection of inputs seen so far to propagate via the hidden state.

A useful perspective is to contrast RNNs with the feedforward networks we have been studying. A feedforward network processes its input in a single forward pass with a fixed number of layers. An RNN, by contrast, can be thought of as applying the *same* set of operations repeatedly at each time step, with the hidden state serving as a form of memory that carries information forward through the sequence. If we were to 'unroll' the RNN across time steps, it would resemble a very deep feedforward network with shared weights at every layer—a perspective that will become especially important when we discuss training.

Simple Recurrence The simplest variant of a recurrent neural network has a hidden state that is computed as:

$$\mathbf{h}_t \;=\; \phi\left(\mathbf{W}_0^T \mathbf{x}_t \;+\; \mathbf{W}_1^T \mathbf{h}_{t-1}\right)$$

where $\phi(\cdot)$ is an activation function such as the ReLU or tanh, $\mathbf{x}_t \in \mathbb{R}^D$ is a D-dimensional input vector for time t, $\mathbf{h}_t \in \mathbb{R}^H$ is the hidden state at time t, $\mathbf{W}_0 \in \mathbb{R}^{D \times H}$ are the 'input-to-hidden' parameters, and $\mathbf{W}_1 \in \mathbb{R}^{H \times H}$ are the 'hidden-to-hidden' parameters. Notice that the parameters $\mathbf{W}_0$ and $\mathbf{W}_1$ are *shared* across all time steps—the same transformation is applied regardless of whether we are at $t = 1$ or $t = 1000$. This weight sharing is what makes RNNs able to handle sequences of variable length with a fixed number of parameters. It also imposes a strong inductive bias: the RNN assumes that whatever transformation is useful at one point in the sequence is useful at every point, which is a temporal analog of the spatial translation equivariance we saw in CNNs.

Choice of Activation Function The choice of $\phi(\cdot)$ has practical consequences that differ somewhat from the feedforward case. Historically, the tanh activation is the most common choice for simple RNNs. Unlike the logistic sigmoid whose output lies in $(0, 1)$, the tanh function outputs values in $(-1, 1)$, which is symmetric about zero. This zero-centered

property helps prevent the hidden state from drifting systematically in one direction over many time steps. The ReLU can also be used, but care must be taken: because the hidden-to-hidden matrix $\mathbf{W}_1$ is applied repeatedly, the hidden state can grow without bound if the spectral radius of $\mathbf{W}_1$ is greater than one, leading to numerical overflow. One remedy is to initialize $\mathbf{W}_1$ as an orthogonal matrix, which preserves the norm of the hidden state through the recurrence (**?**).

Hidden State Initialization To start the recurrence, one must specify $\mathbf{h}_0$. The most common choice is simply $\mathbf{h}_0 = \mathbf{0}$, the zero vector. This has the benefit of not introducing any additional hyperparameters. An alternative is to treat $\mathbf{h}_0$ as a learnable parameter vector that is optimized along with the rest of the model. In practice, the choice of initialization matters most for short sequences and matters less for long sequences, since the influence of $\mathbf{h}_0$ on the hidden state diminishes as the RNN processes more and more tokens.

Example: Sentiment Analysis Let's walk through a simple RNN implementation for the task of sentiment analysis. Let $\mathbf{x}_n = \{\mathbf{x}_{n,1}, \ldots, \mathbf{x}_{n,t}, \ldots, \mathbf{x}_{n,T}\}$ be the n-th T-length sequence representing a paragraph describing a review of a movie or film. Assume each $\mathbf{x}_{n,t} \in \mathbb{R}^D$ is a real-valued vector encoding a word. That is, for each word in a fixed vocabulary, we have a look up operation that maps it to a real-valued vector that could either be trained or not. This look-up operation is sometimes called a *word embedding*: given a vocabulary of V words, we maintain an embedding matrix $\mathbf{E} \in \mathbb{R}^{V \times D}$ such that the v-th row of $\mathbf{E}$ is the D-dimensional representation of the v-th word. When word embeddings are trained jointly with the RNN, the model can learn representations that are tailored to the task at hand. Alternatively, one can use pre-trained embeddings (e.g. Word2Vec, GloVe) that capture distributional semantics from large text corpora, which is especially useful when task-specific training data is limited.

Moving on to the label, each of these reviews has an associated binary label $\mathrm{y}_n \in \{0, 1\}$ that denotes if the review has an overall positive (1) or negative (0) opinion of the movie. To model this binary label, we will use the same GLM techniques we have used for the duration of the course:

$$\begin{aligned} \mathbb{E}\left[\, \mathrm{y}_n \mid \mathbf{x}_{n,1}, \ldots, \mathbf{x}_{n,t}, \ldots, \mathbf{x}_{n,T} \,\right] &= g^{-1}\left(\mathbf{w}_2^T \mathbf{h}_T\right) \\ &= \texttt{logistic}\left(\mathbf{w}_2^T \mathbf{h}_{n,T}\right) \end{aligned} \tag{66}$$

where $\mathbf{w}_2 \in \mathbb{R}^H$ are parameters that transform the hidden state into the expected value of the label given the sequence of words in the review. Of course, if y has some other support, such as all real-values, then g^{-1} and the distributional assumptions would have to be changed. One could add multiple fully-connected feedforward layers between $\mathbf{h}_T$ and the final prediction. The important part is that the prediction be a function of $\mathbf{h}_T$, since that is the only hidden state that contains information about all words in the review.

Now we are ready to derive our training function:

$$
\begin{aligned}
&\mathbb{E}_{\mathrm{x}_1,\ldots,\mathrm{x}_T}\mathbb{KLD}\left[\mathbb{P}(\mathrm{y}|\mathrm{x}_1,\ldots,\mathrm{x}_T)\ ||\ p(\mathrm{y}|\mathrm{x}_1,\ldots,\mathrm{x}_T)\right] \\
&= \mathbb{E}_{\mathbb{P}(\mathrm{y}|\mathrm{x}_1,\ldots,\mathrm{x}_T)}\left[-\log p(\mathrm{y}|\mathrm{x}_1,\ldots,\mathrm{x}_T)\right]\ +\ \text{const.} \\
&\approx \frac{1}{N}\sum_{n=1}^{N} -\log p(y_n|x_{n,1},\ldots,x_{n,T})\ +\ \text{const.} \\
&= \frac{1}{N}\sum_{n=1}^{N} -\log \text{Bernoulli}(y_n;\pi_n = \texttt{logistic}\left(\mathbf{w}_2^T\mathbf{h}_{n,T}\right))\ +\ \text{const.} \\
&= \frac{1}{N}\sum_{n=1}^{N} -y_n\cdot\log\left\{\texttt{logistic}\left(\mathbf{w}_2^T\mathbf{h}_{n,T}\right)\right\} - (1-y_n)\cdot\log\left\{1-\texttt{logistic}\left(\mathbf{w}_2^T\mathbf{h}_{n,T}\right)\right\}\ +\ \text{const.} \\
&= \ell(\mathbf{W}_0,\mathbf{W}_1,\mathbf{w}_2;\mathcal{D}).
\end{aligned}
$$

Backpropagation Through Time Training an RNN is via gradient descent, exploiting the chain rule just as we did for feedforward networks. The analog of backpropagation for RNNs is called *backpropagation through time* (BPTT) since we can think of the errors as following backwards from the loss, as its computed at time T, all the way back to $t = 1$, since time steps are analogous to layers in a feedforward NN. For purposes of demonstration, let's switch to an RNN in which all variables are scalar variables: $w_2, w_1, w_0 \in \mathbb{R}$. Moreover, lets assume that $h_0 = 0$, meaning that the hidden state is initialized to zero, which is common (with the other common alternative being to initialize it to one). Let's work through an example, computing the derivative for w_1, the recurrent-to-recurrent weight.

$$
\begin{aligned}
&\frac{d}{d\ w_1}\ \ell(w_0,w_1,w_2;\mathcal{D})\ = \\
&\frac{d}{d\ w_1}\left[\frac{1}{N}\sum_{n=1}^{N}\ -y_n\cdot\log\left\{\texttt{logistic}\left(w_2\cdot h_{n,T}\right)\right\} - (1-y_n)\cdot\log\left\{1-\texttt{logistic}\left(w_2\cdot h_{n,T}\right)\right\}\ +\ \text{const.}\right] \\
&= \frac{1}{N}\sum_{n=1}^{N}\ -\frac{d}{d\ w_1}\cdot y_n\cdot\log\left\{\texttt{logistic}\left(w_2\cdot h_{n,T}\right)\right\} - \frac{d}{d\ w_1}\cdot(1-y_n)\cdot\log\left\{1-\texttt{logistic}\left(w_2\cdot h_{n,T}\right)\right\} \\
&= \frac{1}{N}\sum_{n=1}^{N}\left(\texttt{logistic}\left(w_2\cdot h_{n,T}\right) - y_n\right)\cdot w_2\cdot\frac{d\ h_{n,T}}{d\ w_1}.
\end{aligned}
$$

Now lets focus on the derivative through the recurrent computation, $\frac{d\ h_T}{d\ w_1}$. For shorthand, lets denote the expression inside the activation function as a_t: $a_{n,t} = w_0\cdot x_{n,t} + w_1\cdot h_{n,t-1}$.

$$
\begin{aligned}
&\frac{d\ h_T}{d\ w_1}\ =\ \phi'(a_{n,T})\cdot\left(h_{n,T-1} + w_1\cdot\frac{d\ h_{n,T-1}}{d\ w_1}\right) \\
&= \phi'(a_{n,T})\cdot\left(h_{n,T-1} + w_1\cdot\phi'(a_{n,T-1})\cdot\left(h_{n,T-2} + w_1\cdot\frac{d\ h_{n,T-2}}{d\ w_1}\right)\right) \\
&= \phi'(a_{n,T})\cdot\left(h_{n,T-1} + w_1\cdot\phi'(a_{n,T-1})\cdot\left(h_{n,T-2} + w_1\cdot\phi'(a_{n,T-2})\cdot\left(h_{n,T-3} + w_1\cdot\frac{d\ h_{n,T-3}}{d\ w_1}\right)\right)\right)
\end{aligned}
$$

and so on, all the way back to h_0, where $\phi'(\cdot)$ is the derivative of the activation function. Collecting terms by multiplying through, we see we have the general form:

$$\frac{d\ h_T}{d\ w_1} \ = \ \sum_{j=T}^{1} w_1^{T-j} \cdot \left(\prod_{i=T}^{j} \phi'(a_{n,i}) \right) \cdot h_{n,j-1}.$$

Note this form only works for scalar variables since, if we have vector / matrix multiplications, we could not re-order the w_1 and ϕ' terms like we can in the scalar case. Also note that if $h_0 = 0$, the the final term in the sum would be zero since it will all be multiplied by h_0.

Vanishing and Exploding Gradients in RNNs The expression above makes explicit a problem that is especially acute for RNNs. Recall that in Section 4, we discussed the vanishing and exploding gradient problem for deep feedforward networks. In RNNs, this problem is amplified because the effective 'depth' of the network equals the sequence length T, which can be hundreds or even thousands of steps. Consider the product $\prod_{i=T}^{j} \phi'(a_{n,i})$. If the tanh activation is used, its derivative satisfies $0 < \phi'(a) \leq 1$ for all a, and the derivative is close to one only near $a = 0$. As the product accumulates over many time steps, it tends towards zero exponentially fast, meaning that the gradient contribution from distant time steps vanishes. In the scalar case, we also see w_1^{T-j} appearing, meaning that if $|w_1| < 1$, the gradient decays exponentially with the distance $T - j$. Conversely, if $|w_1| > 1$, the gradient grows exponentially. In the matrix case, the analogous condition involves the spectral radius of $\mathbf{W}_1$: if the largest singular value of $\mathbf{W}_1$ is less than one, gradients vanish; if it is greater than one, gradients can explode.

The practical consequence of vanishing gradients is that simple RNNs struggle to learn *long-range dependencies*: the model may fail to capture that a word appearing early in a document is important for understanding the sentiment expressed at the end. Exploding gradients, on the other hand, can cause the optimization to diverge entirely. A simple and effective heuristic for the exploding gradient problem is *gradient clipping*: if the norm of the gradient vector exceeds some threshold γ, rescale it so that $\|\nabla_{\boldsymbol{\theta}} \ell\| = \gamma$. This can be written as:

$$\nabla_{\boldsymbol{\theta}} \ell \ \leftarrow \ \begin{cases} \gamma \cdot \frac{\nabla_{\boldsymbol{\theta}} \ell}{\|\nabla_{\boldsymbol{\theta}} \ell\|} & \text{if } \|\nabla_{\boldsymbol{\theta}} \ell\| > \gamma \\ \nabla_{\boldsymbol{\theta}} \ell & \text{otherwise.} \end{cases}$$

Gradient clipping preserves the direction of the gradient but limits its magnitude, preventing catastrophic updates. However, gradient clipping does not help with vanishing gradients—for that, we need architectural solutions, which we discuss next.

Truncated Backpropagation Through Time For very long sequences, computing the full BPTT gradient can be both computationally expensive and numerically unstable. A common practical approximation is *truncated backpropagation through time* (TBPTT). Instead of backpropagating the gradient all the way from time T to $t = 1$, we only backpropagate for a fixed number of steps κ. Formally, when computing $d\mathbf{h}_T / d\mathbf{W}_1$, we truncate the recursion after κ steps, effectively treating $\mathbf{h}_{T-\kappa}$ as if it does not depend on $\mathbf{W}_1$. This introduces a bias into the gradient estimate but dramatically reduces computation and helps avoid the vanishing gradient problem by simply not propagating through the problematic

long-range paths. The truncation length κ is a hyperparameter: too small and the model cannot learn any temporal dependencies beyond κ steps; too large and we recover the problems of full BPTT. In practice, values of κ in the range of 20 to 200 are common.

Long Short-Term Memory (LSTM) A problem with the simple RNN is that it is difficult for the RNN to store all the useful information about the sequence in just the hidden state $\mathbf{h}_t$. We can think of $\mathbf{h}_t$ as the models 'short-term memory' and its weight parameters as its 'long-term memory.' The *long short-term memory* (LSTM) architecture (**?**) adds another storage mechanism, the *cell state* $\mathbf{c}_t \in \mathbb{R}^H$, that provides a place for the RNN to store information that does not need to be in the current hidden state. The LSTM uses three recurrent *gating* functions, all of which look like the form of the simple RNN above, to control what information gets placed where.

Concretely, the LSTM defines three gates and a candidate cell update at each time step:

$$\begin{aligned}
\mathbf{f}_t &= \texttt{logistic}\left(\mathbf{W}_f^T\mathbf{x}_t + \mathbf{U}_f^T\mathbf{h}_{t-1}\right) && \text{(forget gate)} \\
{}_t &= \texttt{logistic}\left(\mathbf{W}_i^T\mathbf{x}_t + \mathbf{U}_i^T\mathbf{h}_{t-1}\right) && \text{(input gate)} \\
\mathbf{o}_t &= \texttt{logistic}\left(\mathbf{W}_o^T\mathbf{x}_t + \mathbf{U}_o^T\mathbf{h}_{t-1}\right) && \text{(output gate)} \\
\tilde{\mathbf{c}}_t &= \tanh\left(\mathbf{W}_c^T\mathbf{x}_t + \mathbf{U}_c^T\mathbf{h}_{t-1}\right) && \text{(candidate cell)}
\end{aligned} \tag{67}$$

where $\mathbf{W}_f, \mathbf{W}_i, \mathbf{W}_o, \mathbf{W}_c \in \mathbb{R}^{D\times H}$ are the input-to-hidden weight matrices and $\mathbf{U}_f, \mathbf{U}_i, \mathbf{U}_o, \mathbf{U}_c \in \mathbb{R}^{H\times H}$ are the hidden-to-hidden weight matrices for each gate. All three gates use the logistic activation, producing values in $(0, 1)$ that act as soft switches. The cell state and hidden state are then updated as:

$$\begin{aligned}
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + {}_t \odot \tilde{\mathbf{c}}_t \\
\mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t)
\end{aligned} \tag{68}$$

where $\odot$ denotes element-wise (Hadamard) multiplication.

The intuition behind these equations is as follows. The *forget gate* $\mathbf{f}_t$ determines what fraction of the old cell state $\mathbf{c}_{t-1}$ to retain. When $\mathbf{f}_t \approx 1$, the cell state is preserved; when $\mathbf{f}_t \approx 0$, the old cell state is erased. The *input gate* ${}_t$ determines how much of the new candidate cell update $\tilde{\mathbf{c}}_t$ is written to the cell state. Together, the forget and input gates allow the LSTM to selectively remember and forget information. The *output gate* $\mathbf{o}_t$ controls how much of the cell state is exposed as the hidden state $\mathbf{h}_t$, which is then used for predictions and passed to the next time step.

The key insight for why the LSTM mitigates vanishing gradients lies in the cell state update equation: $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + {}_t \odot \tilde{\mathbf{c}}_t$. Notice that the cell state is updated via an *additive* interaction with the previous cell state. If the forget gate is close to one and the input gate is close to zero, the cell state is simply copied forward: $\mathbf{c}_t \approx \mathbf{c}_{t-1}$. In this regime, the gradient of $\mathbf{c}_t$ with respect to $\mathbf{c}_{t-1}$ is approximately the identity, meaning that the gradient can flow backwards through many time steps without vanishing. This is directly analogous to how skip connections in feedforward networks (Section 4) allow gradients to bypass problematic layers. The LSTM's cell state can be thought of as an information highway that allows the gradient to flow unimpeded when the gates are configured appropriately.

Gated Recurrent Unit (GRU) The *gated recurrent unit* (GRU) (**?**) serves the same purpose as the LSTM—enabling the model to learn long-range dependencies—but tries to accomplish this in a more parameter-efficient way. The GRU has only two gating mechanisms instead of three, and it does not maintain a separate cell state. The GRU update equations are:

$$\begin{aligned}
\mathbf{z}_t &= \texttt{logistic}\left(\mathbf{W}_z^T \mathbf{x}_t \;+\; \mathbf{U}_z^T \mathbf{h}_{t-1}\right) && \text{(update gate)} \\
\mathbf{r}_t &= \texttt{logistic}\left(\mathbf{W}_r^T \mathbf{x}_t \;+\; \mathbf{U}_r^T \mathbf{h}_{t-1}\right) && \text{(reset gate)} \\
\tilde{\mathbf{h}}_t &= \tanh\left(\mathbf{W}_h^T \mathbf{x}_t \;+\; \mathbf{U}_h^T \left(\mathbf{r}_t \odot \mathbf{h}_{t-1}\right)\right) && \text{(candidate hidden state)} \\
\mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} \;+\; \mathbf{z}_t \odot \tilde{\mathbf{h}}_t && \text{(new hidden state)}
\end{aligned} \tag{69}$$

where $\mathbf{W}_z, \mathbf{W}_r, \mathbf{W}_h \in \mathbb{R}^{D \times H}$ and $\mathbf{U}_z, \mathbf{U}_r, \mathbf{U}_h \in \mathbb{R}^{H \times H}$. The *update gate* $\mathbf{z}_t$ plays a role similar to both the forget and input gates of the LSTM: it determines how much of the previous hidden state to carry forward versus how much to replace with the new candidate $\tilde{\mathbf{h}}_t$. The *reset gate* $\mathbf{r}_t$ controls how much of the previous hidden state is visible when computing the candidate hidden state. When $\mathbf{r}_t \approx 0$, the candidate ignores the previous hidden state entirely, allowing the model to make a 'fresh start.' When $\mathbf{z}_t \approx 0$, the hidden state is simply copied forward: $\mathbf{h}_t \approx \mathbf{h}_{t-1}$, which again provides a gradient highway similar to the LSTM's cell state.

In practice, neither the LSTM nor the GRU is universally better than the other. They tend to perform similarly on most benchmarks, with the GRU having the advantage of fewer parameters (and therefore being faster to train) and the LSTM sometimes having a slight edge on tasks requiring very fine-grained memory control.

Bidirectional RNNs In many sequence modeling tasks—such as named entity recognition, part-of-speech tagging, or machine translation—the prediction at time t can benefit from information that appears *after* time t in the sequence. A standard (unidirectional) RNN can only incorporate information from $\mathrm{x}_1, \ldots, \mathrm{x}_t$ when computing $\mathbf{h}_t$, since the recurrence runs left to right. A *bidirectional RNN* addresses this limitation by running two RNNs over the sequence: one that processes the sequence from left to right (the forward RNN, with hidden states $\overrightarrow{\mathbf{h}}_t$) and one that processes it from right to left (the backward RNN, with hidden states $\overleftarrow{\mathbf{h}}_t$). The two hidden states at each time step are concatenated to form a representation that captures both past and future context:

$$\mathbf{h}_t \;=\; \left[\overrightarrow{\mathbf{h}}_t,\; \overleftarrow{\mathbf{h}}_t\right] \in \mathbb{R}^{2H}.$$

Each directional RNN has its own set of parameters, so a bidirectional RNN has roughly twice the parameters of a unidirectional one. Note that bidirectional RNNs can only be used when the full sequence is available at prediction time; they are not suitable for tasks that require online or real-time predictions, nor for autoregressive generation where future tokens are not yet available.

Stacking RNN Layers Just as we stacked multiple hidden layers in feedforward networks to create deep NNs, we can stack multiple RNN layers to create *deep RNNs*. In a deep RNN

with L recurrent layers, the hidden state at layer l and time t is:

$$\mathbf{h}_t^{(l)} \;=\; \phi\left(\mathbf{W}_0^{(l)T}\mathbf{h}_t^{(l-1)} \;+\; \mathbf{W}_1^{(l)T}\mathbf{h}_{t-1}^{(l)}\right), \quad l = 1, \ldots, L$$

where $\mathbf{h}_t^{(0)} = \mathbf{x}_t$ is the input at time t and $\mathbf{h}_t^{(l)}$ is the hidden state at layer l. Each layer has its own parameters $\mathbf{W}_0^{(l)}$ and $\mathbf{W}_1^{(l)}$. The intuition is that lower layers may learn to represent local features of the sequence (e.g. syntax in language) while higher layers capture more abstract, long-range features (e.g. semantics or discourse structure). In practice, deep RNNs with more than two or three layers are difficult to train without additional techniques such as skip connections between layers or dropout (applied between layers, not between time steps, to avoid disrupting the temporal dynamics).

Chain Rule of Probability and Autoregressive Generation A key idea underlying sequence models is the *chain rule of probability*. For an output sequence $\mathrm{y}_1, \ldots, \mathrm{y}_T$, the chain rule states that its joint distribution can always be factorized as:

$$p(\mathrm{y}_1, \ldots, \mathrm{y}_T) \;=\; \prod_{t=1}^{T} p(\mathrm{y}_t \mid \mathrm{y}_1, \ldots, \mathrm{y}_{t-1}), \tag{70}$$

where the first factor is understood as just $p(\mathrm{y}_1)$. If we also wish to condition on some input x, then the same logic gives:

$$p(\mathrm{y}_1, \ldots, \mathrm{y}_T \mid \mathrm{x}) \;=\; \prod_{t=1}^{T} p(\mathrm{y}_t \mid \mathrm{y}_1, \ldots, \mathrm{y}_{t-1}, \mathrm{x}). \tag{71}$$

This factorization is extremely useful because it converts modeling a complicated joint distribution over an entire sequence into modeling a collection of one-step-ahead conditional distributions.

The above factorization is what makes *autoregressive generation* possible. Once the model has produced a distribution for the first output, we can either sample from that distribution or take its mode:

$$\hat{y}t \;\sim\; p(\mathrm{y}t \mid \hat{y}1, \ldots, \hat{y}t-1, \mathrm{x}) \qquad \text{or} \qquad \hat{y}t \;=\; \arg\max y \; p(\mathrm{y}_t = y \mid \hat{y}1, \ldots, \hat{y}t-1, \mathrm{x}). \tag{72}$$

We then *feed back* this generated token $\hat{y}_t$ into the sequence model so that it can be used when producing the next output. In practice, generation is usually initialized with a special start-of-sequence token such as `<BOS>`, and it stops once the model generates an end-of-sequence token such as `<EOS>` or reaches a maximum allowed length.

This feedback mechanism is what makes sequence models suitable for both the one-to-many and aligned many-to-many settings. In the one-to-many case, there is one conditioning input x, but the model must still generate a sequence of outputs whose later elements depend on the earlier ones:

$$p(\mathrm{y}_1, \ldots, \mathrm{y}_T \mid \mathrm{x}) \;=\; \prod_{t=1}^{T} p(\mathrm{y}_t \mid \mathrm{y}_1, \ldots, \mathrm{y}_{t-1}, \mathrm{x}). \tag{73}$$

In the aligned many-to-many case, the model also receives new input information as time

advances, and so the factorization becomes:

$$p(\mathrm{y}_1, \ldots, \mathrm{y}_T \mid \mathrm{x}_1, \ldots, \mathrm{x}_T) \;=\; \prod_{t=1}^{T} p(\mathrm{y}_t \mid \mathrm{y}_1, \ldots, \mathrm{y}_{t-1}, \mathrm{x}_1, \ldots, \mathrm{x}_t). \tag{74}$$

Thus, in both cases, previous generations serve as part of the context for future generations, allowing the model to produce coherent, temporally dependent outputs rather than predicting each time step independently.

Autoregressive Decoding via Feedback The decoder in a sequence-to-sequence model is precisely an implementation of the chain rule factorization for the unaligned many-to-many case:

$$p(\mathrm{y}_1, \ldots, \mathrm{y}_{T'} \mid \mathrm{x}_1, \ldots, \mathrm{x}_T) \;=\; \prod_{t'=1}^{T'} p(\mathrm{y}_{t'} \mid \mathrm{y}_1, \ldots, \mathrm{y}_{t'-1}, \mathrm{x}_1, \ldots, \mathrm{x}_T). \tag{75}$$

The encoder absorbs the full input sequence $\mathrm{x}_1, \ldots, \mathrm{x}_T$ into a representation, and then the decoder generates the target sequence one token at a time. After producing the token at time $t' - 1$, that generated token is embedded and fed back into the decoder so that it can produce the token at time t'.

If we let $e(\hat{y}t' - 1)$ denote the embedding of the previously generated token, then a simple decoder recurrence can be written as:

$$\mathbf{h}t'^{\mathrm{dec}} \;=\; \phi\left(\mathbf{W}_{\mathrm{dec},0}^{T} e(\hat{y}t' - 1) \;+\; \mathbf{W}\mathrm{dec}, 1^{T}\mathbf{h}_{t'-1}^{\mathrm{dec}}\right), \tag{76}$$

with the predictive distribution obtained from the decoder hidden state:

$$p(\mathrm{y}_{t'} \mid \hat{y}1, \ldots, \hat{y}t' - 1, \mathrm{x}_1, \ldots, \mathrm{x}_T) \;=\; \mathrm{Categorical}\left(\mathrm{y}_{t'}; \boldsymbol{\pi}_{t'} = \mathtt{softmax}\left(\mathbf{V}^{T}\mathbf{h}_{t'}^{\mathrm{dec}}\right)\right). \tag{77}$$

Hence the decoder is a generative feedback loop: hidden state $\rightarrow$ predictive distribution $\rightarrow$ generated token $\rightarrow$ next hidden state.

This also clarifies the difference between training time and generation time. During generation, the decoder must condition on *its own* past outputs, since those are all that are available. During training, however, we usually replace $\hat{y}t' - 1$ with the true previous token $yt' - 1$ from the training sequence. As we discuss next, this strategy is known as *teacher forcing*.

7.2 Overview of Architectures

Having defined the basic recurrent building blocks, we now consider the different input-output configurations in which RNNs can be deployed. The flexibility of the recurrent architecture allows it to handle a variety of structured prediction problems, each with a different relationship between the input and output sequences.

One-to-Many In this problem, we model many outputs given one input, which we can do via the autoregressive factorization:

$$p(\mathrm{y}_1, \ldots, \mathrm{y}_t, \ldots, \mathrm{y}_T|\mathrm{x}) \;=\; \prod_{t=1}^{T} p(\mathrm{y}_t|\mathrm{y}_1, \ldots, \mathrm{y}_{t-1}, \mathrm{x}).$$

An example of a one-to-many task is image captioning: the input x is an image (possibly processed by a CNN to produce a fixed-length feature vector), and the outputs $\mathrm{y}_1, \ldots, \mathrm{y}_T$ are the words of a caption describing the image. Notice the autoregressive structure: each predicted word depends not only on the image but also on all previously predicted words, which is crucial for generating coherent, grammatical sentences.

We derive the loss function for these types of autoregressive factorizations in the usual way, starting with the KL divergence:

$$\begin{aligned}
&\mathbb{E}_{\mathrm{x}}\mathbb{KLD}\left[\mathbb{P}(\mathrm{y}_1, \ldots, \mathrm{y}_T|\mathrm{x}) \;||\; p(\mathrm{y}_1, \ldots, \mathrm{y}_T|\mathrm{x})\right] \;=\; \mathbb{E}_{\mathrm{x}}\mathbb{E}_{\mathbb{P}(\mathrm{y}_1,\ldots,\mathrm{y}_T|\mathrm{x})}\left[-\log p(\mathrm{y}_1, \ldots, \mathrm{y}_T|\mathrm{x})\right] \;+\; \text{const.} \\
&\approx\; \frac{1}{N}\sum_{n=1}^{N} -\log p(y_{n,1}, \ldots, y_{n,T}|x_n) \;+\; \text{const.} \\
&=\; \frac{1}{N}\sum_{n=1}^{N} -\log\left\{\prod_{t=1}^{T} p(y_{n,t}|y_{n,t-1}, \ldots, y_{n,1}, x_n)\right\} \;+\; \text{const.} \\
&=\; \frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T} -\log p(y_{n,t}|y_{n,t-1}, \ldots, y_{n,1}, x) \;+\; \text{const.}
\end{aligned} \tag{78}$$

One would continue the loss function derivation by plugging in the distributional assumptions in the usual way. Notice that this is a sum of cross-entropy terms, one per time step, meaning that the model receives a training signal at every position in the output sequence.

Many-to-One This is the case we already covered in our review sentiment classification example:

$$p(\mathrm{y}|\mathrm{x}_1, \ldots \mathrm{x}_t, \ldots, \mathrm{x}_T).$$

Note there is no auto-regressive factorization here since we have only one output. Other examples of many-to-one tasks include document classification, where a variable-length document is mapped to a single category, and activity recognition from video, where a sequence of frames is mapped to an activity label. A practical consideration for many-to-one tasks is that the final hidden state $\mathbf{h}_T$ must summarize the entire sequence. For long sequences, this can be challenging, which is one motivation for attention mechanisms (discussed in the next section). An alternative to using only $\mathbf{h}_T$ is to aggregate the hidden states across all time steps, for example by averaging: $\bar{\mathbf{h}} = (1/T)\sum_{t=1}^{T} \mathbf{h}_t$, which gives every time step equal influence on the final representation.

Aligned Many-to-Many In an aligned many-to-many problem, when predicting y_t, we observe only $\mathrm{x}_1, \ldots, \mathrm{x}_t$:

$$p(\mathrm{y}_1, \ldots, \mathrm{y}_t, \ldots, \mathrm{y}_T|\mathrm{x}_1, \ldots \mathrm{x}_t, \ldots, \mathrm{x}_T) \;=\; \prod_{t=1}^{T} p(\mathrm{y}_t|\mathrm{y}_1, \ldots, \mathrm{y}_{t-1}, \mathrm{x}_1, \ldots, \mathrm{x}_t).$$

The term 'aligned' refers to the fact that there is a one-to-one correspondence between input and output time steps: each y_t is associated with a specific x_t. Examples include language modeling (predicting the next word given all preceding words), part-of-speech tagging (assigning a grammatical tag to each word), and named entity recognition (labeling

each word as a person, organization, location, etc.). In the language modeling case, the inputs and outputs are shifted by one position: $\mathrm{y}_t = \mathrm{x}_{t+1}$, meaning the model is trained to predict the next token given all preceding tokens. This is the training setup used for most modern large language models prior to the adoption of the transformer architecture.

Unaligned Many-to-Many In an un-aligned many-to-many problem, we see all the inputs before predicting any outputs. Moreover, the number of inputs and outputs can differ (T' outputs, T inputs):

$$p(\mathrm{y}_1, \ldots, \mathrm{y}_{t'}, \ldots, \mathrm{y}_{T'} | \mathrm{x}_1, \ldots \mathrm{x}_t, \ldots, \mathrm{x}_T) \;=\; \prod_{t'=1}^{T'} p(\mathrm{y}_{t'} | \mathrm{y}_1, \ldots, \mathrm{y}_{t'-1}, \mathrm{x}_1, \ldots, \mathrm{x}_T)$$

This is the most general and arguably the most practically important configuration. Machine translation is the canonical example: given an English sentence of T words, produce a French translation of T' words, where T' need not equal T. Other examples include speech recognition (mapping a long sequence of audio frames to a shorter sequence of words), text summarization (mapping a long document to a short summary), and question answering (mapping a question-context pair to an answer).

7.3 Unaligned Sequence-to-Sequence with Encoder-Decoder Architecture

The encoder-decoder architecture is designed for unaligned many-to-many problems (a.k.a. sequence-to-sequence) problems. The encoder and decoder both consist of RNNs (or any of the gated variants such as LSTMs or GRUs). The encoder takes in the input sequence $\mathrm{x}_1, \ldots, \mathrm{x}_T$ and then passes its final hidden state to the decoder, which autoregressively generates $\mathrm{y}_1, \ldots, \mathrm{y}_t, \ldots, \mathrm{y}_T$.

More precisely, the encoder RNN processes the input sequence to produce a sequence of hidden states $\mathbf{h}_1^{\text{enc}}, \ldots, \mathbf{h}_T^{\text{enc}}$, and its final hidden state serves as the *context vector*: $\mathbf{c} = \mathbf{h}_T^{\text{enc}}$. The decoder is then initialized with this context vector, setting $\mathbf{h}_0^{\text{dec}} = \mathbf{c}$, and produces output tokens one at a time. At each decoder time step t', the decoder computes:

$$\mathbf{h}_{t'}^{\text{dec}} \;=\; \phi\left(\mathbf{W}_{\text{dec},0}^T \mathbf{y}_{t'-1} \;+\; \mathbf{W}_{\text{dec},1}^T \mathbf{h}_{t'-1}^{\text{dec}}\right)$$

where $\mathbf{y}_{t'-1}$ is the embedding of the previously generated output token. During training, $\mathbf{y}_{t'-1}$ is set to the ground-truth token at position $t'-1$ (a technique called *teacher forcing*), while during generation, it is the model's own prediction from the previous step.

Teacher Forcing and Exposure Bias Teacher forcing has the benefit of providing the decoder with correct context during training, which stabilizes and accelerates learning. However, it introduces a discrepancy between training and inference: during training, the decoder always sees ground-truth previous tokens, but during inference, it must condition on its own (potentially incorrect) predictions. This discrepancy is known as *exposure bias*. When the model makes an error during generation, that error can compound: an incorrect prediction at step t' leads to a hidden state that the model never encountered during training, potentially causing a cascade of further errors. Strategies to mitigate exposure bias include

scheduled sampling (**?**), which randomly replaces the ground-truth token with the model's own prediction during training with an increasing probability over time, and training with reinforcement learning objectives that directly optimize sequence-level metrics.

This architecture, while having some initial success, is sub-optimal because it is difficult for the encoder's final hidden state to capture all information that is relevant to the decoder. The entire input sequence—which could be hundreds of words long—must be compressed into a single fixed-length vector $\mathbf{c}$. This *information bottleneck* means that details from early parts of the input tend to be lost, particularly for long sequences. This limitation necessitated the invention of the attention mechanism, which we discuss in the next section.

8 Attention & Transformers

The sequence-to-sequence models that were introduced around 2014 have a problem that is somewhat analogous to the problem solved by LSTMs: the information that should be in the hidden state varies through time, and for the encoder-decoder architecture in particular, the information accessible to the decoder is bottlenecked by the encoder's last hidden representation. This motivated the *attention mechanism*: a way to dynamically look back through time and retrieve the information that is relevant to the current time step.

8.1 Attention

Attention is defined as follows. Let $\mathfrak{D} = \{(\mathbf{k}_1, \mathbf{v}_1), \ldots, (\mathbf{k}_M, \mathbf{v}_M)\}$ be a 'database' of M-tuples representing key-value pairs. These key-value pairs will be real-valued vectors: $\mathbf{k}_m \in \mathbb{R}^{D_k}$, $\mathbf{v}_m \in \mathbb{R}^{D_v}$. Moreover, denote the query vector as $\mathbf{q} \in \mathbb{R}^{D_k}$. Attention over $\mathfrak{D}$ is written as:

$$\texttt{Attention}\left(\mathbf{q}; \mathfrak{D}\right) \;=\; \sum_{m=1}^{M} \alpha(\mathbf{q}, \mathbf{k}_m) \cdot \mathbf{v}_m \tag{79}$$

where $\{\alpha(\mathbf{q}, \mathbf{k}_1), \ldots, \alpha(\mathbf{q}, \mathbf{k}_M)\}$ are the attention weights given to every value in the database. Usually they are constrained such that $0 \leq \alpha(\mathbf{q}, \mathbf{k}_m) \leq 1$ and $\sum_{m=1}^{M} \alpha(\mathbf{q}, \mathbf{k}_m) = 1$. The intuition is that these weights quantify the degree of similarity between the query and each key. There are two extreme cases on note. When $\alpha(\mathbf{q}, \mathbf{k}_m) = 1$, then the attention mechanism simply returns $\mathbf{v}_m$. When $\alpha(\mathbf{q}, \mathbf{k}_m) = 1/M \;\; \forall m$, attention returns the average value vector: $(1/M) \sum_{m=1}^{M} \mathbf{v}_m$. Thus we can think of attention as computed the 'weighted average' of the value vectors.

Computing the attention weights Computing the attention weights is commonly done by (i) taking the inner product between the query and key vectors, and then (ii) transforming those inner products by the softmax function—the same one we used as the inverse link for multi-class classification. We can write this computation as:

$$\begin{aligned} \left[\alpha(\mathbf{q}, \mathbf{k}_1), \ldots, \alpha(\mathbf{q}, \mathbf{k}_M)\right] \;&=\; \texttt{softmax}\left(\frac{\mathbf{q}^T \mathbf{K}^T}{\sqrt{D_k}}\right) \\ \text{such that} \;\; \alpha(\mathbf{q}, \mathbf{k}_m) \;&=\; \frac{\exp\left\{\frac{\mathbf{q}^T \mathbf{k}_m}{\sqrt{D_k}}\right\}}{\sum_{j=1}^{M} \exp\left\{\frac{\mathbf{q}^T \mathbf{k}_j}{\sqrt{D_k}}\right\}} \end{aligned} \tag{80}$$

where $\mathbf{K} \in \mathbb{R}^{M \times D_k}$ is a matrix of all the key vectors stacked together such that each row of $\mathbf{K}$ corresponds to a key in the database. Dividing by $\sqrt{D_k}$ is just a normalization heuristic that controls the magnitude of the inner product as the query and key vectors grow in length / dimensionality.

Batched Computation Given a batch of N queries $\mathbf{Q} \in \mathbb{R}^{N \times D_k}$, which represents the query vectors stacked such that each query is a row, we can compute a batched version of attention as follows. This is how it is usually implemented in practice:

$$\texttt{Attention}\left(\mathbf{Q}; \mathfrak{D}\right) \;=\; \texttt{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{D_k}}\right)\mathbf{V}, \tag{81}$$

which will yield an output matrix of size $N \times D_v$, the attention mechanism carried out for all N queries. The linear algebra works out because, first, the product $\mathbf{Q}\mathbf{K}^T$ is of size $N \times M$. Second, applying the softmax to $\mathbf{Q}\mathbf{K}^T/\sqrt{D_k}$ should be done row-wise, such that each row sums to one. Application of the softmax leaves the dimensionality unchanged. We then multiply the $N \times M$ softmax-output with $\mathbf{V} \in \mathbb{R}^{M \times D_v}$, a matrix of all value vectors stacked horizontally. The inner dimensions already match, with the matrix multiplication resulting in a $(N \times D_v)$-sized output.

Multi-Head Attention We may wish to define several attention mechanisms, hoping that ensembling them produces better performance by the database of each representing distinct information. Yet, defining E independent attention mechanisms can be costly in terms of both memory and computation. *Multi-head attention* presents a simple way to balance the benefits of multiple attention mechanisms and these costs. Let $\{\mathbf{A}_1, \ldots, \mathbf{A}_E\}$ denote (batched) outputs of E attention mechanisms such that $\mathbf{a}_e$ is computed as:

$$\mathbf{A}_e \;=\; \texttt{softmax}\left(\frac{\left(\mathbf{Q}\mathbf{W}_{q,e}\right)\left(\mathbf{K}\mathbf{W}_{k,e}\right)^T}{\sqrt{D_{e,k}}}\right)\left(\mathbf{V}\mathbf{W}_{e,v}\right) \tag{82}$$

where $\mathbf{W}_{q,e} \in \mathbb{R}^{D_k \times D_{e,k}}$, $\mathbf{W}_{k,e} \in \mathbb{R}^{D_k \times D_{e,k}}$, and $\mathbf{W}_{v,e} \in \mathbb{R}^{D_v \times D_{e,v}}$ are trainable weight matrices for the e-th attention head. Thus multi-head attention takes a 'base' database and perturbs it by multiplying the queries, keys, and values by parameters that are specific to the e-th head. This drastically saves in memory costs since the overhead of multi-head attention does not scale with M. The final output of multi-head attention is computed by taking a linear transformation of all E head outputs:

$$\texttt{Multi-Head Attention}(\mathbf{A}_1, \ldots, \mathbf{A}_E; \mathbf{W}_o) \;=\; [\mathbf{A}_1, \ldots, \mathbf{A}_E]\mathbf{W}_o$$

where $[\mathbf{A}_1, \ldots, \mathbf{A}_E]$ represents a concatenation of all E heads and is of size $N \times \sum_e D_{e,v}$, and $\mathbf{W}_o$ is a matrix of trainable parameters with size $\sum_e D_{e,v} \times D_o$ and.

Self-Attention Given a sequence of M 'tokens' $\mathbf{X} = \{\mathbf{x}_1, \ldots, \mathbf{x}_m, \ldots, \mathbf{x}_M\}$, $\mathbf{x}_m \in \mathbb{R}^D$, we can define *self-attention* as an attention mechanism with the M tokens serving as all

queries, keys, and values:

$$\texttt{Self-Attention}\left(\mathbf{Q}=\mathbf{X};\mathfrak{D}=\{(\mathbf{x}_m,\mathbf{x}_m)\}_{m=1}^{M}\right) \;=\; \texttt{softmax}\left(\frac{\mathbf{X}\mathbf{X}^T}{\sqrt{D}}\right)\mathbf{X}. \tag{83}$$

We can think of self-attention as returning, for each token $\mathbf{x}_m$, a representation that averages the other tokens, with the most weight being places on the tokens that are most similar to $\mathbf{x}_m$ (according to inner their product). A notable aspect of self-attention is that it admits parallel computation. Thinking of the M tokens as a sequence, self-attention can quickly computing similarities across the sequence. An RNN, on the other hand, does not admit such parallel computations as the sequence must be absorbed token-by-token, requiring M time steps.

8.2 Encoder-Decoder Architecture with Attention

Now let's return to the encoder-decoder sequence-to-sequence model, but now defining the decoder with an attention mechanism. Define a sequence of source tokens $\{\mathbf{x}_1,\ldots,\mathbf{x}_t,\ldots,\mathbf{x}_T\}$, which could be the prompt to a chatbot, and a sequence of target tokens $\{\mathbf{y}_1,\ldots,\mathbf{y}_{t'},\ldots,\mathbf{y}_{T'}\}$, which could represent the chatbot's response. Recall for the previous version of the seq-to-seq encoder, the encoder output a final context vector that was just the last hidden state: $\mathbf{c}=\mathbf{h}_T$. This is the problem: all the information relevant to the decoder must already be represented within $\mathbf{h}_T$.

Instead, we can define a decoder with attention (that is, an *attentive decoder*) by giving it a context vector that changes over time, $\mathbf{c}_{t'}$, and is computed via attention:

$$\mathbf{c}_{t'} \;=\; \sum_{t=1}^{T}\alpha(\mathbf{q}=\mathbf{s}_{t'-1},\mathbf{k}_t=\mathbf{h}_t)\cdot\mathbf{h}_t \tag{84}$$

where $\mathbf{s}_{t'-1}$ is the decoder's hidden state at the previous time step and $\mathbf{h}_t$ is the encoder's hidden state at time step t. Thus we see that the query will be the decoder's previous hidden state, the keys will be the encoder's hidden states, and the values will also be the encoder's hidden states. This formulation allows the decoder, via its previous hidden state $\mathbf{s}_{t'-1}$, to look into the encoder and retrieve hidden states that are the most similar to the decoder's current state. The context will then be used to compute the next hidden state of the decoder: $\mathbf{s}_{t'}=g\left(\mathbf{y}_{t'-1},\mathbf{c}_{t'},\mathbf{s}_{t'-1}\right)$, where $g(\cdot)$ is again the equation for simple recurrence, an LSTM, a GRU, etc.

8.3 The Transformer

Given the success of attention in seq-to-seq models built from recurrent networks, Vaswani et al. (2017) asked: *is attention all you need?* That is, do we actually need recurrent NNs in our sequence-to-sequence models? Can we just building everything from (self) attention layers? This will certainly have a benefit for computational efficiency, given that self-attention can be computed in parallel. The answer is *yes!*: Vaswani et al. (2017) introduced an encoder-decoder architecture that does not use RNNs, instead using multi-head self-attention. This architecture is called the *transformer*, and we'll go into its inner workings below. For purposes of this introduction to the transformer, we will assume an

unaligned sequence-to-sequence task for which we have a T-length source sequence $\boldsymbol{\tau}_x = \{\tau_{x,1}, \ldots, \tau_{x,t}, \ldots, \tau_{x,T}\}$ and a T'-length target sequence $\boldsymbol{\tau}_y = \{\tau_{y,1}, \ldots, \tau_{y,t'}, \ldots, \tau_{y,T'}\}$. For a language application, each $\tau_{x,t} \in \mathbb{N}^{\geq 0}$ and $\tau_{y,t'} \in \mathbb{N}^{\geq 0}$ is an index into a pre-specified vocabulary. This model will be trained with the unaligned sequence-to-sequence loss derived above in Equation XXX:

$$\begin{aligned} &\ell\left(\boldsymbol{\theta}; \{\boldsymbol{\tau}_{x,n}, \boldsymbol{\tau}_{y,n}\}_{n=1}^{N}\right) \; = \\ &\frac{1}{N}\sum_{n=1}^{N}\sum_{t'=1}^{T'} -\log \texttt{softmax}_{\tau_{y,n,t'}}\left(\texttt{Transformer}\left(\tau_{y,n,t'-1}, \ldots, \tau_{y,n,1}, \tau_{x,n,T}, \tau_{x,n,1}\right)\right) \end{aligned} \tag{85}$$

where $\texttt{softmax}_{\tau_{y,n,t'}}$ denotes that we are indexing into to softmax (vector) output at dimension $\tau_{y,n,t'}$, which would correspond to the index mapping for the token observed at time t' of the target sequence. The transformer itself is broken down into an encoder and a decoder:

$$\begin{aligned} &\texttt{Transformer}\left(\tau_{y,n,t'-1}, \ldots, \tau_{y,n,1}, \tau_{x,n,1}, \tau_{x,n,T}\right) \; = \\ &\qquad\qquad \texttt{Decoder}\left(\; \tau_{y,n,t'-1}, \ldots, \tau_{y,n,1}, \texttt{Encoder}\left(\tau_{x,n,1}, \tau_{x,n,T}\right)\; \right) \end{aligned} \tag{86}$$

such that the decoder will take as input the target sequence up to the current time step $(\tau_{y,n,t'-1}, \ldots, \tau_{y,n,1})$ and the output of the encoder. The encoder will take in all of the source sequence $\boldsymbol{\tau}_x$. Below we will examine the encoder and decoder architectures, in that order.

8.3.1 Encoder

The purpose of the transformer's encoder is to take in the information from the source sequence $\boldsymbol{\tau}_x$. The encoder is comprised of L_{enc} layers, each of which has two components or sub-layers: one step of multi-head self-attention and application two fully-connected feed-forward layers applied position-wise to the sequence. Yet before we describe those two steps in more detail, we first must detail how the input $\boldsymbol{\tau}_x$ is processed.

Embeddings For every possible token, we first need to find its corresponding *embedding*, which is just a real-valued representation that will be adapted by learning. Given a matrix $\mathbf{E}_x \in \mathbb{R}^{V \times D_x}$, where V is the total number of tokens that we would consider valid in the input sequence and D_x is a user-specified embedding size, we will extract the rows that correspond to the currently observed sequence $\boldsymbol{\tau}_{x,n}$ to construct a new matrix $\mathbf{X} \in \mathbb{R}^{T \times D_x}$. This matrix has T rows since we assume there are T elements in the source sequence—one row for the representation of every token.

Positional Encodings We next need to address a flaw in self-attention. Like fully-connected layers, self-attention layers are agnostic to the position of each input. That is, if you train two self-attention layers, each with a different permutation of the tokens, gradient descent (up to optimization pathologies) can train each self-attention layer such that they have the same output. This is not true for convolutional layers and recurrent units, which encode a notion of space and time, respectively. Yet if we wish to apply self-attention to data (such as language) for which its order / time / sequence matters, then this is a problem.

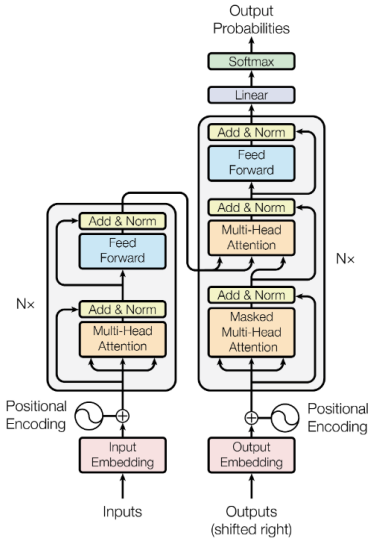


Figure 16: *Transformer Architecture.* Image reproduced from Vaswani et al. (2017).

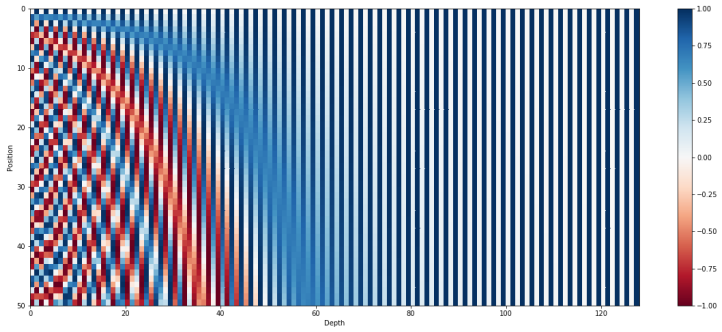


Figure 17: *Visualization of Positional Encodings.* The above heat maps shows the values of $\mathbf{P}$, the matrix of positional encodings. Image reproduced from Kazemnejad (2019).

A somewhat hack-y but commonly successful way to fix this is via *positional encodings*. We want to add some constant value to the tokens such that their position in the sequence can be known (and used) by the model. Given our embedding matrix of the T-length sequence $\mathbf{X} \in \mathbb{R}^{T \times D_x}$, its positional encoders are another matrix $\mathbf{P} \in [-1, 1]^{T \times D_x}$. The two are added together to form a new representation of the sequence:

$$\tilde{\mathbf{X}} \;=\; \mathbf{X} \;+\; \mathbf{P}_x, \quad \text{s.t.} \quad p_{t,d2} = \sin\left(\frac{t}{10,000^{2d/D_x}}\right), \quad p_{t,d2+1} = \cos\left(\frac{t}{10,000^{2d/D_x}}\right).$$

A visualization of $\mathbf{P}$ is provided in Figure 17. The idea is that the sine and cosine functions have a periodicity that changes according to the time index t and the dimension d in order to give a unique positional marker to each value of $\mathbf{X}$. One could imagine more naive approaches—such as populating $\mathbf{P}$ with the indicies directly—but this would drastically vary the scale of $\mathbf{P}$ and likely overwhelm the information in $\mathbf{X}$. The sine and cosine functions do not have this problem since they are bounded in [-1, 1], thereby adding a small but still noticeable (to the transformer) amount of positional information.

Multi-Head Self-Attention We now move on to the core components of the transformer's encoder, shown in the left-side gray box in Figure 16. The first step of computation—now that the embeddings have been retrieved and the positional encodings added—is to pass $\tilde{\mathbf{X}}$ into a multi-head self-attention (MHSA) mechanism. Each head is computed as:

$$\mathbf{A}_e \;=\; \texttt{softmax}\left(\frac{\left(\tilde{\mathbf{X}}\mathbf{W}_{q,e}\right)\left(\tilde{\mathbf{X}}\mathbf{W}_{k,e}\right)^T}{\sqrt{D_{e,k}}}\right)\left(\tilde{\mathbf{X}}\mathbf{W}_{e,v}\right) \tag{87}$$

where $\mathbf{W}_{q,e}$, $\mathbf{W}_{k,e}$, and $\mathbf{W}_{e,v}$ are the usual head-specific parameters. Vaswani et al. (2017) use $E = 8$ attention heads, each with $D_{e,k} = 64$ dimensions.

First Use of LayerNorm & Skip-Connection The next step in the architecture is to take the output of MHSA, add back the input as a skip connection, and pass the sum into a step of LayerNorm:

$$\tilde{\mathbf{X}}_1 \;=\; \texttt{LayerNorm}\left(\tilde{\mathbf{X}} \;+\; \texttt{MHSA}\left(\tilde{\mathbf{X}}\right)\right). \tag{88}$$

Notice that, because of the purely additive skip connection, the output of the MHSA must be the exact same size as its input, i.e. $T \times D_x$.

Position-Wise Feedforward Network The next step in the transformer's encoder is to apply a simple feedforward neural network to each row of $\tilde{\mathbf{X}}_1$. Thus we call the application *position-wise* since this network is applied to each row of $\tilde{\mathbf{X}}_1$ independently and identically:

$$\texttt{PWFF}(\tilde{\mathbf{X}}_1) \;=\; \begin{bmatrix} \texttt{ReLU}\left(\tilde{\mathbf{x}}_{1,1}\mathbf{W}_0\right)\mathbf{W}_1 \\ \vdots \\ \texttt{ReLU}\left(\tilde{\mathbf{x}}_{1,T}\mathbf{W}_0\right)\mathbf{W}_1 \end{bmatrix}$$

where $\tilde{\mathbf{x}}_{1,t} \in \mathbb{R}^{1 \times D_x}$ denotes the t-th row of $\tilde{\mathbf{X}}_1$. Because we will eventually use a skip connection with the PWFF network, like we did before with MHSA, the output must again

preserve the dimensionality of $\tilde{\mathbf{X}}_1$. The PWFF parameters are of sizes: $\mathbf{W}_0 \in \mathbb{R}^{D_x \times H}$ and $\mathbf{W}_1 \in \mathbb{R}^{H \times D_x}$, where the dimensionality H is free to be chosen by the user. Vaswani et al. (2017) use $H = 2048$.

Second Use of LayerNorm & Skip-Connection The final step in each layer of the encoder is to again apply a skip connection and LayerNorm:

$$\tilde{\mathbf{X}}_2 \;=\; \texttt{LayerNorm}\left(\tilde{\mathbf{X}}_1 \;+\; \texttt{PWFF}\left(\tilde{\mathbf{X}}_1\right)\right). \tag{89}$$

Again the dimensionality should be preserved such that $\tilde{\mathbf{X}}$, $\tilde{\mathbf{X}}_1$, and $\tilde{\mathbf{X}}_2$ are all of the same size, specifically $T \times D_x$.

Multiple Layers Encoders are comprised of multiple layers form by repeating the above operations—namely, MHSA, skip connection + LayerNorm, PWFF transformation, and skip connection + LayerNorm. Denote the total number of encoder layers as L_{enc}. We denote the output of each LayerNorm as $\tilde{\mathbf{X}}_{l,1}$ and $\tilde{\mathbf{X}}_{l,2}$, where the l index denotes the layer and 1 or 2 denotes if it's the output of the first or second application of LayerNorm. The transformer's encoder will produce a final representation over which the decoder will attend:

$$\tilde{\mathbf{X}}_{L_{enc},2} \;=\; \texttt{Encoder}\left(\tau_{x,n,1}, \ldots, \tau_{x,n,T}\right).$$

In other words, the decoder has access to the data sequence $\tau_{x,n,1}, \ldots, \tau_{x,n,T}$ via the encoder's final representations for each element of the sequence. Vaswani et al. (2017) use $L_{enc} = 6$ encoder layers.

8.3.2 Decoder

Now moving on to the decoder, recall that it consumes, in addition to the final output of the encoder, the output sequence up to the current time step t': $\tau_{y,n,1}, \ldots, \tau_{y,n,t'-1}$. Yet during training we of course know this full sequence as it is in our training data. It is only during generation time when we do not and the tokens at all time steps come from the model itself. Thus, we need an efficient way to 'hide' the remainder of the sequence, $\tau_{y,n,t'}, \ldots, \tau_{y,n,T'}$ from the decoder. This motivates our first discussion point: masking.

The decoder is comprised of L_{dec} layers, each of which has three sub-layers: (i) masked multi-head self-attention over the target sequence, (ii) multi-head attention over the encoder's output (often called encoder-decoder attention or cross-attention), and (iii) a position-wise feedforward network. As with the encoder, each sub-layer is followed by a skip connection and LayerNorm. Vaswani et al. (2017) use $L_{dec} = 6$ decoder layers. We now describe each component in detail.

Embeddings, Positional Encodings, and Masking Analogous to the encoder, the decoder must first map the target token indices to real-valued representations. Given an embedding matrix $\mathbf{E}_y \in \mathbb{R}^{V' \times D_y}$, where V' is the size of the target vocabulary and D_y is the embedding dimension (usually set equal to D_x), we extract the rows corresponding to the target sequence to form $\mathbf{Y} \in \mathbb{R}^{T' \times D_y}$. Just as in the encoder, we add sinusoidal positional

encodings of the same form:

$$\tilde{\mathbf{Y}} \;=\; \mathbf{Y} \;+\; \mathbf{P}_y$$

where $\mathbf{P}_y \in [-1,1]^{T' \times D_y}$ is constructed with the same sine/cosine scheme used for the encoder's positional encodings.

During training, we have access to the entire target sequence, yet the decoder should only attend to positions up to and including $t'-1$ when predicting the token at position t'. This is enforced via a *causal mask* (sometimes called a 'look-ahead mask'). The mask is a matrix $\mathbf{M} \in \{0, -\infty\}^{T' \times T'}$ defined as:

$$m_{i,j} \;=\; \begin{cases} 0 & \text{if } j \le i, \\ -\infty & \text{if } j > i. \end{cases}$$

This mask is added to the attention logits (i.e. the matrix of query-key inner products) *before* the softmax is applied. Because $\exp\{-\infty\} = 0$, positions in the future receive zero attention weight after the softmax, effectively preventing the decoder from 'seeing' tokens it has not yet generated. This masking strategy is more efficient than creating a separate truncated input $\tilde{\mathbf{Y}}_{t'-1}$ for each time step, as it allows the self-attention for all positions to be computed in a single batched matrix operation during training—a technique sometimes called *teacher forcing with causal masking*.

Masked Multi-Head Self-Attention, Skip-Connection, and LayerNorm The first sub-layer of the decoder applies multi-head self-attention to the target sequence representations, incorporating the causal mask. Each attention head e is computed as:

$$\mathbf{A}_e^{(\text{self})} \;=\; \texttt{softmax}\left(\frac{\left(\tilde{\mathbf{Y}}\mathbf{W}_{q,e}^{(\text{self})}\right)\left(\tilde{\mathbf{Y}}\mathbf{W}_{k,e}^{(\text{self})}\right)^T}{\sqrt{D_{e,k}}} \;+\; \mathbf{M} \right) \left(\tilde{\mathbf{Y}}\mathbf{W}_{v,e}^{(\text{self})}\right) \tag{90}$$

where $\mathbf{W}_{q,e}^{(\text{self})}$, $\mathbf{W}_{k,e}^{(\text{self})}$, and $\mathbf{W}_{v,e}^{(\text{self})}$ are the head-specific query, key, and value projection matrices, and $\mathbf{M}$ is the causal mask defined above. As in the encoder, the outputs of all E heads are concatenated and linearly projected. The result is then combined with the input via a skip connection and passed through LayerNorm:

$$\tilde{\mathbf{Y}}_{t',1} \;=\; \texttt{LayerNorm}\left(\tilde{\mathbf{Y}}_{t'} \;+\; \texttt{Masked-MHSA}\left(\tilde{\mathbf{Y}}_{t'}\right)\right). \tag{91}$$

Notice that, just as in the encoder, the skip connection requires that the output of the masked MHSA be the same size as its input, i.e. $T' \times D_y$. The key difference from the encoder's self-attention is the addition of $\mathbf{M}$: without it, position t' could attend to position $t'+1$, leaking information about future tokens and violating the autoregressive factorization that the model is trained to respect.

Multi-Head Attention with Encoder Representations The crucial step that links the encoder and decoder is so-called *encoder-decoder attention* (also known as *cross-attention*). In this sub-layer, the queries come from the decoder's previous sub-layer output $\tilde{\mathbf{Y}}_{t',1}$, while the keys and values come from the final output of the encoder $\tilde{\mathbf{X}}_{L_{enc},2}$. For each attention

head e:

$$\mathbf{A}_e^{(\text{cross})} \;=\; \texttt{softmax}\left(\frac{\left(\tilde{\mathbf{Y}}_{t',1}\mathbf{W}_{q,e}^{(\text{cross})}\right)\left(\tilde{\mathbf{X}}_{L_{enc},2}\mathbf{W}_{k,e}^{(\text{cross})}\right)^T}{\sqrt{D_{e,k}}}\right)\left(\tilde{\mathbf{X}}_{L_{enc},2}\mathbf{W}_{v,e}^{(\text{cross})}\right) \tag{92}$$

where $\mathbf{W}_{q,e}^{(\text{cross})} \in \mathbb{R}^{D_y \times D_{e,k}}$, $\mathbf{W}_{k,e}^{(\text{cross})} \in \mathbb{R}^{D_x \times D_{e,k}}$, and $\mathbf{W}_{v,e}^{(\text{cross})} \in \mathbb{R}^{D_x \times D_{e,v}}$ are trainable parameters specific to each head. This is the mechanism by which the decoder can 'look into' the encoder and retrieve source-sequence information that is relevant to the current step of generation. Intuitively, the decoder forms a query based on what it has generated so far and uses it to find the most relevant parts of the source sequence, weighted by the attention distribution over the encoder's hidden states. Note that no causal mask is applied here: the decoder should be able to attend to all positions in the source sequence, since the entire source is available at decoding time.

The output of all E cross-attention heads is concatenated and linearly projected, just as described for multi-head attention in the encoder. The result is of size $T' \times D_y$.

Skip-Connection and LayerNorm Following the cross-attention, we again apply a skip connection and LayerNorm:

$$\tilde{\mathbf{Y}}_{t',2} \;=\; \texttt{LayerNorm}\left(\tilde{\mathbf{Y}}_{t',1} \;+\; \texttt{MHA}\left(\mathbf{Q} = \tilde{\mathbf{Y}}_{t',1},\; \mathbf{K} = \tilde{\mathbf{X}}_{L_{enc},2},\; \mathbf{V} = \tilde{\mathbf{X}}_{L_{enc},2}\right)\right). \tag{93}$$

The skip connection here adds back $\tilde{\mathbf{Y}}_{t',1}$—the output of the first sub-layer—so that the cross-attention need only learn a residual update to the representation, rather than reconstructing it from scratch.

Position-Wise Feed-Forward Network, Skip-Connection, and LayerNorm The third and final sub-layer of each decoder layer is a position-wise feedforward network (PWFF), identical in form to the one used in the encoder. It is applied independently and identically to each row of $\tilde{\mathbf{Y}}_{t',2}$:

$$\texttt{PWFF}(\tilde{\mathbf{Y}}_{t',2}) \;=\; \begin{bmatrix} \texttt{ReLU}\left(\tilde{\mathbf{y}}_{t',2,1}\mathbf{W}_0^{(\text{dec})}\right)\mathbf{W}_1^{(\text{dec})} \\ \vdots \\ \texttt{ReLU}\left(\tilde{\mathbf{y}}_{t',2,T'}\mathbf{W}_0^{(\text{dec})}\right)\mathbf{W}_1^{(\text{dec})} \end{bmatrix}$$

where $\tilde{\mathbf{y}}_{t',2,j} \in \mathbb{R}^{1 \times D_y}$ denotes the j-th row of $\tilde{\mathbf{Y}}_{t',2}$, and $\mathbf{W}_0^{(\text{dec})} \in \mathbb{R}^{D_y \times H}$ and $\mathbf{W}_1^{(\text{dec})} \in \mathbb{R}^{H \times D_y}$ are the PWFF parameters. Note that these parameters are separate from those of the encoder's PWFF layers, though the functional form is identical. As with the encoder, the intermediate dimensionality H is a hyperparameter; Vaswani et al. (2017) use $H = 2048$. A skip connection and LayerNorm are applied after the PWFF:

$$\tilde{\mathbf{Y}}_{t',3} \;=\; \texttt{LayerNorm}\left(\tilde{\mathbf{Y}}_{t',2} \;+\; \texttt{PWFF}\left(\tilde{\mathbf{Y}}_{t',2}\right)\right). \tag{94}$$

Again, the dimensionality is preserved: $\tilde{\mathbf{Y}}_{t',1}$, $\tilde{\mathbf{Y}}_{t',2}$, and $\tilde{\mathbf{Y}}_{t',3}$ are all of size $T' \times D_y$.

Multiple Layers and Final Output As with the encoder, the decoder is comprised of multiple layers formed by repeating the three sub-layers described above—masked MHSA, cross-attention, and PWFF, each followed by a skip connection and LayerNorm. Denote the total number of decoder layers as L_{dec}. After passing through all L_{dec} layers, the decoder produces a final representation $\tilde{\mathbf{Y}}_{L_{dec},3} \in \mathbb{R}^{T' \times D_y}$. To obtain a distribution over the target vocabulary for each position, we apply a linear projection followed by a softmax:

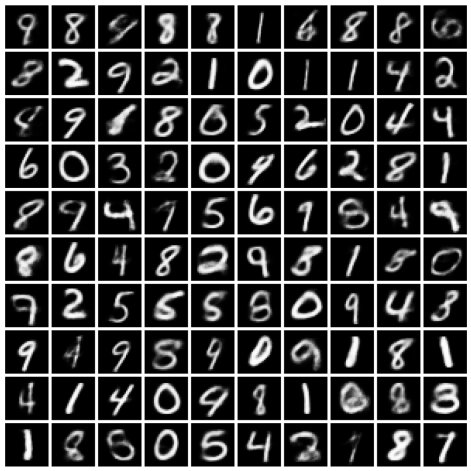
$$p(\boldsymbol{\tau}_{y,t'} = v \mid \boldsymbol{\tau}_{y,1}, \ldots, \boldsymbol{\tau}_{y,t'-1}, \boldsymbol{\tau}_x) \;=\; \texttt{softmax}_v\left(\tilde{\mathbf{y}}_{L_{dec},3,t'}\ \mathbf{W}_{\text{out}}\right),$$

where $\tilde{\mathbf{y}}_{L_{dec},3,t'} \in \mathbb{R}^{1 \times D_y}$ is the t'-th row of the decoder's final output and $\mathbf{W}_{\text{out}} \in \mathbb{R}^{D_y \times V'}$ projects from the model's hidden dimension to the target vocabulary size. It is common to *tie* the output projection weights with the target embedding matrix, i.e. setting $\mathbf{W}_{\text{out}} = \mathbf{E}_y^T$, which reduces the number of parameters and has been found to improve performance in practice.

Summary To summarize, the decoder differs from the encoder in two key respects. First, the decoder's self-attention is *masked* to enforce the autoregressive property: when predicting token t', the model can only attend to tokens at positions $1, \ldots, t'-1$. Second, the decoder includes an additional cross-attention sub-layer that allows it to attend over the encoder's output representations, providing a mechanism for the decoder to condition on the entire source sequence. Together with the encoder, the decoder completes the transformer architecture, enabling end-to-end training for sequence-to-sequence tasks without any recurrent components.

9 Autoencoders and Deep Generative Models

Our focus up to this point has been on supervised learning. But since the early days of NN research there has always been significant interest in *unsupervised learning*, motivated broadly by ideas from artificial intelligence and cognitive science. For example, can NNs minic the ability of humans to learn structure from perceptual signals (e.g., audio, visual) from the world around them? As a concrete example, consider the images of digits shown in Figure 18a. Do these look like images from the MNIST data set? Despite their visual similarity to MNIST, they are *not* from the data set but rather *samples* generated from an NN fit to MNIST. Next consider Figure 18b. These images are not of real people. Rather, the images were also generated by a NN; this one trained on a data set of celebrity images known as *CelebA*. These are cases of what is known as *generative modeling* in DL: the primary goal is to generate novel samples that plausibly could have been part of the training set. We hope to capture the true distribution $\mathbb{P}(\mathbf{x})$ as faithfully as possible. Models based on unsupervised learning have applications ranging from dimensionality reduction to data synthesis, although much of the excitement in this area stems from the desire to build intelligent systems. The intuition is that if our models can perfectly capture the training distribution, then they must 'understand' the data. Models that only discriminate (e.g. classifiers) are then performing an easier 'cognitive' task—just like it is easier to recognize quality art than to produce it.



(a) Samples from a VAE trained on MNIST



(b) Samples from Glow trained on CelebA

Figure 18: Samples from Neural Generative Models. The VAE samples were produced by Doersch (2016), and the Glow samples by Kingma and Dhariwal (2018)—both used with permission of the authors.

9.1 Dimensionality Reduction with Autoencoders

To introduce this class of models, consider the task of dimensionality reduction: we wish to learn a new representation of the data that discards noise and otherwise unimportant information. Deep NNs are performing dimensionality reduction by nature of learning their hidden layers. Yet in that case, the dimensionality reduction is done with respect to the supervision signal (e.g. the class label) so that the information that informs the prediction is preserved rather than a general summary of the data.

The *autoencoder* (AE) (a.k.a. a *diablo network* or *auto-associator*) (Cottrell, 1989; Baldi and Hornik, 1989; Bourlard and Kamp, 1988; Hinton and Salakhutdinov, 2006) is the simplest NN architecture designed for unsupervised learning and dimensionality reduction. The AE's goal is to reconstruct the data from a lossy representation of that same data. Specifically, the model takes an observation $\mathbf{x}$ as input, computes at least one hidden layer $\boldsymbol{h}$, and then tries to predict the observation $\mathbf{x}$ back from $\boldsymbol{h}$. An AE with multiple hidden layers can be defined as:

$$\mathbb{E}\left[\mathbf{x}|\mathbf{x}\right] = \mathbf{g}^{-1}(\mathbf{W}_L^T \boldsymbol{h}_{L-1}), \quad \boldsymbol{h}_l = \phi(\mathbf{W}_l^T \boldsymbol{h}_{l-1}), \quad \boldsymbol{h}_0 = \mathbf{x}, \tag{95}$$

where $\mathbb{E}\left[\mathbf{x}|\mathbf{x}\right]$ is an abuse of notation (and the concept of conditional expectation) but captures that $\mathbf{x}$ serves as both the features and labels in the autoencoding problem. $\mathbf{g}^{-\mathbf{1}}$ is again a link function that maps to the expected value of the reconstruction. $\mathbf{W}$, $\boldsymbol{h}$, and σ are defined as before for feedforward NNs. A depiction of a simple one-hidden-layer AE can be seen in Figure 19a.

As for training an AE, one can derive a loss function just as we did for supervised learning, treating $\tilde{\mathbf{x}}$ as the prediction and $\mathbf{x}$ as the true label. The formulation of $\mathbb{E}\left[\tilde{\mathbf{x}}|\mathbf{x}\right]$ (and in turn, the choice of g^{-1}) will depend on the support of $\mathbf{x}$. If $\mathbf{x}$ is binary valued, then it is natural to choose $p(\tilde{\mathbf{x}}|\mathbf{x}) = \text{Bernoulli}\left(\tilde{\mathbf{x}}; \boldsymbol{\pi} = \mathbf{g}^{-1}(\mathbf{W}_L^T \boldsymbol{h}_{L-1})\right)$. If $\mathbf{x} \in \mathbb{R}$, then g^{-1} would be the identity function and $p(\tilde{\mathbf{x}}|\mathbf{x})$ could be sensibly chosen as the normal distribution.

9.2 Regularized Autoencoders

The basic autoencoder introduced in the previous subsection relies on the *bottleneck*—the fact that the hidden representation $\boldsymbol{h}$ has fewer dimensions than the input $\mathbf{x}$—to prevent the model from learning a trivial identity mapping. If the hidden layer is *undercomplete*, meaning $\dim(\boldsymbol{h}) < \dim(\mathbf{x})$, then the encoder is forced to compress and the representation will capture the dominant structure in the data. But what if we want the hidden layer to be *overcomplete*, i.e. $\dim(\boldsymbol{h}) \geq \dim(\mathbf{x})$? An overcomplete representation has the capacity to encode richer features, but without additional constraints, the AE can simply copy the input through to the output, resulting in a useless representation.

The solution is to add a *regularization penalty* to the AE's reconstruction loss. Recall from Section 3 that we introduced regularized loss functions of the general form:

$$\widetilde{\ell}(\mathbf{W}; \mathcal{D}, \lambda) \;=\; \ell(\mathbf{W}; \mathcal{D}) \;+\; \lambda \cdot \mathcal{R}(\mathbf{W}),$$

where $\ell(\mathbf{W}; \mathcal{D})$ is the data-fit term and $\mathcal{R}(\mathbf{W})$ is a complexity penalty controlled by $\lambda \in \mathbb{R}^+$. In that context, we penalized the *weights* of the model (e.g. via $L2$ regularization) to discourage overfitting. For autoencoders, we will instead (or additionally) regularize the *hidden representation* $\boldsymbol{h}$ or the *behavior of the encoder function* itself. This leads to a family of models known as *regularized autoencoders*, each of which encodes a different inductive bias about what constitutes a good representation. We will examine three prominent members of this family below.

Sparse Autoencoder The *sparse autoencoder* (SAE) (**???**) encourages the hidden representation to be *sparse*: for any given input $\boldsymbol{x}_n$, most of the hidden units should be inactive (i.e. close to zero), with only a small number of units active. The motivation is that many natural signals can be well described by a small number of active features—for instance, a given image patch might activate a few edge detectors and a texture unit while leaving the remaining units dormant. Sparsity thus encourages the AE to discover a set of specialized features, each of which 'fires' only for a particular type of input structure.

There are several ways to impose sparsity. The most straightforward is to add an $L1$ penalty on the hidden activations:

$$\widetilde{\ell}(\mathbf{W}_1, \ldots, \mathbf{W}_L; \mathcal{D}, \lambda) \;=\; \frac{1}{N} \sum_{n=1}^{N} \ell_{\text{recon}}(\boldsymbol{x}_n, \tilde{\boldsymbol{x}}_n) \;+\; \lambda \sum_{d=1}^{D_h} |h_d| \,, \tag{96}$$

where $\ell_{\text{recon}}(\boldsymbol{x}_n, \tilde{\boldsymbol{x}}_n)$ is the reconstruction loss (e.g. the squared loss or negative log-likelihood), h_d is the d-th element of the hidden representation $\boldsymbol{h}$, D_h is the dimensionality of the hidden layer, and λ controls the strength of the sparsity penalty. Just as the $L1$ penalty on weights encourages many weights to be exactly zero (a connection to the Lasso from the statistics literature), the $L1$ penalty on activations encourages many hidden units to be exactly zero for a given input.

An alternative approach that is more natural when the hidden units use sigmoid activations (so that $h_d \in [0, 1]$) is to penalize the average activation of each hidden unit across

the training set. Define the empirical average activation of the d-th hidden unit as:

$$\hat{\rho}_d \;=\; \frac{1}{N}\sum_{n=1}^{N} h_{n,d},$$

where $h_{n,d}$ is the activation of hidden unit d for input $\boldsymbol{x}_n$. We then choose a target sparsity level $\rho \in (0,1)$—typically a small value such as $\rho = 0.05$—and penalize deviations from this target via the KL divergence between two Bernoulli distributions:

$$\mathcal{R}_{\text{sparse}} \;=\; \sum_{d=1}^{D_h} \text{KLD}\left[\text{Bernoulli}(\rho) \;||\; \text{Bernoulli}(\hat{\rho}_d)\right] \;=\; \sum_{d=1}^{D_h} \rho \log\frac{\rho}{\hat{\rho}_d} + (1-\rho)\log\frac{1-\rho}{1-\hat{\rho}_d}. \quad (97)$$

This penalty is zero when $\hat{\rho}_d = \rho$ for all d and grows as the average activations deviate from the target. The KL-based formulation has the advantage of being invariant to the scale of the activations and of penalizing hidden units that are 'always on' more severely than the $L1$ penalty.

In either formulation, the key insight is the same: by constraining which units are active for a given input, sparsity prevents the overcomplete AE from learning the identity and instead encourages it to learn a *distributed, parts-based* representation—one in which each data point is described by a unique combination of a small number of active features.

Denoising Autoencoder The *denoising autoencoder* (DAE) (Vincent et al., 2008, 2010) takes a different approach to regularization: instead of modifying the loss function, it modifies the *input*. During training, the DAE receives a corrupted version of the input and must reconstruct the original, clean version. Specifically, given a data point $\boldsymbol{x}_n$, we sample a corruption $\tilde{\boldsymbol{x}}_n \sim q(\tilde{\mathbf{x}}|\boldsymbol{x}_n)$ from some noise model q and train the AE to recover $\boldsymbol{x}_n$ from $\tilde{\boldsymbol{x}}_n$. The loss function is therefore:

$$\ell(\mathbf{W}_1, \ldots, \mathbf{W}_L; \mathcal{D}) \;=\; \frac{1}{N}\sum_{n=1}^{N} \ell_{\text{recon}}\left(\boldsymbol{x}_n,\; f_{\text{dec}}\left(f_{\text{enc}}(\tilde{\boldsymbol{x}}_n)\right)\right), \quad (98)$$

where $f_{\text{enc}}(\cdot)$ and $f_{\text{dec}}(\cdot)$ denote the encoder and decoder sub-networks, respectively, and $\tilde{\boldsymbol{x}}_n$ is the corrupted input. Note that the reconstruction target is the *clean* input $\boldsymbol{x}_n$, not the corrupted one $\tilde{\boldsymbol{x}}_n$.

Common choices for the corruption process $q(\tilde{\mathbf{x}}|\boldsymbol{x})$ include additive Gaussian noise, $\tilde{\mathbf{x}} = \mathbf{x} + \boldsymbol{\epsilon}$ where $\boldsymbol{\epsilon} \sim \text{N}(\mathbf{0}, \sigma^2\mathbb{I})$, and *masking noise*, where each element of $\mathbf{x}$ is independently set to zero with some probability p. The masking noise variant has a particularly modern flavor: it is conceptually identical to the corruption used in masked language models such as BERT and in masked autoencoders (MAEs) for vision, both of which can be viewed as descendants of the DAE.

Why does denoising help? The intuition is that a model which can reconstruct the original signal from a corrupted version must have learned something about the *structure* of the data manifold, not just a pointwise mapping. In effect, the DAE learns a function that 'projects' corrupted inputs back onto the data manifold, which requires capturing the manifold's local geometry. Vincent (2011) formalized this intuition by showing that, in the limit of small Gaussian noise, the DAE's learning objective is equivalent to learning the

score function $\nabla_{\mathbf{x}} \log \mathbb{P}(\mathbf{x})$—the gradient of the log-density of the data distribution. This connection is notable as it links the DAE to the foundations of score-based generative models and diffusion models, which have become a dominant paradigm in generative modeling.

Unlike the sparse AE, which constrains how many units can be simultaneously active, the DAE constrains the *mapping* by requiring it to be robust to perturbations. The model cannot simply learn the identity function because the identity would map corrupted inputs to corrupted outputs, which would be penalized by the clean reconstruction target.

Contractive Autoencoder The *contractive autoencoder* (CAE) (**?**) regularizes the AE by directly penalizing the sensitivity of the encoder to small perturbations of the input. While the DAE encourages robustness implicitly through corrupted inputs, the CAE does so explicitly by penalizing the *Jacobian* of the encoder.

Consider a single-hidden-layer encoder $f_{\text{enc}}(\mathbf{x}) = \phi(\mathbf{W}_1^T \mathbf{x})$, where $\phi(\cdot)$ is an element-wise activation function. The Jacobian of the encoder is the matrix of partial derivatives:

$$\mathbf{J}_f(\boldsymbol{x}) \;=\; \frac{\partial\, f_{\text{enc}}(\mathbf{x})}{\partial\, \mathbf{x}} \;\in\; \mathbb{R}^{D_h \times D},$$

where D_h is the dimensionality of $\boldsymbol{h}$ and D is the dimensionality of $\mathbf{x}$. Intuitively, row d of $\mathbf{J}_f(\boldsymbol{x})$ tells us how the d-th hidden unit responds to infinitesimal changes in each input dimension. If the Jacobian has large entries, the encoder is highly sensitive to small input perturbations; if the entries are small, the encoder is *contractive*—it maps a neighborhood of $\boldsymbol{x}$ to a smaller region in the hidden space.

The CAE's training objective augments the reconstruction loss with the squared Frobenius norm of the Jacobian:

$$\widetilde{\ell}(\mathbf{W}_1, \ldots, \mathbf{W}_L; \mathcal{D}, \lambda) \;=\; \frac{1}{N} \sum_{n=1}^{N} \ell_{\text{recon}}(\boldsymbol{x}_n, \tilde{\boldsymbol{x}}_n) \;+\; \lambda \, \|\mathbf{J}_f(\boldsymbol{x}_n)\|_F^2 \,, \tag{99}$$

where $\|\mathbf{J}_f(\boldsymbol{x})\|_F^2 = \sum_{d=1}^{D_h} \sum_{j=1}^{D} \left(\frac{\partial h_d}{\partial x_j}\right)^2$ is the sum of the squared partial derivatives. For the single-hidden-layer encoder with element-wise activation, this penalty has a convenient closed form. Writing $\boldsymbol{a} = \mathbf{W}_1^T \boldsymbol{x}$ for the pre-activations, we have $h_d = \phi(a_d)$ and thus $\frac{\partial h_d}{\partial x_j} = \phi'(a_d) \cdot W_{1,jd}$, giving:

$$\|\mathbf{J}_f(\boldsymbol{x})\|_F^2 \;=\; \sum_{d=1}^{D_h} \phi'(a_d)^2 \sum_{j=1}^{D} W_{1,jd}^2 \;=\; \sum_{d=1}^{D_h} \phi'(a_d)^2 \cdot \|\mathbf{w}_{1,d}\|^2, \tag{100}$$

where $\mathbf{w}_{1,d}$ denotes the d-th column of $\mathbf{W}_1$ (the weights feeding into hidden unit d). This expression reveals how the penalty operates: it discourages both large weights (similar to $L2$ weight decay) *and* activations in regions where ϕ' is large (i.e. the steep, near-linear part of a sigmoid). If ϕ is the sigmoid function, for example, then $\phi'(a)$ is close to zero when a is far from the origin (in the saturation regime) and is maximal near $a = 0$. Thus the CAE penalty encourages the hidden units to operate in the saturated regime—where they are insensitive to input perturbations—except along the directions that are needed to achieve a low reconstruction loss. The directions along which the encoder *must* remain sensitive correspond to the tangent directions of the data manifold.

Comparison and Unifying Perspective Each of the three regularized autoencoders can be understood as encoding a different prior about what makes a good representation, while all sharing the same reconstruction-based backbone as the basic AE:

- The **sparse AE** assumes a good representation is one in which only a few features are active for any given input, encouraging the encoder to learn a parts-based decomposition.
- The **denoising AE** assumes a good representation is one that is robust to corruption, encouraging the encoder to learn the structure of the data manifold.
- The **contractive AE** assumes a good representation is one that is locally insensitive to input perturbations, encouraging the encoder to be contractive in directions orthogonal to the data manifold.

It is also worth noting the connection to weight decay (Section 4). Weight decay penalizes the weights $\mathbf{W}_l$ directly, which provides a global constraint on model complexity. The regularized AE penalties instead constrain properties of the *hidden representation* or the *encoder mapping*, which provides a more targeted form of regularization that is specific to the unsupervised learning setting. Naturally, one can combine these approaches—for instance, training a sparse AE with weight decay on the parameters.

9.3 Contrastive Representation Learning

The autoencoder, as introduced in the previous subsection, learns representations by reconstructing the input: the encoder maps $\mathbf{x}$ to a hidden representation $\boldsymbol{h}$, and the decoder maps $\boldsymbol{h}$ back to $\tilde{\mathbf{x}}$, with the loss penalizing discrepancies between $\mathbf{x}$ and $\tilde{\mathbf{x}}$. The implicit assumption is that a good representation is one that *preserves enough information to reconstruct the input*. Yet reconstruction may not be the ideal objective for learning representations that are useful for downstream tasks such as classification. By demanding pixel-level (or feature-level) fidelity, the AE's representation may devote capacity to encoding nuisance factors—lighting conditions, background textures, sensor noise—that are irrelevant to the semantic content of the data.

Contrastive learning offers an alternative: instead of training a representation to reconstruct $\mathbf{x}$, we train it to be *invariant* to semantically irrelevant transformations while remaining *discriminative* across distinct data points. The *Simple Framework for Contrastive Learning of Visual Representations* (SimCLR) (**?**) is one of the most influential instantiations of this idea.

Data Augmentation as an Inductive Bias Central to SimCLR is the use of *data augmentation* to define what information the representation should discard. Let $\mathcal{T}$ denote a family of stochastic transformations (e.g. random cropping, color jittering, Gaussian blurring). Given a data point $\boldsymbol{x}_n$, we sample two transformations $t, t' \sim \mathcal{T}$ independently and apply them to produce two *views*:

$$\tilde{\boldsymbol{x}}_n^{(1)} = t(\boldsymbol{x}_n), \qquad \tilde{\boldsymbol{x}}_n^{(2)} = t'(\boldsymbol{x}_n).$$

These two views form a *positive pair*: they depict the same underlying content but differ in the specific nuisance factors that $\mathcal{T}$ varies. The role of the augmentation family $\mathcal{T}$ is analogous to the choice of bottleneck size in an AE—both encode assumptions about what information is and is not important—but $\mathcal{T}$ does so more explicitly and flexibly. In a basic AE, the bottleneck forces the model to prioritize information that minimizes reconstruction error, which may or may not align with semantic relevance. In SimCLR, the augmentations directly specify which factors (e.g. color, crop position) should *not* influence the representation.

Architecture SimCLR consists of three components. First, an *encoder* $f(\cdot)$—typically a deep CNN such as a ResNet—maps each augmented view to a representation vector:

$$\mathbf{h}_n^{(1)} = f\left(\tilde{\boldsymbol{x}}_n^{(1)}\right), \qquad \mathbf{h}_n^{(2)} = f\left(\tilde{\boldsymbol{x}}_n^{(2)}\right),$$

where $\mathbf{h} \in \mathbb{R}^{D_h}$. Second, a small *projection head* $g(\cdot)$—a feedforward NN with one or two hidden layers—maps each representation to a lower-dimensional space in which the contrastive loss is computed:

$$\mathbf{z}_n^{(1)} = g\left(\boldsymbol{h}_n^{(1)}\right), \qquad \mathbf{z}_n^{(2)} = g\left(\boldsymbol{h}_n^{(2)}\right),$$

where $\mathbf{z} \in \mathbb{R}^{D_z}$ with $D_z \leq D_h$. In practice, it has been found that applying the contrastive loss in this projected space $\mathbf{z}$ rather than in the representation space $\mathbf{h}$ leads to better downstream performance. After training, the projection head $g(\cdot)$ is discarded and only $f(\cdot)$ is used for downstream tasks. Third, a *contrastive loss function* (described next) provides the training signal.

Note that, unlike the AE, there is **no decoder** in this architecture. SimCLR does not attempt to reconstruct $\mathbf{x}$ from $\mathbf{h}$. This is a fundamental departure: the learning signal comes not from reconstruction but from *comparison* between representations.

The NT-Xent Loss SimCLR uses the *normalized temperature-scaled cross-entropy* (NT-Xent) loss (**???**). Consider a minibatch of N data points. After augmentation, we have $2N$ views in total. For a given positive pair $(\boldsymbol{z}_n^{(1)}, \boldsymbol{z}_n^{(2)})$, the remaining $2(N-1)$ views in the minibatch serve as *negative* examples. Define the cosine similarity between two vectors as:

$$\text{sim}(\mathbf{z}_i, \mathbf{z}_j) \;=\; \frac{\mathbf{z}_i^T \mathbf{z}_j}{\|\mathbf{z}_i\| \cdot \|\mathbf{z}_j\|}.$$

The loss for the positive pair indexed by n (taking view 1 as the anchor) is then:

$$\ell_n^{(1)} \;=\; -\log \frac{\exp\left(\text{sim}\left(\boldsymbol{z}_n^{(1)}, \boldsymbol{z}_n^{(2)}\right)/\tau\right)}{\sum_{\substack{k=1 \\ k\neq n}}^{N} \left[\exp\left(\text{sim}\left(\boldsymbol{z}_n^{(1)}, \boldsymbol{z}_k^{(1)}\right)/\tau\right) + \exp\left(\text{sim}\left(\boldsymbol{z}_n^{(1)}, \boldsymbol{z}_k^{(2)}\right)/\tau\right)\right] + \exp\left(\text{sim}\left(\boldsymbol{z}_n^{(1)}, \boldsymbol{z}_n^{(2)}\right)/\tau\right)}, \tag{101}$$

where $\tau \in \mathbb{R}^+$ is a temperature parameter that controls the sharpness of the distribution over similarities. A symmetric loss $\ell_n^{(2)}$ is computed analogously by taking view 2 as the

anchor. The full training objective over the minibatch is:

$$\ell(\theta; \mathcal{B}) \;=\; \frac{1}{2N} \sum_{n=1}^{N} \left[\ell_n^{(1)} + \ell_n^{(2)} \right], \tag{102}$$

where θ denotes all learnable parameters in $f(\cdot)$ and $g(\cdot)$, and $\mathcal{B}$ denotes the minibatch.

Interpreting the Loss The NT-Xent loss has the form of a softmax-based cross-entropy, which we encountered earlier in the context of classification (Section 2). Here, however, the 'classes' are not semantic labels but rather the identities of the data points. The numerator encourages the two views of the same data point to have similar representations, while the denominator—which sums over all negative pairs—pushes the representations of different data points apart. This can be thought of as a form of *instance discrimination*: each data point is treated as its own class, and the model must learn to identify which views belong together.

The temperature τ plays a role similar to that of temperature scaling in calibration (Section **??**). As $\tau \to 0^+$, the softmax sharpens and the loss focuses only on the hardest negatives. As $\tau \to \infty$, all negatives contribute equally. In practice, τ is typically set to a small value (e.g. $\tau = 0.07$ or $\tau = 0.5$) and can have a significant impact on the quality of the learned representations.

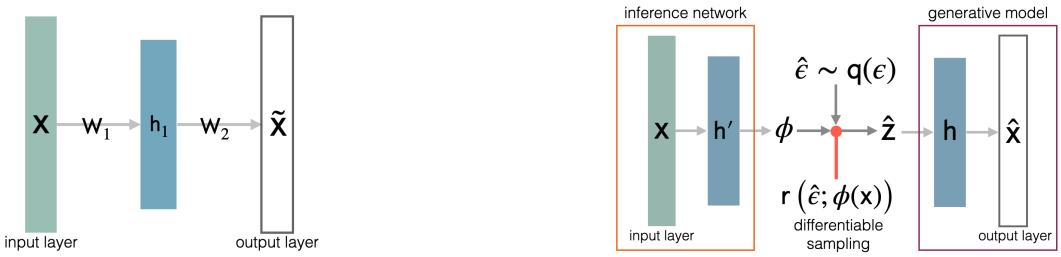
Connection to Autoencoders It is instructive to compare the AE and SimCLR side by side. Both learn an encoder $f(\cdot)$ that maps observations $\mathbf{x}$ to a representation $\mathbf{h}$. The key difference lies in the training signal used to shape this representation:

- **Autoencoder**: The loss is $\ell_{\text{AE}} \propto \|\boldsymbol{x} - \tilde{\boldsymbol{x}}\|^2$ (or the negative log-likelihood of the appropriate distribution), where $\tilde{\boldsymbol{x}}$ is the output of a *decoder* applied to $\boldsymbol{h}$. The representation is shaped by the requirement that it contain sufficient information to reconstruct the input.

- **SimCLR**: The loss is the NT-Xent (Equation 101), which operates on *pairs of representations* $(\boldsymbol{z}^{(1)}, \boldsymbol{z}^{(2)})$. There is no decoder and no reconstruction. The representation is shaped by the requirement that it be invariant to the augmentations in $\mathcal{T}$ while remaining discriminative across data points.

One way to unify these perspectives is to observe that both methods define a notion of 'what information matters.' The AE does so implicitly through the bottleneck: information that survives the compression-decompression pipeline is deemed important. SimCLR does so explicitly through the augmentation policy: information that is shared across augmented views is deemed important, and information that varies across views is discarded. This explicit control is a practical advantage, as one can engineer $\mathcal{T}$ to encode domain-specific knowledge about which factors are semantically meaningful.

Evaluation Since SimCLR is trained without labels, the quality of the learned representations is typically evaluated via a *linear probe*: after training $f(\cdot)$ with the contrastive objective, one freezes the encoder parameters and trains a linear classifier $\hat{\text{y}} = \texttt{softmax}(\mathbf{W}^T f(\boldsymbol{x}))$ on a labeled data set. The accuracy of this linear classifier serves as a proxy for the quality



(a) Autoencoder

(b) Variational Autoencoder

Figure 19: Autoencoder diagrams.

of f's representations. Remarkably, representations learned by SimCLR (and its successors) via this self-supervised procedure can rival those of fully supervised models, especially when labeled data is scarce but unlabeled data is abundant.

9.4 The Variational Autoencoder: a Probabilistic Autoencoder for Generative Modeling

A direct probabilistic interpretation of the AE can be had by thinking of AE-like architectures as latent variable models. The earliest work in this direction is the *density network* (MacKay and Gibbs, 1999), which defines a latent variable $\mathbf{z}$ and assumes the data is generated by a NN-parameterized conditional distribution:

$$\mathbf{x} \sim p\left(\mathbf{x}|\mathbf{z}\right), \quad \mathbf{z} \sim p\left(\mathbf{z}\right), \quad \mathbb{E}[\mathbf{x}|\mathbf{z}] = \mathbf{g}^{-1}(\mathbf{W}_L^T \boldsymbol{h}_{L-1}), \quad \boldsymbol{h}_0 = \mathbf{z}, \tag{103}$$

where $p\left(\mathbf{z}\right)$ denotes a user-specified prior distribution on the latent variable. A NN with L layers of parameters $\mathbf{W} = \{\mathbf{W}_1, \ldots, \mathbf{W}_L\}$ takes as input $\mathbf{z}$ and outputs the mean of the conditional distribution.

Kingma and Welling (2014) and Rezende et al. (2014) noticed that an end-to-end-differentiable neural architecture could be used to perform inference for the latent variables. This insight leads to a unified model known as the *variational autoencoder* (VAE). The core idea is to define an *inference network* (or *encoder*) to form a posterior approximation:

$$q\left(\mathbf{z}; \boldsymbol{\psi}(\boldsymbol{x})\right) \approx p(\mathbf{z}|\mathbf{x}), \quad \boldsymbol{\psi}(\boldsymbol{x}) = \mathbf{g}^{-1}(\mathbf{U}_L^T \boldsymbol{h}'_{L-1}), \quad \boldsymbol{h}'_0 = \boldsymbol{x} \tag{104}$$

where $\boldsymbol{\psi}(\boldsymbol{x})$ are the parameters of the posterior approximation (as a function of a given $\boldsymbol{x}$) and $\mathbf{U}_1, \ldots, \mathbf{U}_L$ are the parameters of the inference NN. Both networks can be trained

simultaneously using a reparameterized stochastic *evidence lower bound* (ELBO):

$$\begin{aligned}
\log p(\mathbf{x}) &= \log \int_{\mathbf{z}} p(\mathbf{x}|\mathbf{z}) \cdot p(\mathbf{z}) \; d\mathbf{z} \\
&= \log \int_{\mathbf{z}} \frac{q(\mathbf{z};\boldsymbol{\psi}(\boldsymbol{x})}{q(\mathbf{z};\boldsymbol{\psi}(\boldsymbol{x})} \cdot p(\mathbf{x}|\mathbf{z}) \cdot p(\mathbf{z}) \; d\mathbf{z} \\
&\geq \mathbb{E}_{q(\mathbf{z};\boldsymbol{\psi}(\boldsymbol{x}))} \left[\log p\left(\mathbf{x}|\mathbf{z}\right)\right] - \mathrm{KLD}\left[q(\mathbf{z};\boldsymbol{\psi}(\boldsymbol{x}))||p(\mathbf{z})\right] \\
&= \mathbb{E}_{q(\boldsymbol{\epsilon})} \left[\log p\left(\mathbf{x}|\mathbf{r}(\boldsymbol{\epsilon};\boldsymbol{\psi}(\boldsymbol{x}))\right)\right] - \mathrm{KLD}\left[q(\mathbf{z};\boldsymbol{\psi}(\boldsymbol{x}))||p(\mathbf{z})\right] \\
&\approx \frac{1}{S}\sum_{s=1}^{S} \log p\left(\mathbf{x}|\mathbf{r}(\hat{\boldsymbol{\epsilon}}_s;\boldsymbol{\psi}(\boldsymbol{x}))\right) - \mathrm{KLD}\left[q(\mathbf{z};\boldsymbol{\psi}(\boldsymbol{x}))||p(\mathbf{z})\right],
\end{aligned} \tag{105}$$

where s indexes the samples in the Monte Carlo expectation and $\mathrm{KLD}\left[q(\mathbf{z};\boldsymbol{\psi}(\boldsymbol{x}))||p(\mathbf{z})\right]$ denotes the Kullback-Leibler divergence between the approximate posterior and the prior. Most crucially, $\mathbf{r}(\boldsymbol{\epsilon};\boldsymbol{\psi}(\boldsymbol{x}))$ represents a reparameterization that allows us to draw samples from $q(\mathbf{z};\boldsymbol{\psi}(\boldsymbol{x}))$ via a fixed distribution $q(\boldsymbol{\epsilon})$. One example of such a function is the location-scale form for Normals: $\hat{\boldsymbol{z}} = \mathbf{r}(\hat{\boldsymbol{\epsilon}};\boldsymbol{\mu}_\psi(\boldsymbol{x}),\boldsymbol{\sigma}_\psi(\boldsymbol{x})) = \boldsymbol{\mu}_\psi(\boldsymbol{x}) + \boldsymbol{\sigma}_\psi(\boldsymbol{x}) \odot \hat{\boldsymbol{\epsilon}}$ where $\hat{\boldsymbol{\epsilon}} \sim \mathrm{N}(\mathbf{0}, \mathbb{I})$. Another example would be inverse transform sampling using $q(\mathbf{z})$'s CDF. Representing the stochastic variable $\mathbf{z}$ in this way allows for end-to-end differentiation as we now have access to the partials w.r.t. the inference network's parameters: $\partial\hat{\boldsymbol{z}}/\partial\mathbf{U}_l = (\partial\hat{\boldsymbol{z}}/\partial\boldsymbol{\psi})(\partial\boldsymbol{\psi}/\partial\boldsymbol{h}'_L)\ldots(\partial\boldsymbol{h}'_l/\partial\mathbf{U}_l)$. Figure 19b shows a diagram of the VAE, with the inference and generative networks being composed via $\mathbf{r}(\boldsymbol{\epsilon};\boldsymbol{\phi}(\boldsymbol{x}))$. When the inference and generative processes are thought of as a unified computational pipeline, the resulting structure resembles a traditional AE, which is how the VAE got its name. The VAE was one of the first modern generative models that showed a compelling ability to generate high-fidelity samples, as was demonstrated in Figure 18a.

9.5 Denoising Diffusion Models

Denoising diffusion models (**??**) are a class of latent variable generative models that have achieved remarkable sample quality, rivaling and often surpassing GANs for image generation. The core idea is conceptually simple: we define a *forward process* that gradually destroys structure in the data by adding noise over many steps until the data is indistinguishable from pure noise, and then we train a neural network to learn a *reverse process* that inverts this corruption, recovering data from noise step by step. Once the reverse process is learned, we can generate new samples by starting from pure noise and iteratively denoising.

Like the VAE of Section 9.4, diffusion models are latent variable models and can be motivated via a variational bound on the log-likelihood. However, unlike the VAE, which uses a single stochastic layer to map between data and latents, diffusion models use a long chain of intermediate latent variables $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T$, each of the same dimensionality as the data $\mathbf{x}_0$. The index $t \in \{0, 1, \ldots, T\}$ here denotes the step in the diffusion process, not a position in a temporal sequence as it did in the RNN sections. We will use $\mathbf{x}_0$ to denote a data point (corresponding to what we previously called $\mathbf{x}$) and $\mathbf{x}_T$ to denote the fully noised version.

Forward (Noising) Process The forward process defines a fixed (non-learned) Markov chain that progressively adds Gaussian noise to the data. Starting from a data sample

$\boldsymbol{x}_0 \sim \mathbb{P}(\mathbf{x})$, the forward process generates a sequence of increasingly noisy versions:

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) \;=\; \prod_{t=1}^{T} q(\mathbf{x}_t|\mathbf{x}_{t-1}), \tag{106}$$

where each step adds a small amount of Gaussian noise according to a variance schedule $\beta_1, \ldots, \beta_T$:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) \;=\; \mathrm{N}\left(\mathbf{x}_t;\; \boldsymbol{\mu} = \sqrt{1-\beta_t}\;\mathbf{x}_{t-1},\; \sigma^2 = \beta_t \mathbb{I}\right). \tag{107}$$

The hyperparameters $\beta_t \in (0,1)$ control the noise level at each step. Common choices include a linear schedule that increases β_t from a small value (e.g. $\beta_1 = 10^{-4}$) to a larger value (e.g. $\beta_T = 0.02$) over T steps, where T is typically on the order of 1000. The factor $\sqrt{1-\beta_t}$ multiplying $\mathbf{x}_{t-1}$ gently shrinks the signal at each step, ensuring that, as $T \to \infty$, the distribution $q(\mathbf{x}_T)$ converges to the standard normal $\mathrm{N}(\mathbf{0}, \mathbb{I})$ regardless of the data distribution.

A particularly useful property of this Gaussian forward process is that we can sample $\mathbf{x}_t$ at any arbitrary step t directly from $\mathbf{x}_0$, without iterating through the intermediate steps. Defining $\alpha_t \triangleq 1 - \beta_t$ and $\bar{\alpha}_t \triangleq \prod_{s=1}^{t} \alpha_s$, one can show by induction (repeatedly applying the reparameterization trick and combining Gaussian random variables) that:

$$q(\mathbf{x}_t|\mathbf{x}_0) \;=\; \mathrm{N}\left(\mathbf{x}_t;\; \boldsymbol{\mu} = \sqrt{\bar{\alpha}_t}\;\mathbf{x}_0,\; \sigma^2 = (1-\bar{\alpha}_t)\mathbb{I}\right). \tag{108}$$

Equivalently, using a reparameterization of the form introduced in the VAE section, we can write:

$$\mathbf{x}_t \;=\; \sqrt{\bar{\alpha}_t}\;\mathbf{x}_0 \;+\; \sqrt{1-\bar{\alpha}_t}\;\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathrm{N}(\mathbf{0}, \mathbb{I}).$$

This is an important identity for training, as it allows us to jump directly from a clean data point $\mathbf{x}_0$ to any noisy version $\mathbf{x}_t$ in a single step. Intuitively, as t increases, $\bar{\alpha}_t \to 0$, and the data signal $\mathbf{x}_0$ is increasingly attenuated while the noise contribution grows, until at $t = T$ the sample is approximately pure Gaussian noise.

Reverse (Denoising) Process If we could reverse the forward process, then we could generate data by starting from noise $\mathbf{x}_T \sim \mathrm{N}(\mathbf{0}, \mathbb{I})$ and iteratively denoising. The true reverse conditional $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is intractable in general because it depends on the entire data distribution. However, a key observation from **?** is that when the noise added at each step is small (i.e. β_t is small), the reverse conditional $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is also approximately Gaussian. This motivates parameterizing the reverse process as a Gaussian whose parameters are learned by a neural network:

$$p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) \;=\; \mathrm{N}\left(\mathbf{x}_{t-1};\; \boldsymbol{\mu} = \boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}_t, t),\; \sigma^2 = \sigma_t^2 \mathbb{I}\right), \tag{109}$$

where $\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)$ is a neural network that takes the current noisy sample $\mathbf{x}_t$ and the time step t as inputs and outputs the predicted mean of the denoised sample. The variance σ_t^2 can either be fixed (e.g. set to β_t or to $\tilde{\beta}_t$, defined below) or learned; **?** found that fixing it

works well in practice. The full reverse process is then:

$$p_{\boldsymbol{\theta}}(\mathbf{x}_{0:T}) \;=\; p(\mathbf{x}_T)\prod_{t=1}^{T} p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t), \qquad p(\mathbf{x}_T) = \mathrm{N}(\mathbf{x}_T; \mathbf{0}, \mathbb{I}). \tag{110}$$

Note that both the forward and reverse processes are Markov chains of the same length T, but the forward process runs from $t = 0$ to $t = T$ (adding noise), while the reverse runs from $t = T$ to $t = 0$ (removing noise). The neural network $\boldsymbol{\mu}_{\boldsymbol{\theta}}$ is shared across all time steps, but it receives t as an input so that it can adapt its behavior to the noise level at each step.

Training Objective: Variational Lower Bound As with the VAE, we can derive a training objective by writing a variational lower bound on the log-likelihood $\log p_{\boldsymbol{\theta}}(\mathbf{x}_0)$. The latent variables in this model are $\mathbf{x}_{1:T}$, and the approximate posterior is the forward process $q(\mathbf{x}_{1:T}|\mathbf{x}_0)$. Applying the standard variational bound:

$$\log p_{\boldsymbol{\theta}}(\mathbf{x}_0) \;\geq\; \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\left[\log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\right] \;\triangleq\; \mathcal{L}(\boldsymbol{\theta}; \mathbf{x}_0). \tag{111}$$

This bound can be decomposed, after some algebra, into a sum of KL divergence terms that compare the reverse process to the tractable *forward posteriors*. The forward posterior $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ is the distribution of $\mathbf{x}_{t-1}$ given both the current noisy version $\mathbf{x}_t$ and the original data $\mathbf{x}_0$. It can be computed in closed form by applying Bayes' rule to the Gaussian forward process:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \;=\; \mathrm{N}\left(\mathbf{x}_{t-1};\; \boldsymbol{\mu} = \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0),\; \sigma^2 = \tilde{\beta}_t \mathbb{I}\right), \tag{112}$$

where the mean and variance are given by:

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) \;=\; \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\,\mathbf{x}_0 \;+\; \frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\,\mathbf{x}_t, \qquad \tilde{\beta}_t \;=\; \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\cdot\beta_t.$$

Using these forward posteriors, the variational bound decomposes as:

$$\begin{aligned} -\mathcal{L}(\boldsymbol{\theta}; \mathbf{x}_0) \;=\; & \underbrace{\mathrm{KLD}\left[q(\mathbf{x}_T|\mathbf{x}_0) \;||\; p(\mathbf{x}_T)\right]}_{L_T} \;+\; \sum_{t=2}^{T} \underbrace{\mathrm{KLD}\left[q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \;||\; p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)\right]}_{L_{t-1}} \\ & \underbrace{-\,\mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)}\left[\log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1)\right]}_{L_0}. \end{aligned} \tag{113}$$

Let us interpret each term. The first term L_T compares the endpoint of the forward process to the prior $p(\mathbf{x}_T) = \mathrm{N}(\mathbf{0}, \mathbb{I})$; since the forward process has no learnable parameters, this term is a constant and can be ignored during optimization. The final term L_0 is a reconstruction term that measures how well the model can recover the data from the first latent $\mathbf{x}_1$; it is analogous to the reconstruction term in the VAE's ELBO. The middle terms $L_1, \ldots, L_{T-1}$ are the most important: each one is a KL divergence between two Gaussians, asking the learned reverse step $p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)$ to match the forward posterior $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$. Because both distributions are Gaussian, these KL divergences can be computed in closed form.

Noise Prediction and the Simplified Objective While one could directly train the network $\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)$ to predict the mean of the reverse step, **?** proposed a reparameterization that leads to a simpler and more effective training objective. Using the forward process identity $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\ \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\ \boldsymbol{\epsilon}$, we can substitute for $\mathbf{x}_0$ in the expression for $\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0)$ to obtain:

$$\tilde{\boldsymbol{\mu}}_t \ = \ \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t \ - \ \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\ \boldsymbol{\epsilon} \right).$$

This suggests parameterizing the reverse mean as:

$$\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) \ = \ \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t \ - \ \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\ \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) \right), \tag{114}$$

where $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)$ is a neural network that predicts the noise $\boldsymbol{\epsilon}$ that was added to $\mathbf{x}_0$ to produce $\mathbf{x}_t$. In other words, instead of predicting the denoised mean directly, the network predicts *the noise itself*. Substituting this parameterization into the KL divergence terms and dropping terms that do not depend on $\boldsymbol{\theta}$, the variational bound simplifies (up to a weighting across time steps) to:

$$\ell_{\text{simple}}(\boldsymbol{\theta}; \mathbf{x}_0) \ = \ \mathbb{E}_{t \sim \text{Uniform}\{1,\ldots,T\}}\ \mathbb{E}_{\boldsymbol{\epsilon} \sim \text{N}(\mathbf{0}, \mathbb{I})} \left[\ \left\| \boldsymbol{\epsilon} \ - \ \boldsymbol{\epsilon}_{\boldsymbol{\theta}} \left(\sqrt{\bar{\alpha}_t}\ \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\ \boldsymbol{\epsilon},\ t \right) \right\|^2 \right]. \tag{115}$$

This objective is remarkably simple: sample a random time step t, sample random noise $\boldsymbol{\epsilon}$, construct the noisy input $\mathbf{x}_t$ via Equation 108, and train $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}$ to predict $\boldsymbol{\epsilon}$ from $\mathbf{x}_t$ using the squared error loss. **?** found that this simplified objective, which drops the time-step-dependent weighting that arises from the full variational bound, performs better in practice than the weighted version. The training procedure over the full data set is then:

$$\boldsymbol{\theta}^* \ = \ \arg\min_{\boldsymbol{\theta}} \ \frac{1}{N} \sum_{n=1}^{N} \ell_{\text{simple}}(\boldsymbol{\theta}; \boldsymbol{x}_{0,n}).$$

Neural Network Architecture The noise prediction network $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)$ is typically implemented as a *U-Net* (**?**), a convolutional architecture that was originally designed for image segmentation. The U-Net has an encoder-decoder structure with skip connections between corresponding layers, which allows it to capture both coarse and fine-grained spatial information. This is important for denoising, as the network must remove noise at multiple spatial scales. The time step t is incorporated via a sinusoidal embedding (similar to the positional encodings used in transformers) that is injected into each layer of the U-Net. Self-attention layers are also commonly included at lower spatial resolutions to capture long-range dependencies. Notably, the network takes as input $\mathbf{x}_t$, which has the same dimensionality as the original data $\mathbf{x}_0$, and outputs a prediction of $\boldsymbol{\epsilon}$ that is also of the same dimensionality.

Sampling Once the model is trained, generating a new sample proceeds by first drawing $\hat{\boldsymbol{x}}_T \sim \text{N}(\mathbf{0}, \mathbb{I})$ and then iteratively applying the learned reverse process. At each step $t = T, T-1, \ldots, 1$:

$$\hat{\boldsymbol{x}}_{t-1} \ = \ \frac{1}{\sqrt{\alpha_t}} \left(\hat{\boldsymbol{x}}_t \ - \ \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\ \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\hat{\boldsymbol{x}}_t, t) \right) \ + \ \sigma_t\ \boldsymbol{z}, \quad \boldsymbol{z} \sim \text{N}(\mathbf{0}, \mathbb{I}),$$

where σ_t is a variance hyperparameter (typically set to $\sqrt{\beta_t}$ or $\sqrt{\tilde{\beta}_t}$) and the noise $\boldsymbol{z}$ is omitted at the final step $t = 1$. The generation process thus requires T forward passes through the neural network, one for each denoising step. This is considerably more expensive than sampling from a VAE or GAN (which require a single forward pass) and is one of the main practical drawbacks of diffusion models. Various techniques have been proposed to accelerate sampling, including using fewer steps with non-Markovian diffusion processes (**?**) and distilling the multi-step process into fewer steps.

Connection to Score Matching An alternative perspective on diffusion models comes from the framework of *score-based generative models* (**??**). The *score function* of a distribution $p(\mathbf{x})$ is defined as $\nabla_{\mathbf{x}} \log p(\mathbf{x})$—the gradient of the log-density with respect to the data. One can show that the noise prediction network $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)$ is closely related to the score of the noisy data distribution at time t:

$$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) \;\approx\; -\frac{1}{\sqrt{1-\bar{\alpha}_t}}\, \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t).$$

This means that training a diffusion model to predict the noise is essentially equivalent to performing *denoising score matching* (Vincent, 2011)—learning the score function at multiple noise levels. **?** unified the DDPM and score-based perspectives under a common framework of stochastic differential equations (SDEs), showing that the forward process can be described by a continuous-time SDE and the reverse process by the corresponding reverse-time SDE, with the score function determining the drift of the reverse process. This continuous-time view also provides a connection to the theory of *normalizing flows*, as the reverse SDE has an equivalent ordinary differential equation (ODE) formulation—called the *probability flow ODE*—that deterministically maps noise to data.

Summary Denoising diffusion models provide an elegant framework for generative modeling. They share the variational training perspective of the VAE: both are latent variable models trained via a bound on the log-likelihood. However, diffusion models differ in that their latent space has the same dimensionality as the data and is structured as a long Markov chain, and the approximate posterior (the forward process) is fixed rather than learned. This design results in a very simple training objective—predicting the noise added to data—that is stable to optimize and scales well to high-resolution generation tasks. The primary trade-off is that sampling requires iterating through many denoising steps, making generation slower than single-pass models like VAEs and GANs.

9.6 Other Types of Neural Generative Models

A variety of other deep generative models have been developed, and we briefly outline them here. One of the most popular is the *generative adversarial network* (GAN) (Goodfellow et al., 2014). GANs re-formulate the task of density modeling into an adversarial game in which a 'generator' NN tries to simulate data so that a 'discriminator' NN cannot tell the difference between the generated and observed samples. The assumption is that if the discriminator cannot tell the two apart, then the generator must be a good model of the data. The concept is similar in spirit to *approximate Bayesian computation* (ABC) (Rubin,

1984), which compares simulated data to the observations via some statistic or metric and retains the parameters that generated the simulation—treating them as a posterior sample—if the statistic is within some threshold. In GANs, the discriminator serves as the metric comparing the 'fake' and real data. The major difference between ABC and GANs is that GANs are trained by differentiating through the adversarial process, treating it as an optimization objective. Mohamed and Lakshminarayanan (2016) discuss GANs from a generalized framework, showing various proper scoring rules resulting in valid discriminators. The GAN framework can also be used for approximate inference for model parameters (Tran et al., 2017; Mescheder et al., 2017), although using GANs for inference is made difficult by their inability to provide a density estimate.

Another type of neural generative model is the *normalzing flow* (NF) (Tabak and Turner, 2013; Rezende and Mohamed, 2015; Papamakarios et al., 2021). These models use NNs to reparameterize a simple distriubtion into one with richer complexity. Specificially, the data density $p(\boldsymbol{x})$ is modeled as: $p(\mathbf{x};\boldsymbol{\psi}) = p_{\mathrm{z}}\left(T_{\boldsymbol{\psi}}^{-1}(\mathbf{x})\right)\left|\partial\, T_{\boldsymbol{\psi}}^{-1}/\partial\mathbf{x}\right|$ where $p_{\mathrm{z}}(\mathbf{z})$ is the simple base distribution that is being reparameterized via the NN function $T_{\boldsymbol{\psi}}$, and where $\boldsymbol{\psi}$ are the parameters (weights) of the NN. After performing maximum likelihood estimation for $\boldsymbol{\psi}$, samples can be drawn via $\hat{\boldsymbol{z}} \sim p(\mathbf{z})$, $\hat{\boldsymbol{x}} = T_{\boldsymbol{\psi}}(\hat{\boldsymbol{z}})$. The NNs are carefully designed so that the volume element $\left|\partial\, T_{\boldsymbol{\psi}}^{-1}\partial\mathbf{x}\right|$ is easy to compute (i.e. doesn't require computing an arbitrary Jacobian determinant). For instance, autoregressive flows allow for a triangular Jacobian matrix whose determinant is just the product of the diagonal terms (Kingma et al., 2016; Papamakarios et al., 2017; Huang et al., 2018). The images shown in Figure 18b were generated by a particular NF model known as a *Glow* (Kingma and Dhariwal, 2018). NFs have the added benefit that their density function can usually be evaluated quickly, as is necessary for model fitting.

Bibliography

Pierre Baldi and Kurt Hornik. Neural networks and principal component analysis: learning from examples without local minima. *Neural Networks*, 2(1):53–58, 1989.

Hervé Bourlard and Yves Kamp. Auto-association by multilayer perceptrons and singular value decomposition. *Biological Cybernetics*, 59(4-5):291–294, 1988.

Garrison W. Cottrell. Image compression by back propagation: a demonstration of extensional programming. *Models of Cognition*, 3:208–240, 1989.

Carl Doersch. Tutorial on variational autoencoders. *arXiv preprint arXiv:1606.05908*, 2016.

Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics*, pages 249–256, 2010.

Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K.Q. Weinberger, editors, *Advances in Neural Information Processing Systems 27 (NIPS 2014)*, pages 2672—-2680, Red Hook, NY, 2014. Curran.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

Geoffrey E. Hinton and Ruslan R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006.

Chin-Wei Huang, David Krueger, Alexandre Lacoste, and Aaron Courville. Neural autoregressive flows. *PMLR*, 80:2078–2087, 2018.

Amirhossein Kazemnejad. Transformer architecture: The positional encoding. *kazemnejad.com*, 2019. URL `https://kazemnejad.com/blog/transformer_architecture_positional_encoding/`.

Diederik Kingma and Prafulla Dhariwal. Glow: generative flow with invertible 1x1 convolutions. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, pages 10236—-10245, Red Hook, NY, 2018. Curran.

Diederik Kingma and Max Welling. Auto-encoding variational Bayes. *International Conference on Learning Representations*, 2014.

Diederik Kingma, Tim Salimans, Rafal Jozefowicz, Xi Chen, Ilya Sutskever, and Max Welling. Improved variational inference with inverse autoregressive flow. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, editors, *Advances in Neural Information Processing Systems 29 (NIPS 2016)*, pages 4743—-4751, Red Hook, NY, 2016. Curran.

Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv Preprint arXiv:1412.6980*, 2014.

David J.C. MacKay and Mark N. Gibbs. Density networks. In Jim W Kay and D Mike Titterington, editors, *Statistics and Neural Networks: Advances at the Interface*, pages 129–146. Oxford University Press, Oxford, 1999.

Lars M. Mescheder, Sebastian Nowozin, and Andreas Geiger. Adversarial variational Bayes: unifying variational autoencoders and generative adversarial networks. *PMLR*, 70:2391—-2400, 2017.

Shakir Mohamed and Balaji Lakshminarayanan. Learning in implicit generative models. *arXiv:1610.03483*, 2016.

George Papamakarios, Theo Pavlakou, and Iain Murray. Masked autoregressive flow for density estimation. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, Red Hook, NY, 2017. Curran.

George Papamakarios, Eric Nalisnick, Danilo Jimenez Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing flows for probabilistic modeling and inference. *Journal of Machine Learning Research*, 22(57):1–64, 2021. URL `http://jmlr.org/papers/v22/19-1028.html`.

Danilo Rezende and Shakir Mohamed. Variational inference with normalizing flows. *PMLR*, 37:1530–1538, 2015.

Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic backpropagation and approximate inference in deep generative models. *PMLR*, 32:1278—-1286, 2014.

Donald B. Rubin. Bayesianly justifiable and relevant frequency calculations for the applied statistician. *The Annals of Statistics*, 12(4):1151–1172, 1984.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

Esteban G. Tabak and Cristina V. Turner. A family of nonparametric density estimation algorithms. *Communications on Pure and Applied Mathematics*, 66(2):145–164, 2013.

Matus Telgarsky. Benefits of depth in neural networks. In *Conference on Learning Theory*, pages 1517–1539. PMLR, 2016.

Dustin Tran, Rajesh Ranganath, and David M. Blei. Hierarchical implicit models and likelihood-free variational inference. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, pages 5529—5539, Red Hook, NY, 2017. Curran.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is All You Need. *Advances in Neural Information Processing Systems*, 30, 2017.

Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural Computation*, 23(7):1661–1674, 2011.

Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In William Cohen, Sam Roweis, and Andrew McCallum, editors, *Proceedings of the 25th International Conference on Machine Learning*, pages 1096–1103, New York, NY, 2008. ACM Press.

Pascal Vincent, Hugo Larochelle, Isabelle Lajoie, Yoshua Bengio, and Pierre-Antoine Manzagol. Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion. *Journal of Machine Learning Research*, 11(Dec):3371–3408, 2010.